Đại học Phenikaa
Khoa Hệ thống thông tin

AN TOÀN PHẦN MỀM

Software Security

Giáo trình học phần CSE703093

Bốn trụ cột của học phần

- Ch.1 – Khái niệm và nền tảng an toàn phần mềm
- Ch.2 – Lập trình C an toàn: Quản lý bộ nhớ
- Ch.3 – Phát hiện lỗ hổng: Phân tích tĩnh
- Ch.4 – Phát hiện lỗ hổng: Phân tích động

Biên soạn:
ThS. Vũ Quang Dũng
với sự tham gia của TS. Trịnh Thanh Bình, TS. Đoàn Trung Sơn

Hà Nội, 2026

Lời nói đầu

Phần mềm ngày nay là hạ tầng vận hành của gần như mọi lĩnh vực trọng yếu: ngân hàng, y tế, giao thông, năng lượng, quốc phòng, và hàng tỷ thiết bị Internet of Things. Cùng với sự phổ biến đó, phần mềm cũng trở thành bề mặt tấn công (*attack surface*) lớn nhất mà xã hội số phải đối mặt. Theo thống kê liên tục được công bố trong cơ sở dữ liệu CVE (Common Vulnerabilities and Exposures) và CWE (Common Weakness Enumeration), phần lớn sự cố an ninh mạng nghiêm trọng — từ Heartbleed (2014), EternalBlue/WannaCry (2017), Log4Shell (2021) cho đến các vụ rò rỉ dữ liệu quy mô lớn — đều bắt nguồn từ những lỗi lập trình có thể được phát hiện sớm nếu áp dụng đúng phương pháp, kỹ thuật và công cụ kiểm chứng phần mềm.

Giáo trình này được biên soạn cho học phần **CSE703093 – An toàn phần mềm** (Software Security), thuộc chương trình đào tạo ngành Công nghệ thông tin / Hệ thống thông tin, Trường Đại học Phenikaa. Nội dung giáo trình bám sát đề cương chi tiết học phần đã được phê duyệt, đồng thời mở rộng theo chiều sâu lý thuyết và chiều rộng thực hành nhằm cung cấp một tài liệu tự học đầy đủ, có thể sử dụng độc lập với slide bài giảng.

Cấu trúc giáo trình

Giáo trình gồm bốn chương chính, được tổ chức theo một mạch tư duy xuyên suốt: **hiểu vấn đề** → **viết mã an toàn** → **kiểm chứng bằng phân tích tĩnh** → **kiểm chứng bằng phân tích động**.

- **Chương 1 – Giới thiệu về an toàn phần mềm.** Xây dựng nền tảng khái niệm: mô hình CIA, các thuộc tính mở rộng (AAA), khung phân loại STRIDE/CWE/CVE, các lớp lỗ hổng phổ biến, và đặc biệt là hai công cụ hình thức xuyên suốt cả giáo trình: *Kripke Structure* (biểu diễn hành vi hệ thống dưới dạng đồ thị trạng thái) và các kỹ thuật thiết kế test case cổ điển theo chuẩn ISTQB.
- **Chương 2 – Lập trình C an toàn: Quản lý bộ nhớ.** Đi sâu vào ngôn ngữ C – nơi phần lớn lỗi bộ nhớ trong thực tế xảy ra (hệ điều hành, trình biên dịch, firmware, hệ thống nhúng). Chương trình bày đánh giá rủi ro, cấu trúc dữ liệu động, các nguyên tắc lập trình an toàn, căn chỉnh bộ nhớ (alignment), và cách sử dụng công cụ phân tích tĩnh thực tế (cppcheck, clang-tidy, valgrind).
- **Chương 3 – Phát hiện lỗ hổng phần mềm: Phân tích tĩnh.** Xây dựng nền tảng lý thuyết của phân tích tĩnh: kiểm chứng và thẩm định phần mềm, soundness/completeness, SAT/SMT solving (DPLL, CDCL, DPLL(T)), và đỉnh cao là Bounded Model Checking – kỹ thuật kết hợp Kripke Structure của Chương 1 với SAT/SMT solving để kiểm chứng chương trình một cách tự động.

- **Chương 4 – Phát hiện lỗ hổng phần mềm: Phân tích động.** Chuyển hướng sang kiểm thử bảo mật bằng cách thực sự thực thi chương trình: code coverage, theo dõi thực thi (execution tracing), taint tracking, black-box fuzzing và white-box fuzzing (kết hợp lại SMT solving của Chương 3 để giải quyết ”magic byte problem”).

Mỗi chương được thiết kế với cấu trúc thống nhất: mục tiêu học tập, nội dung lý thuyết có ví dụ minh họa, sơ đồ/kiến trúc/lưu đồ, mã nguồn thực thi được, case study thực tế, hộp ”Kiểm soát chất lượng” liên hệ tiêu chuẩn công nghiệp, hộp ”Xu hướng hiện đại” cập nhật ứng dụng AI/ML trong an toàn phần mềm, hoạt động trên lớp, bài tập tự luận và bài tập lập trình, và tổng kết chương.

Bộ công cụ đi kèm – `softsec-toolkit`

Song song với giáo trình, một bộ công cụ Python mã nguồn mở `softsec-toolkit` được phát triển xuyên suốt bốn chương, cho phép sinh viên thực hành trực tiếp các khái niệm lý thuyết: cài đặt Kripke Structure và kiểm tra reachability (Chương 1), quét mẫu lỗi bộ nhớ trên mã C (Chương 2), một SAT solver DPLL/CDCL thu nhỏ cùng bounded model checker (Chương 3), và một bộ theo dõi coverage cùng fuzzer đen/trắng đơn giản (Chương 4). Mã nguồn đầy đủ, có chú thích, và có bộ kiểm thử `pytest` đi kèm, được trình bày chi tiết trong Phụ lục A.

Đối tượng sử dụng

Giáo trình phù hợp cho sinh viên đại học ngành CNTT/HTTT đã có kiến thức nền về lập trình C/C++, cấu trúc dữ liệu và giải thuật, và kiến trúc máy tính cơ bản. Giáo trình cũng có thể dùng làm tài liệu tham khảo cho kỹ sư phần mềm, kỹ sư kiểm thử (QA/QC), và các nhà nghiên cứu bảo mật muốn hệ thống hóa lại nền tảng lý thuyết đằng sau các công cụ phân tích tĩnh/động họ sử dụng hàng ngày.

Hà Nội, 2026
ThS. Vũ Quang Dũng
Khoa Hệ thống thông tin, Trường Đại học Phenikaa

Mục lục

Lời nói đầu **i**

Danh sách hình vẽ **vi**

Danh sách bảng **vii**

1 Giới thiệu về an toàn phần mềm **1**
1.1 Vì sao an toàn phần mềm quan trọng 1
1.2 Các khái niệm nền tảng của an toàn phần mềm 2
1.2.1 Security, Safety và Reliability 2
1.2.2 Mô hình CIA 2
1.2.3 Asset, Threat, Vulnerability, Risk, Attack, Exploit 4
1.2.4 Đánh giá rủi ro: Risk = Likelihood × Impact 4
1.2.5 Cây tấn công (Attack Tree) 5
1.2.6 Authentication vs. Authorization; Least Privilege 6
1.3 Một số vấn đề về an toàn phần mềm 6
1.3.1 Khung phân loại CWE và cơ sở dữ liệu CVE 6
1.3.2 Các lớp lỗi phần mềm phổ biến 7
1.3.3 Khung phân tích mối đe dọa STRIDE 8
1.4 Kiểm thử và kiểm chứng phần mềm (Phần I) 9
1.4.1 Verification vs. Validation 9
1.4.2 Kiểm thử tĩnh và động (Static vs. Dynamic Testing) . . . 9
1.4.3 Mô hình chữ V (V-Model) 10
1.4.4 Kỹ thuật thiết kế test case hộp đen (Black-box) 10
1.5 Kiểm thử và kiểm chứng phần mềm (Phần II) 11
1.5.1 Các kỹ thuật thiết kế test case bổ sung 11
1.6 Cấu trúc Kripke (Kripke Structure) 12
1.6.1 Path, Reachability và Temporal Logic 13
1.7 OWASP Top 10 và CWE Top 25 15
1.7.1 OWASP Top 10 15
1.7.2 CWE Top 25 Most Dangerous Software Weaknesses . . . 17
1.8 Phân tích rủi ro định lượng 17
1.9 Mở rộng về Kripke Structure và Temporal Logic 18
1.9.1 CTL và các toán tử định lượng theo path 18
1.9.2 Thuật toán gán nhãn CTL (Labeling Algorithm) 18
1.10 Thực hành: Threat Modeling STRIDE đầy đủ 19
1.11 Thực hành từng bước: truy vết BFS trên Kripke Structure . . . 20
1.12 Tổng kết chương 21

2 Lập trình C an toàn: Quản lý bộ nhớ 23
2.1 Vì sao C vẫn được dùng rộng rãi dù có rủi ro cao 23
2.2 Đánh giá rủi ro (Risk Assessment) cho mã C 24
2.2.1 Attack Surface trong mã C 24
2.2.2 Mô hình bộ nhớ tiến trình (Process Memory Layout) . . . 25
2.2.3 Ma trận rủi ro cho mã C 26
2.3 Cấu trúc dữ liệu động (Dynamic Data Structures) 27
2.3.1 Con trỏ và Ownership 27
2.3.2 Linked List an toàn 27
2.4 Lập trình C an toàn (Secure C Coding) 28
2.4.1 Undefined Behavior (UB) 28
2.4.2 Các hàm chuỗi an toàn 30
2.5 Sắp xếp cấu trúc dữ liệu (Data Structure Alignment) 31
2.5.1 Vì sao CPU yêu cầu dữ liệu được căn chỉnh 31
2.5.2 Padding và ảnh hưởng bảo mật của Structure Layout . . 31
2.6 Công cụ phân tích tĩnh tìm lỗi bảo mật 32
2.6.1 Phân loại công cụ 33
2.6.2 Module `softsec.ch2_memsafe`: quét mẫu lỗi bằng Python . 34
2.7 Cơ chế cấp phát bộ nhớ động (Heap Allocator Internals) 35
2.7.1 Cấu trúc chunk trong glibc malloc 35
2.7.2 Các cơ chế giảm nhẹ ở tầng hệ điều hành/trình biên dịch 36
2.8 Lỗi định dạng chuỗi (Format String Vulnerabilities) 38
2.9 Tràn số nguyên (Integer Overflow) chuyên sâu 38
2.10 Cấu trúc dữ liệu động nâng cao: Hash Table và Cây 39
2.11 Case Study lịch sử: Morris Worm (1988) và `gets()` 40
2.12 Thực hành từng bước: đọc thông báo lỗi AddressSanitizer . . . 41
2.13 Tổng kết chương 42

3 Phát hiện lỗ hổng phần mềm: Phân tích tĩnh 44
3.1 Giới thiệu về kiểm chứng và thẩm định phần mềm 44
3.2 Soundness và Completeness 45
3.3 Sự khác biệt giữa phân tích tĩnh, kiểm thử và gỡ lỗi 46
3.4 SAT và SMT Solving 47
3.4.1 Bài toán SAT 47
3.4.2 Thuật toán DPLL 47
3.4.3 Từ SAT đến SMT 49
3.5 Kiểm chứng mô hình phần mềm (Bounded Model Checking) . . 50
3.5.1 Giới hạn của Model Checking đầy đủ 50
3.5.2 Ví dụ trải phẳng chương trình 50
3.6 Mô hình bộ nhớ (Memory Model) 51
3.6.1 Sắp xếp mô hình bộ nhớ (Aligned Memory Model) 52
3.7 Symbolic Model Checking bằng BDD 52
3.8 Trừu tượng hóa (Abstract Interpretation) 53
3.9 K-Induction: mở rộng BMC thành chứng minh không giới hạn . 54
3.10 Thực hành SMT solving nâng cao với Z3 54
3.11 Case Study: Kiểm chứng hình thức hạt nhân hệ điều hành seL4 55
3.12 Thực hành từng bước: giải DPLL bằng tay với truy vết đầy đủ . 56
3.13 Tổng kết chương 57

4 Phát hiện lỗ hổng phần mềm: Phân tích động 59
4.1 Giới thiệu về kiểm thử bảo mật . . . 59
4.1.1 Security Testing khác Functional Testing ở đâu? . . . 60
4.2 Phủ mã nguồn (Code Coverage) . . . 60
4.2.1 Các mức độ coverage . . . 60
4.2.2 Coverage-guided Fuzzing . . . 61
4.3 Theo dõi thực thi chương trình . . . 62
4.3.1 Execution Tracing và Taint Tracking . . . 62
4.3.2 Magic Byte Problem . . . 62
4.4 Tạo ra các đường thực thi của chương trình . . . 62
4.5 Kiểm thử hộp đen (Black-box Fuzzing) . . . 63
4.6 Kiểm thử hộp trắng (White-box Fuzzing) . . . 64
4.6.1 Symbolic Execution và Concolic Execution . . . 64
4.6.2 Giải quyết Magic Byte Problem bằng White-box Fuzzing 65
4.7 Sanitizer: cầu nối giữa Static và Dynamic Analysis . . . 66
4.7.1 AddressSanitizer, UndefinedBehaviorSanitizer, MemorySanitizer 66
4.8 Corpus Minimization và Crash Triage . . . 67
4.8.1 Tối thiểu hóa corpus . . . 67
4.8.2 Crash Triage và Deduplication . . . 67
4.9 Structure-aware Fuzzing (Kiểm thử mờ có cấu trúc) . . . 67
4.9.1 Vấn đề với input có cấu trúc phức tạp . . . 67
4.10 Differential Fuzzing . . . 68
4.11 Case Study: Syzkaller và Fuzzing Nhân Hệ điều hành . . . 69
4.12 Thực hành từng bước: một vòng lặp fuzzing hoàn chỉnh . . . 69
4.13 Tổng kết chương . . . 71

A Bộ công cụ `softsec-toolkit` 73
A.1 Module `ch1_models` – Chương 1 . . . 73
A.1.1 `kripke.py` . . . 73
A.1.2 `test_design.py` . . . 76
A.1.3 `risk.py` . . . 78
A.2 Module `ch2_memsafe` – Chương 2 . . . 79
A.2.1 `pattern_scanner.py` . . . 79
A.2.2 `memory_trace.py` . . . 82
A.2.3 `struct_layout.py` . . . 85
A.3 Module `ch3_staticsat` – Chương 3 . . . 87
A.3.1 `sat_solver.py` . . . 87
A.3.2 `smt_wrapper.py` . . . 91
A.3.3 `bounded_checker.py` . . . 94
A.4 Module `ch4_dynafuzz` – Chương 4 . . . 97
A.4.1 `coverage_tracker.py` . . . 97
A.4.2 `blackbox_fuzzer.py` . . . 99
A.4.3 `whitebox_fuzzer.py` . . . 102

B Gợi ý giải bài tập chọn lọc 108
B.1 Chương 1 . . . 108
B.2 Chương 2 . . . 108
B.3 Chương 3 . . . 108

B.4 Chương 4 . . . 108
B.5 Lời giải chi tiết bổ sung . . . 109
B.5.1 Chương 1 – Mở rộng `classify_risk` thành 5 mức . . . 109
B.5.2 Chương 2 – Mở rộng `DANGEROUS_FUNCTIONS` . . . 110
B.5.3 Chương 3 – Kiểm tra mảng với biểu thức chỉ số phức tạp 110
B.5.4 Chương 4 – Đo hiệu quả random mutation vs. SMT solving 111

C Đề thi mẫu và đáp án chi tiết 112
C.1 Đề thi mẫu Chương 1: Giới thiệu về an toàn phần mềm . . . 112
C.2 Đề thi mẫu Chương 2: Lập trình C an toàn . . . 114
C.3 Đề thi mẫu Chương 3: Phân tích tĩnh . . . 117
C.4 Đề thi mẫu Chương 4: Phân tích động . . . 119
C.5 Đề thi cuối kỳ tổng hợp (mẫu) . . . 121

D Hướng dẫn bài tập lớn (Capstone Project) 124
D.1 Mục tiêu . . . 124
D.2 Yêu cầu kỹ thuật . . . 125
D.3 Cấu trúc dự án đề xuất . . . 125
D.4 Lịch trình đề xuất (8 tuần) . . . 126
D.5 Ví dụ file mã nguồn mẫu cho kiểm thử pipeline . . . 126
D.6 Quy định nộp bài và bảo vệ . . . 127

E Bảng thuật ngữ Việt – Anh 128
E.1 Bảng đối chiếu nhanh . . . 128
E.2 Định nghĩa thuật ngữ cốt lõi (tra cứu nhanh) . . . 130

Tài liệu tham khảo 132

Danh sách hình vẽ

1.1 Mô hình CIA – ba thuộc tính cốt lõi của an toàn thông tin . . . 3
1.2 Quan hệ giữa các khái niệm nền tảng trong phân tích rủi ro . . 4
1.3 Ví dụ Attack Tree: chiếm quyền máy chủ . . . 5
1.4 Mô hình chữ V: mỗi giai đoạn phát triển tương ứng một cấp độ kiểm thử . . . 10
1.5 Kripke Structure cho hệ thống đăng nhập ba trạng thái . . . 13

2.1 Mô hình bố trí bộ nhớ tiến trình điển hình (kiến trúc x86/x86-64 Linux) . . . 25
2.2 Cấu trúc chunk trong glibc malloc (đơn giản hóa) . . . 35

3.1 Quy trình Bounded Model Checking . . . 50

4.1 Vòng lặp coverage-guided fuzzing . . . 61

4.2 Vòng lặp concolic execution giải quyết magic byte problem . . 65

Danh sách bảng

1.1 Phân biệt Security, Safety, Reliability 2
1.2 Các khái niệm nền tảng trong phân tích rủi ro bảo mật 4
1.3 Ma trận rủi ro (Risk Matrix) 3x3 5
1.4 Một số CWE phổ biến nhất và CVE minh họa 7
1.5 Khung STRIDE và thuộc tính bảo mật bị vi phạm tương ứng . . 9
1.6 Decision Table: quy tắc giảm giá 10% nếu là thành viên và đơn hàng $>$ 500k 12
1.7 Các toán tử LTL/CTL cơ bản 14
1.8 OWASP Top 10 (phiên bản gần nhất) và liên hệ CWE 16
1.9 Một số CWE trong Top 25 liên quan trực tiếp tới nội dung học phần 17
1.10 Các công thức CTL cơ bản kết hợp lượng từ path và toán tử thời gian 18
1.11 Threat modeling STRIDE cho hệ thống đăng ký học phần (trích) 20
1.12 Truy vết từng bước của BFS ngược trong `find_path("locked")` . 21

2.1 Đánh giá rủi ro cho các lớp lỗi bộ nhớ phổ biến trong C 26
2.2 Các vi phạm ownership phổ biến 27
2.3 Một số Undefined Behavior phổ biến nhất trong C 29
2.4 Hàm nguy hiểm và hàm thay thế an toàn hơn 30
2.5 Công cụ phân tích mã C thực tế theo lớp kỹ thuật 33
2.6 Các cơ chế giảm nhẹ (mitigation) tầng hệ điều hành/trình biên dịch 37

3.1 Ma trận Soundness/Completeness và các loại công cụ thực tế . 45
3.2 So sánh Static Analysis, Testing, Debugging 46
3.3 So sánh Symbolic Model Checking (BDD) và Bounded Model Checking (SAT/SMT) 53
3.4 Truy vết DPLL từng bước trên φ 56

4.1 Kiểm thử chức năng vs. Kiểm thử bảo mật 60
4.2 Các mức độ code coverage 60
4.3 Họ Sanitizer của LLVM/GCC 66
4.4 Công cụ structure-aware fuzzing tiêu biểu 68
4.5 Truy vết các iteration then chốt của fuzzing loop 70

D.1 Tiêu chí đánh giá bài tập lớn 125
D.2 Lịch trình thực hiện bài tập lớn 126

CHƯƠNG 1

Giới thiệu về an toàn phần mềm

Mục tiêu chương

Sau khi hoàn thành chương này, sinh viên có khả năng:

- Giải thích được các khái niệm nền tảng của an toàn phần mềm: CIA, AAA, asset/threat/vulnerability/risk, và phân biệt được security với safety và reliability.
- Phân loại được các lớp lỗ hổng phần mềm phổ biến theo CWE, liên hệ với các CVE thực tế đã gây hậu quả nghiêm trọng.
- Áp dụng được khung phân tích mối đe dọa STRIDE và ma trận rủi ro để đánh giá một hệ thống phần mềm đơn giản.
- Trình bày được các kỹ thuật thiết kế test case cổ điển theo chuẩn ISTQB: equivalence partitioning, boundary value analysis, decision table, state transition testing, use case testing.
- Xây dựng được một *Kripke Structure* cho một hệ thống trạng thái hữu hạn đơn giản, và sử dụng nó để kiểm tra tính đúng đắn của một thuộc tính thời gian (temporal property).

1.1 Vì sao an toàn phần mềm quan trọng

Phần mềm điều khiển hầu hết hạ tầng hiện đại: hệ thống ngân hàng lõi (core banking), thiết bị y tế, hệ thống điều khiển giao thông, mạng lưới điện thông minh, xe ô tô tự lái, và hàng tỷ thiết bị IoT. Một lỗi phần mềm tưởng chừng nhỏ có thể dẫn đến hậu quả ở quy mô toàn cầu: rò rỉ dữ liệu hàng trăm triệu người dùng, gián đoạn dịch vụ thiết yếu, thiệt hại tài chính hàng tỷ đô la, và trong các hệ thống an toàn tính mạng (safety-critical), thậm chí gây nguy hiểm trực tiếp đến con người.

Case Study: Heartbleed (CVE-2014-0160)

Heartbleed là một lỗi *buffer over-read* trong phần mở rộng `heartbeat` của thư viện mật mã OpenSSL, được công bố năm 2014. Nguyên nhân là hàm xử lý gói tin heartbeat tin tưởng vào trường độ dài (`payload length`) do client cung cấp mà không kiểm tra nó có khớp với độ dài thực tế của dữ liệu hay không. Kẻ tấn công có thể gửi một gói tin khai báo độ dài lớn hơn nhiều so với dữ liệu thực gửi kèm, khiến server đọc và trả về tối đa 64KB bộ nhớ tiến trình liền kề – có thể chứa khóa riêng tư TLS, mật khẩu, cookie phiên đăng nhập. Ước tính tại thời điểm công bố, khoảng

17% (nửa triệu) máy chủ web sử dụng chứng chỉ TLS bị ảnh hưởng. Bài học cốt lõi: **một lỗi kiểm tra biên bộ nhớ đơn giản (thiếu input validation) có thể phá vỡ toàn bộ mô hình bảo mật mật mã học phía trên nó**. Đây là ví dụ kinh điển cho thấy phần lớn lỗ hổng bảo mật bắt nguồn từ lỗi lập trình cấp thấp, không phải từ thuật toán mật mã yếu.

1.2 Các khái niệm nền tảng của an toàn phần mềm

1.2.1 Security, Safety và Reliability

Ba thuật ngữ này thường bị nhầm lẫn nhưng mang ý nghĩa kỹ thuật khác nhau rõ rệt:

Bảng 1.1: Phân biệt Security, Safety, Reliability

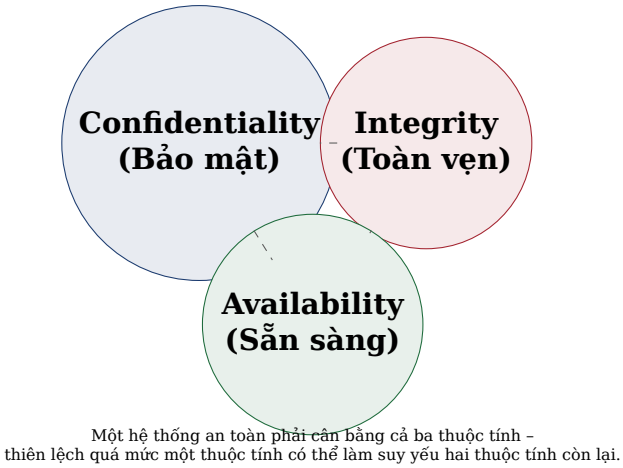
Thuật ngữ	Ý nghĩa
Security (An toàn bảo mật)	Bảo vệ hệ thống trước các tác nhân *có chủ đích* (attacker) muốn khai thác điểm yếu để đạt mục tiêu xấu.
Safety (An toàn vận hành)	Đảm bảo hệ thống không gây hại ngay cả khi *không có* tác nhân tấn công – tức là chống lại lỗi ngẫu nhiên, hỏng hóc phần cứng, hay sai sót con người.
Reliability (Độ tin cậy)	Hệ thống hoạt động đúng chức năng đã đặc tả một cách liên tục, trong điều kiện vận hành xác định, không xét đến tác nhân ác ý.

Ví dụ

Một hệ thống điều khiển bay tự động (autopilot) có thể được chứng nhận rất an toàn vận hành (safety-certified theo DO-178C) – tức không tự rơi do lỗi ngẫu nhiên – nhưng vẫn hoàn toàn mất an toàn bảo mật nếu tin tặc có thể chiếm quyền điều khiển từ xa qua một kênh liên lạc không được xác thực. Ngược lại, một hệ thống có thể chống được mọi cuộc tấn công mạng (an toàn bảo mật cao) nhưng vẫn có thể gây tai nạn do lỗi logic điều khiển thuần túy (mất an toàn vận hành). Hai thuộc tính này *trực giao* với nhau và đòi hỏi các phương pháp kiểm chứng khác nhau, dù có chung nền tảng toán học (formal methods, model checking).

1.2.2 Mô hình CIA

Mô hình CIA (Confidentiality – Integrity – Availability) là bộ ba thuộc tính cốt lõi mà mọi hệ thống an toàn cần bảo vệ.



Hình 1.1: Mô hình CIA – ba thuộc tính cốt lõi của an toàn thông tin

Confidentiality (Bảo mật)

Đảm bảo thông tin chỉ được tiết lộ cho các thực thể được ủy quyền. *Vi phạm điển hình*: rò rỉ dữ liệu người dùng, đọc trộm bộ nhớ tiến trình (Heartbleed). *Cơ chế bảo vệ*: mã hóa (encryption), kiểm soát truy cập (access control), che dữ liệu nhạy cảm (data masking).

Integrity (Toàn vẹn)

Đảm bảo dữ liệu và chương trình không bị thay đổi trái phép hoặc ngoài ý muốn. *Vi phạm điển hình*: sửa đổi giá trị biến qua lỗi tràn bộ nhớ, chèn mã độc, tấn công man-in-the-middle. *Cơ chế bảo vệ*: hàm băm (hash), chữ ký số (digital signature), kiểm tra ràng buộc dữ liệu đầu vào (input validation).

Availability (Sẵn sàng)

Đảm bảo hệ thống và dữ liệu luôn sẵn sàng phục vụ người dùng hợp lệ khi cần. *Vi phạm điển hình*: tấn công từ chối dịch vụ (DoS/DDoS), vòng lặp vô hạn do lỗi lập trình. *Cơ chế bảo vệ*: dự phòng (redundancy), cân bằng tải (load balancing), giới hạn tài nguyên (rate limiting).

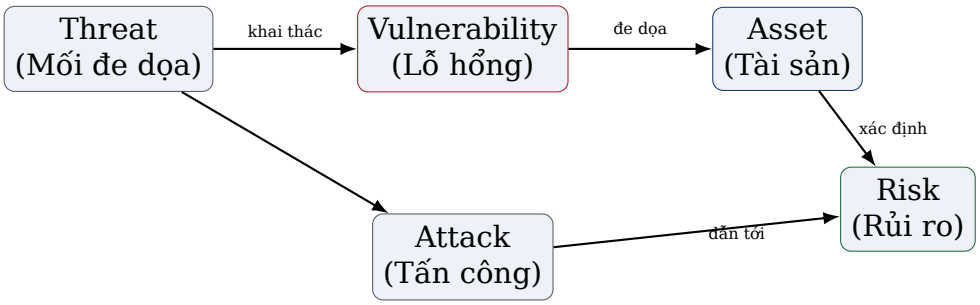
Ngoài CIA, các thuộc tính mở rộng thường được xét thêm, đặc biệt trong bối cảnh hệ thống có yêu cầu trách nhiệm pháp lý:

- ***Authenticity*** (Xác thực): xác nhận danh tính đúng của chủ thể hoặc nguồn gốc đúng của dữ liệu.
- ***Non-repudiation*** (Không thể chối bỏ): chủ thể không thể phủ nhận hành động đã thực hiện – cơ sở của chữ ký số và giao dịch điện tử.
- ***Accountability*** (Trách nhiệm giải trình): mọi hành động trong hệ thống có thể được truy vết (audit log) đến chủ thể thực hiện.

1.2.3 Asset, Threat, Vulnerability, Risk, Attack, Exploit

Bảng 1.2: Các khái niệm nền tảng trong phân tích rủi ro bảo mật

Khái niệm	Định nghĩa
Asset (Tài sản)	Bất kỳ thứ gì có giá trị cần bảo vệ: dữ liệu, mã nguồn, uy tín, dịch vụ.
Threat (Mối đe dọa)	Khả năng xảy ra sự kiện gây hại cho tài sản.
Vulnerability (Lỗ hổng)	Điểm yếu trong hệ thống có thể bị khai thác.
Attack (Tấn công)	Hành động cố ý khai thác lỗ hổng.
Exploit	Đoạn mã hoặc kỹ thuật cụ thể dùng để khai thác lỗ hổng.
Risk (Rủi ro)	Xác suất và mức độ thiệt hại nếu mối đe dọa xảy ra.



Hình 1.2: Quan hệ giữa các khái niệm nền tảng trong phân tích rủi ro

Ví dụ

Buffer overflow (*vulnerability*) trong hàm xử lý input có thể bị kẻ tấn công (*threat agent*) khai thác qua một exploit cụ thể để chiếm quyền điều khiển chương trình (*attack*), gây rủi ro mất dữ liệu (*risk*) trên tài sản (*asset*) là máy chủ.

1.2.4 Đánh giá rủi ro: Risk = Likelihood × Impact

Rủi ro được định lượng (hoặc bán định lượng) bằng công thức kinh điển:

$$\text{Risk} = \text{Likelihood} \times \text{Impact}$$

trong đó *Likelihood* là khả năng xảy ra (khả năng lỗ hổng bị phát hiện và khai thác thành công) và *Impact* là mức độ ảnh hưởng nếu sự kiện xảy ra (thiệt hại tài chính, uy tín, pháp lý, an toàn con người).

Bảng 1.3: Ma trận rủi ro (Risk Matrix) 3x3

Impact \ Likelihood	Thấp	Trung bình	Cao
Cao	Trung bình	Cao	Cao
Trung bình	Thấp	Trung bình	Cao
Thấp	Thấp	Thấp	Trung bình

Ma trận rủi ro giúp ưu tiên xử lý: lỗ hổng có khả năng khai thác cao *và* ảnh hưởng nghiêm trọng cần được vá trước tiên, trong khi lỗ hổng khả năng thấp và ảnh hưởng thấp có thể được chấp nhận rủi ro (risk acceptance) hoặc xử lý sau.

Module `softsec.ch1_models.risk` (xem Phụ lục A) cài đặt công thức này bằng Python:

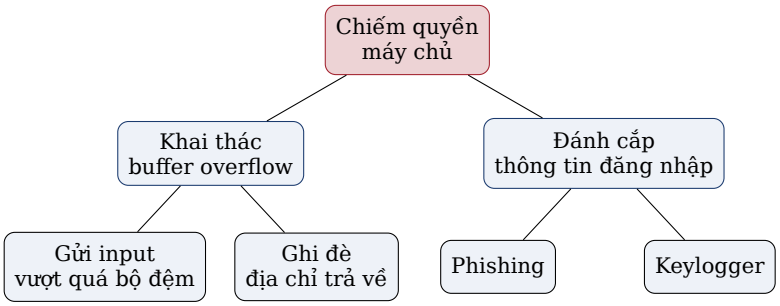
```python
def risk_score(likelihood: int, impact: int) -> int:
    """likelihood, impact trong thang 1..5. Tra ve diem rui ro 1..25."""
    for v, name in ((likelihood, "likelihood"), (impact, "impact")):
        if not (1 <= v <= 5):
            raise ValueError(f"{name} phai trong khoang [1,5], nhan {v}")
    return likelihood * impact

def classify_risk(score: int) -> str:
    """Phan loai diem rui ro thanh Thap / Trung binh / Cao."""
    if score <= 6:
        return "Thap"
    elif score <= 14:
        return "Trung binh"
    else:
        return "Cao"
```

Listing 1.1: Tính điểm rủi ro – `ch1_models/risk.py`

1.2.5 Cây tấn công (Attack Tree)

Attack tree là một kỹ thuật phân rã mục tiêu tấn công thành các bước con, hỗ trợ phân tích rủi ro có hệ thống. Gốc cây là mục tiêu cuối cùng của kẻ tấn công; các nút con biểu diễn các cách khác nhau (thường nối bằng OR) hoặc các bước cần thiết đồng thời (nối bằng AND) để đạt mục tiêu đó.



Hình 1.3: Ví dụ Attack Tree: chiếm quyền máy chủ

1.2.6 Authentication vs. Authorization; Least Privilege

Authentication	Xác minh *bạn là ai* (mật khẩu, sinh trắc học, chứng chỉ số, OTP, MFA).
Authorization	Xác định *bạn được phép làm gì* sau khi đã xác thực (quyền truy cập, RBAC/ABAC).

Lưu ý / Cảnh báo

Đăng nhập vào hệ thống bằng tài khoản hợp lệ (authentication thành công) *không có nghĩa* người dùng được phép xóa dữ liệu của người khác (authorization). Đây là hai bước độc lập, và nhầm lẫn giữa chúng là một lỗi thiết kế phổ biến dẫn đến lỗ hổng ***Broken Access Control*** – hạng mục đứng đầu OWASP Top 10 nhiều năm liên tiếp.

Nguyên tắc đặc quyền tối thiểu (Principle of Least Privilege): mỗi thành phần (người dùng, tiến trình, module) chỉ nên có quyền tối thiểu cần thiết để hoàn thành công việc, nhằm giảm thiểu thiệt hại nếu thành phần đó bị xâm nhập. Ví dụ vi phạm kinh điển: chạy web server với quyền `root` – khi đó một lỗ hổng nhỏ trong ứng dụng web có thể dẫn đến chiếm toàn bộ hệ thống thay vì chỉ tiến trình web server.

1.3 Một số vấn đề về an toàn phần mềm

1.3.1 Khung phân loại CWE và cơ sở dữ liệu CVE

CWE (Common Weakness Enumeration) là một hệ thống phân loại chuẩn hóa các *loại* lỗi phần mềm/thiết kế có thể dẫn đến lỗ hổng bảo mật, do MITRE duy trì. ***CVE*** (Common Vulnerabilities and Exposures) là danh mục các lỗ hổng *cụ thể* đã được phát hiện trong các phần mềm/phiên bản thực tế, mỗi mục có mã định danh duy nhất (ví dụ CVE-2014-0160 chính là Heartbleed). Mối quan hệ: một CVE thường được gán một hoặc nhiều mã CWE mô tả loại lỗi gốc.

Bảng 1.4: Một số CWE phổ biến nhất và CVE minh họa

CWE	Mô tả	CVE minh họa
CWE-119/CWE-125/CWE-787	Đọc/ghi ngoài biên bộ nhớ (buffer over-read/write)	CVE-2014-0160 (Heartbleed)
CWE-416	Use-after-free	CVE-2019-0708 (BlueKeep)
CWE-89	SQL Injection	CVE-2017-5638 (Equifax, thực chất OGNL nhưng cùng họ injection)
CWE-79	Cross-Site Scripting (XSS)	Hàng nghìn CVE trên nền tảng web
CWE-362	Race Condition (TOCTOU)	CVE-2016-5195 (Dirty COW)
CWE-798	Hard-coded Credentials	Nhiều CVE trên thiết bị IoT
CWE-502	Deserialization dữ liệu không tin cậy	CVE-2021-44228 (Log4Shell, kết hợp JNDI lookup)

1.3.2 Các lớp lỗi phần mềm phổ biến

Buffer Overflow (Tràn bộ đệm)

Xảy ra khi chương trình ghi dữ liệu vượt quá kích thước bộ đệm đã cấp phát, ghi đè lên vùng nhớ liền kề – có thể là biến khác, con trỏ, hoặc trên stack là địa chỉ trả về của hàm. Ghi đè địa chỉ trả về là kỹ thuật kinh điển để chiếm quyền điều khiển luồng thực thi (*control-flow hijacking*).

```c
#include <string.h>

void vulnerable(char *input) {
    char buffer[16];
    strcpy(buffer, input);   /* Khong kiem tra do dai! */
}

int main(int argc, char **argv) {
    vulnerable(argv[1]);     /* argv[1] co the dai hon 16 byte */
    return 0;
}
```

Listing 1.2: Buffer overflow kinh điển trên stack

Use-After-Free

Xảy ra khi chương trình tiếp tục sử dụng một con trỏ sau khi vùng nhớ mà nó trỏ tới đã được giải phóng (`free`). Vùng nhớ đó có thể đã được cấp phát lại cho mục đích khác, dẫn đến việc đọc/ghi dữ liệu không xác định hoặc bị kẻ tấn công kiểm soát (heap spraying).

```
char *p = malloc(64);
strcpy(p, "du lieu nhay cam");
free(p);
/* ... nhieu dong code sau ... */
printf("%s\n", p);   /* UNDEFINED BEHAVIOR: doc vung nho da giai phong */
```

Listing 1.3: Use-after-free

Injection (SQL Injection, Command Injection)

Xảy ra khi dữ liệu đầu vào không được kiểm tra/thoát ký tự đúng cách trước khi được nối trực tiếp vào một câu lệnh được thông dịch (SQL, shell, LDAP...), cho phép kẻ tấn công chèn thêm lệnh của riêng mình.

```
# SAI: noi chuoi truc tiep -> de bi injection
query = f"SELECT * FROM users WHERE name = '{username}'"
# input: ' OR '1'='1  ->  bo qua toan bo dieu kien WHERE

# DUNG: dung tham so hoa (parameterized query)
cursor.execute("SELECT * FROM users WHERE name = ?", (username,))
```

Listing 1.4: SQL Injection: sai và đúng

Race Condition / TOCTOU

Time-Of-Check-To-Time-Of-Use: xảy ra khi có khoảng thời gian giữa lúc chương trình kiểm tra một điều kiện (ví dụ quyền truy cập file) và lúc sử dụng kết quả kiểm tra đó, trong khi trạng thái hệ thống có thể đã thay đổi bởi một tiến trình khác trong khoảng thời gian đó.

1.3.3 Khung phân tích mối đe dọa STRIDE

STRIDE (Microsoft) là khung phân loại mối đe dọa theo sáu loại, ánh xạ trực tiếp tới các thuộc tính bảo mật bị vi phạm:

Bảng 1.5: Khung STRIDE và thuộc tính bảo mật bị vi phạm tương ứng

STRIDE	Vi phạm	Mô tả
Spoofing	Authenticity	Giả mạo danh tính người dùng/hệ thống khác
Tampering	Integrity	Thay đổi trái phép dữ liệu/mã nguồn
Repudiation	Non-repudiation	Chối bỏ hành động đã thực hiện
Information Disclosure	Confidentiality	Tiết lộ thông tin cho bên không được ủy quyền
Denial of Service	Availability	Làm hệ thống ngừng phục vụ
Elevation of Privilege	Authorization	Nâng quyền trái phép

Hoạt động trên lớp

Chia nhóm 3–4 sinh viên. Chọn một ứng dụng quen thuộc (ví dụ hệ thống đăng ký học phần trực tuyến). Với mỗi luồng dữ liệu chính (đăng nhập, nộp bài, xem điểm), xác định ít nhất một mối đe dọa cho mỗi chữ cái trong STRIDE. Trình bày kết quả dưới dạng bảng và đề xuất một biện pháp giảm thiểu (*mitigation*) cho mỗi mối đe dọa.

1.4 Kiểm thử và kiểm chứng phần mềm (Phần I)

1.4.1 Verification vs. Validation

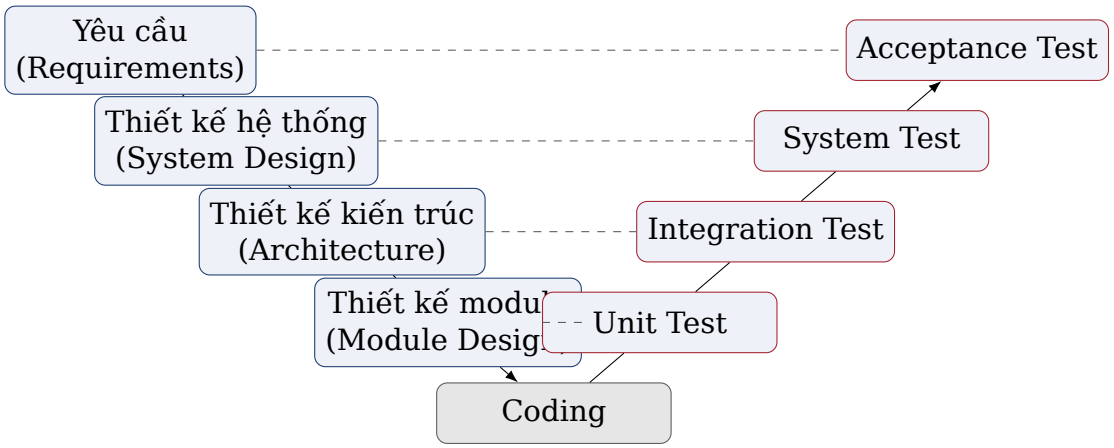
Verification (Kiểm chứng) vs. Validation (Thẩm định)

Verification: ”Are we building the product right?” – xác nhận sản phẩm được xây dựng đúng theo đặc tả thiết kế (kiểm tra tính nhất quán nội tại).
Validation: ”Are we building the right product?” – xác nhận sản phẩm đáp ứng đúng nhu cầu thực tế của người dùng/khách hàng. Một hệ thống có thể *verified* hoàn hảo (đúng 100% theo đặc tả) nhưng vẫn *invalid* nếu đặc tả ban đầu sai lệch với nhu cầu thực tế.

1.4.2 Kiểm thử tĩnh và động (Static vs. Dynamic Testing)

Kiểm thử tĩnh (static testing) kiểm tra phần mềm mà *không* thực thi chương trình: review mã nguồn, walkthrough, inspection, phân tích tĩnh tự động. ***Kiểm thử động*** (dynamic testing) yêu cầu *thực thi* chương trình với dữ liệu đầu vào cụ thể và quan sát kết quả đầu ra thực tế. Đây chính là trục phân chia giữa Chương 3 (phân tích tĩnh) và Chương 4 (phân tích động) của giáo trình này.

1.4.3 Mô hình chữ V (V-Model)



Hình 1.4: Mô hình chữ V: mỗi giai đoạn phát triển tương ứng một cấp độ kiểm thử

1.4.4 Kỹ thuật thiết kế test case hộp đen (Black-box)

Equivalence Partitioning (Phân vùng tương đương)

Chia miền giá trị đầu vào thành các lớp tương đương, sao cho mọi giá trị trong cùng một lớp được kỳ vọng gây ra hành vi giống nhau. Chỉ cần kiểm thử một đại diện của mỗi lớp thay vì toàn bộ miền giá trị.

Ví dụ

Trường nhập tuổi hợp lệ [0, 120]. Các lớp tương đương: $\{x < 0\}$ (không hợp lệ), $\{0 \le x \le 120\}$ (hợp lệ), $\{x > 120\}$ (không hợp lệ). Chỉ cần 3 test case đại diện (ví dụ -5, 30, 150) thay vì kiểm thử mọi giá trị từ $-\infty$ đến $+\infty$.

Boundary Value Analysis (Phân tích giá trị biên)

Lỗi phần mềm thường tập trung tại các giá trị biên (off-by-one errors). Kỹ thuật này kiểm thử tại và ngay sát các điểm biên của mỗi lớp tương đương: với miền $[0, 120]$, kiểm thử tại $-1, 0, 1$ và $119, 120, 121$.

Module `softsec.ch1_models.test_design` tự động sinh các lớp tương đương và giá trị biên từ một khoảng giá trị cho trước:

```
from dataclasses import dataclass

@dataclass
class EquivalenceClass:
    name: str
    low: float
    high: float

def boundary_values(low: int, high: int) -> list[int]:
    """Sinh 6 gia tri bien chuan: low-1, low, low+1, high-1, high, high+1."""
```

```
    return [low - 1, low, low + 1, high - 1, high, high + 1]

def equivalence_classes(low: int, high: int) -> list[EquivalenceClass]:
    return [
        EquivalenceClass("khong hop le (duoi)", float("-inf"), low - 1),
        EquivalenceClass("hop le", low, high),
        EquivalenceClass("khong hop le (tren)", high + 1, float("inf")),
    ]
```

Listing 1.5: Sinh test case tự động – `ch1_models/test_design.py`

Kiểm soát chất lượng

Chuẩn ISTQB (International Software Testing Qualifications Board) – tài liệu tham khảo chính của học phần này – xếp equivalence partitioning và boundary value analysis vào nhóm kỹ thuật *black-box* bắt buộc trong chứng chỉ Foundation Level. Trong quy trình QA công nghiệp, hai kỹ thuật này thường được kết hợp: mỗi lớp tương đương được test bằng một giá trị "điển hình" (nằm giữa khoảng), *cộng thêm* các giá trị biên của chính lớp đó, đảm bảo vừa có độ phủ về số lượng lớp vừa có độ phủ về điểm dễ gây lỗi off-by-one.

Bài tập

Cho hàm kiểm tra mật khẩu hợp lệ: độ dài từ 8 đến 20 ký tự. Hãy:

1. Xác định các lớp tương đương.
2. Liệt kê đầy đủ 6 giá trị biên theo công thức đã học.
3. Viết một test case bằng `pytest` kiểm thử hàm `is_valid_password(s: str) -> bool` với các giá trị trên.

1.5 Kiểm thử và kiểm chứng phần mềm (Phần II)

1.5.1 Các kỹ thuật thiết kế test case bổ sung

Decision Table Testing

Kiểm thử dựa trên tổ hợp các điều kiện logic (nhiều biến điều kiện độc lập), đảm bảo mọi tổ hợp quan trọng đều được kiểm thử – đặc biệt hữu ích khi logic nghiệp vụ có nhiều điều kiện kết hợp bằng AND/OR.

Bảng 1.6: Decision Table: quy tắc giảm giá 10% nếu là thành viên và đơn hàng > 500k

	TC1	TC2	TC3	TC4
Là thành viên?	Đ	Đ	S	S
Đơn > 500k?	Đ	S	Đ	S
Giảm giá 10%?	Có	Không	Không	Không

State Transition Testing

Kiểm thử dựa trên các trạng thái và chuyển đổi trạng thái (state transitions) của hệ thống – nền tảng trực tiếp dẫn tới Kripke Structure ở mục 1.6.

Ví dụ

Áp dụng cho mô hình trạng thái đăng nhập (module Python `ch1_models.kripke`):

- Test case theo trạng thái: $idle \rightarrow authenticated$ (đăng nhập đúng).
- Test case theo trạng thái: $idle \rightarrow locked$ (sai quá số lần cho phép).
- Test case chuyển đổi *không hợp lệ*: từ $locked$ có được phép chuyển thẳng sang $authenticated$ không? Đây chính là câu hỏi mà Kripke Structure và model checking sẽ trả lời một cách hình thức.

Use Case Testing

Thiết kế test case dựa trên các kịch bản sử dụng thực tế của hệ thống (main flow và các alternative/exception flow).

Ví dụ

Use case ”Đăng nhập”: main flow (nhập đúng thông tin), alternative flow (quên mật khẩu), exception flow (nhập sai 5 lần liên tiếp $\rightarrow$ khóa tài khoản).

Experience-based Testing

Error Guessing: dựa vào kinh nghiệm để đoán các lỗi thường gặp (input rỗng, ký tự đặc biệt, số âm, ký tự Unicode ngoài bảng mã dự kiến). **Exploratory Testing**: kiểm thử viên vừa học vừa thiết kế test case song song trong lúc tương tác trực tiếp với hệ thống, không có kịch bản cố định trước.

1.6 Cấu trúc Kripke (Kripke Structure)

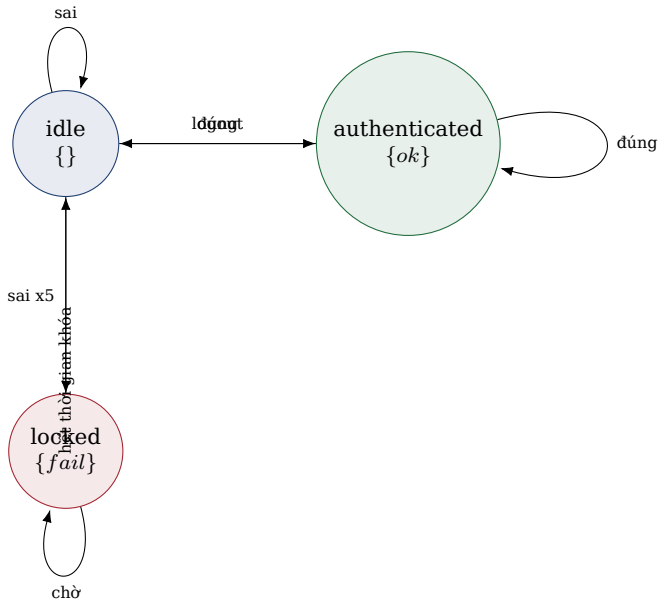
Kripke Structure là công cụ hình thức trung tâm của giáo trình này, được sử dụng lại xuyên suốt Chương 3 (Bounded Model Checking) như nền tảng

biểu diễn không gian trạng thái chương trình.

Kripke Structure

Một Kripke Structure là một bộ tứ $K = (S, S_0, R, L)$ trong đó:

- S: tập hữu hạn các trạng thái (states).
- $S_0 \subseteq S$: tập trạng thái khởi tạo.
- $R \subseteq S \times S$: quan hệ chuyển trạng thái (transition relation), với yêu cầu *total* (mọi trạng thái đều có ít nhất một trạng thái kế tiếp).
- $L : S \to 2^{AP}$: hàm gán nhãn (labeling function), gán cho mỗi trạng thái tập các mệnh đề nguyên tử (atomic propositions) đúng tại trạng thái đó.



Hình 1.5: Kripke Structure cho hệ thống đăng nhập ba trạng thái

1.6.1 Path, Reachability và Temporal Logic

Một ***path*** (đường đi) trong Kripke Structure là một dãy trạng thái vô hạn $s_0, s_1, s_2, \ldots$ sao cho $s_0 \in S_0$ và $(s_i, s_{i+1}) \in R$ với mọi i. Bài toán ***reachability*** hỏi: có tồn tại một path từ S_0 đến một trạng thái đích s_g hay không?

Để đặc tả các thuộc tính cần kiểm chứng, ta dùng ***Linear Temporal Logic (LTL)*** hoặc ***Computation Tree Logic (CTL)*** với các toán tử thời gian cơ bản:

Bảng 1.7: Các toán tử LTL/CTL cơ bản

Ký hiệu	Tên	Ý nghĩa
$G\varphi$ (hay $\Box\varphi$)	Globally	φ đúng tại *mọi* trạng thái trên path
$F\varphi$ (hay $\Diamond\varphi$)	Finally	φ đúng tại *ít nhất một* trạng thái trong tương lai
$X\varphi$	Next	φ đúng tại trạng thái *kế tiếp*
$\varphi\, U\, \psi$	Until	φ đúng cho đến khi ψ trở thành đúng

Ví dụ

Thuộc tính an toàn (*safety property*): ”hệ thống không bao giờ vào trạng thái $locked$ mà không từng trải qua 5 lần sai liên tiếp” – được biểu diễn dưới dạng $G(\neg bad)$: tại mọi trạng thái trên mọi path, mệnh đề ”bad” (trạng thái không mong muốn) không xảy ra. Thuộc tính sống (*liveness property*): ”cuối cùng hệ thống sẽ luôn thoát khỏi trạng thái locked” được biểu diễn là $G(locked \rightarrow F(\neg locked))$.

Module `softsec.ch1_models.kripke` cài đặt cấu trúc dữ liệu này và thuật toán tìm kiếm reachability (BFS) cùng khả năng xuất mã TikZ để trực quan hóa:

```python
from collections import deque
from dataclasses import dataclass, field

@dataclass
class KripkeStructure:
    states: set[str]
    initial: set[str]
    transitions: set[tuple[str, str]]
    labels: dict[str, set[str]] = field(default_factory=dict)

    def successors(self, s: str) -> set[str]:
        return {t for (u, t) in self.transitions if u == s}

    def is_reachable(self, target: str) -> bool:
        """BFS tu tap trang thai khoi tao toi trang thai dich."""
        visited, frontier = set(self.initial), deque(self.initial)
        while frontier:
            s = frontier.popleft()
            if s == target:
                return True
            for nxt in self.successors(s):
                if nxt not in visited:
                    visited.add(nxt)
                    frontier.append(nxt)
        return target in visited

    def find_path(self, target: str) -> list[str] | None:
        """Tim mot duong di (counterexample) den trang thai dich."""
        parent: dict[str, str | None] = {s: None for s in self.initial}
        frontier = deque(self.initial)
```

```
        while frontier:
            s = frontier.popleft()
            if s == target:
                path = [s]
                while parent[path[-1]] is not None:
                    path.append(parent[path[-1]])
                return list(reversed(path))
            for nxt in self.successors(s):
                if nxt not in parent:
                    parent[nxt] = s
                    frontier.append(nxt)
        return None
```

Listing 1.6: Cài đặt Kripke Structure – trích `ch1_models/kripke.py`

Hoạt động trên lớp

Sử dụng `KripkeStructure` ở trên, xây dựng mô hình cho hệ thống đăng nhập ở Hình 1.5. Viết đoạn code Python gọi `find_path("locked")` để tìm một *counterexample path* minh họa làm sao hệ thống có thể đi từ `idle` đến `locked`. So sánh kết quả với sơ đồ trạng thái đã vẽ tay.

1.7 OWASP Top 10 và CWE Top 25

1.7.1 OWASP Top 10

OWASP (Open Web Application Security Project) công bố định kỳ danh sách *OWASP Top 10* – mười hạng mục rủi ro bảo mật ứng dụng web nghiêm trọng nhất, dựa trên dữ liệu thực tế thu thập từ hàng trăm tổ chức. Danh sách này là tài liệu tham chiếu phổ biến nhất trong công nghiệp để ưu tiên đầu tư an ninh ứng dụng.

Bảng 1.8: OWASP Top 10 (phiên bản gần nhất) và liên hệ CWE

Hạng mục	Mô tả	CWE liên quan
A01: Broken Access Control	Kiểm soát truy cập bị phá vỡ: người dùng thực hiện được hành động ngoài phạm vi quyền hạn	CWE-284, CWE-862
A02: Cryptographic Failures	Sử dụng mật mã yếu, lộ khóa, truyền dữ liệu nhạy cảm không mã hóa	CWE-327, CWE-329
A03: Injection	SQL/NoSQL/Command/LDAP injection do thiếu kiểm tra đầu vào	CWE-89, CWE-78
A04: Insecure Design	Lỗi thiết kế kiến trúc từ giai	CWE-1021

1.7.2 CWE Top 25 Most Dangerous Software Weaknesses

Khác với OWASP Top 10 (tập trung ứng dụng web), danh sách *CWE Top 25* của MITRE bao trùm mọi loại phần mềm, được xếp hạng dựa trên tần suất xuất hiện trong CVE và điểm CVSS trung bình.

Bảng 1.9: Một số CWE trong Top 25 liên quan trực tiếp tới nội dung học phần

Hạng	CWE	Tên
1	CWE-787	Out-of-bounds Write
2	CWE-79	Cross-Site Scripting
3	CWE-89	SQL Injection
4	CWE-416	Use After Free
5	CWE-78	OS Command Injection
7	CWE-125	Out-of-bounds Read
14	CWE-476	NULL Pointer Dereference
19	CWE-190	Integer Overflow or Wraparound

Kiểm soát chất lượng

Có thể quan sát rằng 4 trong số 7 vị trí cao nhất của CWE Top 25 (out-of-bounds write, use-after-free, out-of-bounds read) đều là lỗi bộ nhớ trong các ngôn ngữ không an toàn bộ nhớ như C/C++ – củng cố trực tiếp lý do Chương 2 của giáo trình này dành trọn vẹn cho chủ đề lập trình C an toàn.

1.8 Phân tích rủi ro định lượng

Ngoài ma trận rủi ro định tính (mục 1.2), các tổ chức lớn thường áp dụng phân tích rủi ro *định lượng* (quantitative risk analysis) bằng các công thức tài chính cụ thể:

ALE – Annualized Loss Expectancy

$$ALE = SLE \times ARO$$

trong đó ***SLE*** (Single Loss Expectancy) là thiệt hại ước tính cho một lần sự cố xảy ra, và ***ARO*** (Annualized Rate of Occurrence) là số lần sự cố dự kiến xảy ra trong một năm.

Ví dụ

Một lỗ hổng SQL injection trên hệ thống thương mại điện tử: nếu bị khai thác, thiệt hại ước tính (bao gồm bồi thường, phạt hành chính GDPR, mất uy tín) là 2 tỷ đồng ($SLE = 2 \times 10^9$). Dựa trên dữ liệu ngành, xác suất một lỗ hổng SQL injection chưa vá bị khai thác trong một năm là khoảng 15% ($ARO = 0.15$). Vậy $ALE = 2 \times 10^9 \times 0.15 = 300$ triệu đồng/năm – đây là con số dùng để so sánh với chi phí đầu tư khắc phục (ví dụ thuê kiểm thử bảo

mật định kỳ chi phí 100 triệu đồng/năm), giúp ra quyết định đầu tư dựa trên ROI (Return on Security Investment) thay vì cảm tính.

1.9 Mở rộng về Kripke Structure và Temporal Logic

1.9.1 CTL và các toán tử định lượng theo path

Trong khi LTL (mục 1.6) đặc tả thuộc tính trên *một* path cụ thể, ***CTL*** (Computation Tree Logic) cho phép định lượng trên *tập hợp các path* xuất phát từ một trạng thái, bằng hai toán tử lượng từ: A (for All paths – mọi nhánh có thể) và E (there Exists a path – tồn tại ít nhất một nhánh).

Bảng 1.10: Các công thức CTL cơ bản kết hợp lượng từ path và toán tử thời gian

Công thức	Ý nghĩa
$AG\varphi$	Trên *mọi* nhánh, tại *mọi* trạng thái, φ đúng (safety mạnh nhất)
$EF\varphi$	*Tồn tại* một nhánh mà tại một trạng thái nào đó trong tương lai, φ đúng (reachability)
$AF\varphi$	Trên *mọi* nhánh, φ cuối cùng sẽ đúng (liveness mạnh)
$EG\varphi$	*Tồn tại* một nhánh mà φ đúng mãi mãi

Ví dụ

Với Kripke Structure đăng nhập ở Hình 1.5: thuộc tính $EF(locked)$ hỏi ”có tồn tại một kịch bản dẫn tới trạng thái locked không?” (câu trả lời: có, qua 5 lần sai). Thuộc tính $AG(\neg(authenticated \wedge locked))$ khẳng định ”không bao giờ đồng thời ở cả hai trạng thái authenticated và locked” – một bất biến cấu trúc hiển nhiên đúng vì S là tập trạng thái rời nhau, nhưng minh họa cách CTL diễn đạt các bất biến an toàn một cách hình thức để công cụ tự động kiểm tra thay vì con người tự suy luận thủ công (dễ sai sót với hệ thống lớn).

1.9.2 Thuật toán gán nhãn CTL (Labeling Algorithm)

Model checker CTL cổ điển (ví dụ NuSMV) sử dụng thuật toán gán nhãn (labeling algorithm) quy nạp trên cấu trúc cú pháp của công thức: để kiểm tra $K, s \models \varphi$ cho mọi trạng thái s, thuật toán tính tập $Sat(\varphi) = \{s \in S \mid K, s \models \varphi\}$ từ trong ra ngoài (bottom-up theo cây cú pháp của φ), bắt đầu từ các mệnh đề nguyên tử và kết hợp dần qua các toán tử logic và temporal.

Algorithm 1: Tính $Sat(EX\,\varphi)$ từ $Sat(\varphi)$ đã biết

Đầu vào: Kripke Structure $K = (S, S_0, R, L)$, tập $Sat(\varphi) \subseteq S$
Đầu ra: $Sat(EX\,\varphi)$

```
result ← ∅;
foreach s ∈ S do
    if ∃s' ∈ Sat(φ) sao cho (s, s') ∈ R then
        result ← result ∪ {s};
return result;
```

Với $EF\varphi \equiv \varphi \vee EX(EF\varphi)$ (định nghĩa đệ quy điểm bất động nhỏ nhất – *least fixpoint*), thuật toán lặp cho tới khi tập $Sat(EF\varphi)$ không thay đổi – chính là thuật toán ***BFS ngược*** đã cài đặt ở Listing 1.6 (`find_path`), chỉ khác là ở đây ta tính *tập* trạng thái thỏa mãn thay vì một đường đi cụ thể.

Hoạt động trên lớp

Mở rộng bài tập ATM ở mục trước: với mô hình 5 trạng thái đã xây dựng, viết bằng tay tập $Sat(\varphi)$ cho $\varphi = AG(\neg giao_dich \vee da_xac_thuc)$ theo từng bước của thuật toán gán nhãn (bắt đầu từ mệnh đề nguyên tử, sau đó áp dụng phủ định và AG).

1.10 Thực hành: Threat Modeling STRIDE đầy đủ

Để tổng hợp toàn bộ kiến thức chương 1, phần này trình bày một quy trình threat modeling STRIDE đầy đủ cho một hệ thống ví dụ: *Hệ thống đăng ký học phần trực tuyến*.

Bảng 1.11: Threat modeling STRIDE cho hệ thống đăng ký học phần (trích)

STRIDE	Mối đe dọa cụ thể	Biện pháp giảm thiểu
Spoofing	Sinh viên đăng nhập giả danh sinh viên khác qua session cookie bị đánh cắp	MFA, cookie có cờ `HttpOnly`/`Secure`, kiểm tra IP/thiết bị bất thường
Tampering	Sửa điểm số qua request giả mạo tới API cập nhật điểm	Kiểm tra chữ ký/HMAC trên request nội bộ, phân quyền RBAC chặt cho endpoint
Repudiation	Giảng viên phủ nhận đã sửa điểm sau khi bị phát hiện sai sót	Audit log không thể sửa (append-only, có chữ ký thời gian)
Information Disclosure	Rò rỉ danh sách điểm toàn lớp qua lỗi phân quyền IDOR (Insecure Direct Object Reference)	Kiểm tra quyền sở hữu tài nguyên ở tầng server cho mọi request, không chỉ ẩn ID trên giao diện
Denial of Service	Tấn công vào thời điểm mở đăng ký học phần (traffic đỉnh điểm)	Rate limiting, hàng đợi (queue), CDN, auto-scaling
Elevation of Privilege	Sinh viên khai thác lỗ hổng để có quyền giảng viên/quản trị	Nguyên tắc đặc quyền tối thiểu, kiểm tra phân quyền ở mọi tầng (defense in depth)

1.11 Thực hành từng bước: truy vết BFS trên Kripke Structure

Để củng cố sự liên kết giữa lý thuyết (mục 1.6) và cài đặt (Listing 1.6), phần này trình bày từng bước thực thi thuật toán `find_path` trên ví dụ hệ thống đăng nhập ba trạng thái.

```
k = KripkeStructure(
    states={"idle", "authenticated", "locked"},
    initial={"idle"},
    transitions={
        ("idle", "idle"), ("idle", "authenticated"), ("idle", "locked"),
        ("authenticated", "authenticated"), ("authenticated", "idle"),
        ("locked", "locked"), ("locked", "idle"),
    },
)
path = k.find_path("locked")
print(path)   # Ket qua mong doi: ['idle', 'locked']
```

Listing 1.7: Khởi tạo mô hình đăng nhập ba trạng thái để truy vết

Bảng 1.12: Truy vết từng bước của BFS ngược trong `find_path("locked")`

Bước	Frontier (hàng đợi)	Trạng thái xử lý	Ghi chú
1	`[idle]`	`idle`	Khởi tạo: `parent[idle]=None`
2	`[idle, authenticated, locked]`	–	Thêm 3 successor của `idle` (bao gồm chính nó – bỏ qua vì đã có trong `parent`)
3	`[authenticated, locked]`	`idle` (lặp)	Đã xử lý, bỏ qua vì đã trong `visited`
4	`[locked]`	`authenticated`	Không phải đích, thêm successor (đã có trong parent, bỏ qua)
5	`[]`	`locked`	**Trùng đích!** Dừng, truy ngược `parent`: `locked` → `idle`

Kết quả trả về `["idle", "locked"]` chính xác khớp với cạnh trực tiếp trong Hình 1.5: hệ thống có thể chuyển từ `idle` sang `locked` chỉ sau "sai x5" – đây là counterexample ngắn nhất (độ dài path = 2) chứng minh trạng thái `locked` *reachable được* từ trạng thái khởi tạo, đúng như kỳ vọng thiết kế ban đầu của hệ thống.

Hoạt động trên lớp

Sửa đổi `transitions` ở trên để loại bỏ hoàn toàn cạnh `("idle", "locked")` (giả sử hệ thống bị lỗi thiết kế không bao giờ khóa tài khoản). Chạy lại `find_path("locked")` và giải thích kết quả trả về (`None`) có ý nghĩa gì đối với thuộc tính $EF(locked)$ đã học ở mục mở rộng CTL.

1.12 Tổng kết chương

Tổng kết chương

- An toàn phần mềm khác biệt nhưng liên quan mật thiết với an toàn vận hành (safety) và độ tin cậy (reliability); mô hình CIA (Confidentiality, Integrity, Availability) cùng các mở rộng AAA (Authenticity, Accountability, Non-repudiation) là nền tảng để phân tích mọi yêu cầu bảo mật.

- Các khái niệm asset/threat/vulnerability/risk/attack/exploit tạo thành ngôn ngữ chung để phân tích rủi ro; công thức $Risk = Likelihood \times Impact$ và ma trận rủi ro giúp ưu tiên xử lý.
- CWE/CVE là hai cơ sở dữ liệu chuẩn hóa nền tảng của toàn ngành an toàn thông tin; STRIDE là khung phân tích mối đe dọa thực dụng cho thiết kế hệ thống.
- Các kỹ thuật kiểm thử hộp đen cổ điển (equivalence partitioning, boundary value analysis, decision table, state transition, use case, experience-based) theo chuẩn ISTQB là nền tảng thực hành kiểm thử bảo mật.
- Kripke Structure (S, S_0, R, L) là công cụ hình thức để biểu diễn không gian trạng thái hệ thống, cho phép đặt câu hỏi reachability và kiểm chứng các thuộc tính temporal logic (safety/liveness) – nền tảng trực tiếp cho Bounded Model Checking ở Chương 3.

Xu hướng hiện đại

Các công cụ phân tích rủi ro hiện đại (ví dụ nền tảng threat intelligence thương mại) đang tích hợp mô hình học máy để tự động gợi ý điểm CVSS (Common Vulnerability Scoring System) và ưu tiên bản vá dựa trên dữ liệu khai thác thực tế trên Internet (*Exploit Prediction Scoring System – EPSS*), thay vì chỉ dựa vào đánh giá thủ công Likelihood×Impact tĩnh. Các mô hình ngôn ngữ lớn (LLM) cũng đang được thử nghiệm để tự động sinh attack tree và kịch bản STRIDE từ đặc tả kiến trúc hệ thống viết bằng ngôn ngữ tự nhiên, rút ngắn đáng kể thời gian threat modeling thủ công.

Bài tập

Bài tập tự luận.

1. Trình bày ba ví dụ thực tế (ngoài Heartbleed) về lỗ hổng bảo mật gây hậu quả nghiêm trọng; với mỗi ví dụ, xác định mã CWE tương ứng.
2. So sánh verification và validation bằng một ví dụ cụ thể khác với ví dụ trong giáo trình.
3. Vẽ Kripke Structure cho một máy trạng thái ATM đơn giản (3–5 trạng thái: chờ thẻ, nhập PIN, giao dịch, khóa thẻ) và xác định một thuộc tính safety cần kiểm chứng.

Bài tập lập trình.

4. Cài đặt hàm `classify_risk` mở rộng hỗ trợ 5 mức (Rất thấp, Thấp, Trung bình, Cao, Rất cao) thay vì 3 mức, viết test bằng `pytest`.
5. Sử dụng `KripkeStructure` để cài đặt và kiểm tra reachability cho mô hình ATM ở câu 3.

CHƯƠNG 2

Lập trình C an toàn: Quản lý bộ nhớ

Mục tiêu chương

Sau khi hoàn thành chương này, sinh viên có khả năng:

- Đánh giá được rủi ro bảo mật đặc thù của ngôn ngữ C thông qua khái niệm attack surface, undefined behavior, và mô hình bộ nhớ tiến trình.
- Phân tích được các cấu trúc dữ liệu động (con trỏ, heap, linked list) dưới góc độ an toàn bộ nhớ, xác định ownership và lifetime.
- Áp dụng được các nguyên tắc và hàm an toàn khi lập trình C (bounds checking, safe string functions, kiểm tra giá trị trả về của `malloc`).
- Giải thích được nguyên lý căn chỉnh bộ nhớ (memory alignment) và ảnh hưởng của nó đến hiệu năng và bảo mật.
- Sử dụng thành thạo các công cụ phân tích tĩnh thực tế (cppcheck, clang-tidy) và công cụ động (Valgrind, AddressSanitizer) để phát hiện lỗi bộ nhớ trong mã C.

2.1 Vì sao C vẫn được dùng rộng rãi dù có rủi ro cao

Ngôn ngữ C, ra đời năm 1972, vẫn là nền tảng của phần lớn hạ tầng phần mềm hiện đại: nhân hệ điều hành (Linux kernel, Windows NT), trình biên dịch, thư viện chuẩn, firmware, driver, và hệ thống nhúng. Lý do C tiếp tục thống trị các lĩnh vực này:

- ***Hiệu năng cao***: không có garbage collector, không có runtime overhead của các ngôn ngữ quản lý bộ nhớ tự động.
- ***Kiểm soát trực tiếp bộ nhớ và phần cứng***: cần thiết cho lập trình hệ thống, driver thiết bị, lập trình nhúng tài nguyên hạn chế.
- ***Tính di động và hệ sinh thái lâu đời***: hầu như mọi kiến trúc CPU đều có trình biên dịch C; hàng thập kỷ thư viện và công cụ đã được xây dựng xung quanh nó.

Đánh đổi cho hiệu năng đó là C *không* có cơ chế bảo vệ bộ nhớ tích hợp sẵn: không kiểm tra biên mảng tự động, quản lý bộ nhớ thủ công (malloc/free),

và một lượng lớn hành vi được đặc tả là ***undefined behavior (UB)*** – tức trình biên dịch được phép làm *bất cứ điều gì* khi chương trình vi phạm các ràng buộc đó, kể cả những điều gây mất an toàn nghiêm trọng.

2.2 Đánh giá rủi ro (Risk Assessment) cho mã C

2.2.1 Attack Surface trong mã C

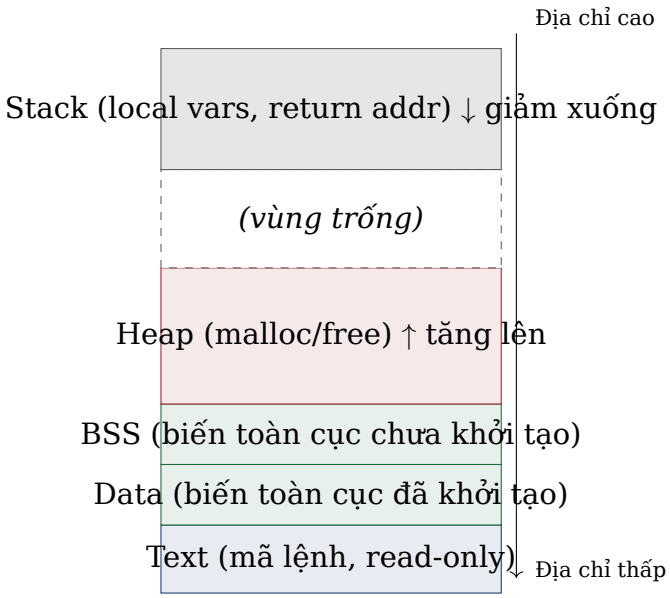
Attack surface là tổng hợp tất cả các điểm mà kẻ tấn công có thể tương tác với hệ thống để đưa dữ liệu đầu vào không tin cậy vào chương trình. Trong mã C, các điểm attack surface điển hình gồm:

- Tham số dòng lệnh (`argv`), biến môi trường.
- Dữ liệu đọc từ file, socket mạng, pipe liên tiến trình.
- Dữ liệu người dùng nhập qua giao diện (kể cả giao diện dòng lệnh).
- Dữ liệu từ các thư viện bên thứ ba (định dạng file phức tạp: hình ảnh, video, tài liệu – nguồn phổ biến của lỗ hổng parser).

Kiểm soát chất lượng

Trong quy trình phát triển phần mềm an toàn (Secure SDLC), bước đầu tiên của threat modeling luôn là vẽ ***Data Flow Diagram (DFD)*** để xác định mọi ***trust boundary*** – ranh giới nơi dữ liệu đi từ vùng tin cậy thấp sang vùng tin cậy cao hơn. Mọi điểm vượt trust boundary *bắt buộc* phải có input validation, bất kể dữ liệu ”trông có vẻ” đã được kiểm tra ở lớp trước đó (nguyên tắc *defense in depth*).

2.2.2 Mô hình bộ nhớ tiến trình (Process Memory Layout)



Hình 2.1: Mô hình bố trí bộ nhớ tiến trình điển hình (kiến trúc x86/x86-64 Linux)

Heap phát triển từ địa chỉ thấp lên cao; stack phát triển từ địa chỉ cao xuống thấp. Buffer overflow trên stack (ghi đè return address, saved base pointer, hoặc biến local khác) và buffer overflow trên heap (ghi đè metadata của chunk kế tiếp, hoặc dữ liệu của object khác) là hai lớp lỗ hổng có cơ chế khai thác khác nhau đáng kể, nhưng chung một nguyên nhân gốc: thiếu kiểm tra biên.

2.2.3 Ma trận rủi ro cho mã C

Bảng 2.1: Đánh giá rủi ro cho các lớp lỗi bộ nhớ phổ biến trong C

Lỗi	Likelihood	Impact	Ghi chú
Stack buffer overflow	Cao	Cao	Kỹ thuật khai thác đã hoàn thiện (ROP, shellcode giảm nhẹ bởi stack canary, ASLR, DEP/NX
Heap buffer overflow	Trung bình	Cao	Khai thác phức tạp hơn (phụ thuộc allocator) nhưng vẫn phổ biến
Use-after-free	Trung bình	Cao	Nền tảng của nhiều lỗ hổng trình duyệt hiện đại (UAF trong DOM)
Integer overflow dẫn tới buffer overflow	Trung bình	Cao	Dễ bị bỏ sót khi

2.3 Cấu trúc dữ liệu động (Dynamic Data Structures)

2.3.1 Con trỏ và Ownership

Khái niệm ***ownership*** (quyền sở hữu vùng nhớ) tuy không tồn tại như một cấu trúc ngôn ngữ trong C (khác với Rust), nhưng là một *quy ước thiết kế bắt buộc* mà lập trình viên phải tự duy trì bằng kỷ luật: mỗi vùng nhớ cấp phát động cần có *đúng một* thành phần chịu trách nhiệm giải phóng nó, tại *đúng một* thời điểm.

Bảng 2.2: Các vi phạm ownership phổ biến

Vi phạm	Hậu quả
Double free	Gọi `free()` hai lần trên cùng một con trỏ → hỏng metadata heap allocator, có thể dẫn tới RCE
Memory leak	Không bao giờ gọi `free()` → cạn kiệt bộ nhớ dần (Availability)
Dangling pointer	Con trỏ vẫn được dùng sau khi vùng nhớ đã giải phóng (use-after-free)
Unclear ownership	Nhiều con trỏ cùng trỏ tới một vùng nhớ mà không rõ ai chịu trách nhiệm giải phóng

```c
char *buf = malloc(128);
process(buf);
free(buf);
/* ... logic loi, nhanh xu ly khac cung goi free tren buf ... */
free(buf);   /* DOUBLE FREE: hong heap metadata */
```

Listing 2.1: Double free – lỗi ownership kinh điển

Lưu ý / Cảnh báo

Sau khi gọi `free(ptr)`, giá trị của `ptr` vẫn còn (con trỏ không tự động được đặt về `NULL`). Đây là lý do khiến *dangling pointer* là lỗi cực kỳ dễ mắc phải; quy ước phòng thủ tốt là luôn gán `ptr = NULL;` ngay sau `free(ptr);`.

2.3.2 Linked List an toàn

```c
typedef struct Node {
    int data;
    struct Node *next;
} Node;

Node *create_node(int value) {
    Node *n = malloc(sizeof(Node));
    if (n == NULL) {           /* LUON kiem tra gia tri tra ve cua malloc */
        return NULL;
```

```c
    }
    n->data = value;
    n->next = NULL;
    return n;
}

void free_list(Node **head) {
    Node *cur = *head;
    while (cur != NULL) {
        Node *next = cur->next; /* luu truoc khi free */
        free(cur);
        cur = next;
    }
    *head = NULL;               /* tranh dangling pointer o cap goi */
}
```

Listing 2.2: Linked list với kiểm tra lỗi malloc và giải phóng an toàn

Case Study: CWE-416 trong trình duyệt web hiện đại

Phần lớn lỗ hổng RCE (Remote Code Execution) nghiêm trọng được vá trong Chrome/Firefox những năm gần đây thuộc lớp *use-after-free* trong cây DOM: một đối tượng DOM bị giải phóng bởi một luồng xử lý sự kiện JavaScript, nhưng một tham chiếu (con trỏ C++) tới đối tượng đó vẫn còn tồn tại trong engine render và bị truy cập lại sau đó. Kẻ tấn công có thể ”phun” (spray) dữ liệu có kiểm soát vào đúng vùng nhớ vừa giải phóng để biến dangling pointer thành công cụ thực thi mã tùy ý. Đây là lý do các trình duyệt hiện đại đầu tư mạnh vào các allocator cứng hóa (*hardened allocators*: PartitionAlloc, MiraclePtr) như một lớp phòng thủ bổ sung ở tầng runtime, thay vì chỉ dựa vào việc lập trình viên không bao giờ mắc lỗi.

2.4 Lập trình C an toàn (Secure C Coding)

2.4.1 Undefined Behavior (UB)

Undefined Behavior

Chuẩn ngôn ngữ C (ISO/IEC 9899) định nghĩa một số hành vi là ”không xác định” – trình biên dịch *không có nghĩa vụ* tạo ra bất kỳ hành vi nhất quán hay dự đoán được nào khi chương trình vi phạm. Trong thực tế, trình biên dịch tối ưu hóa hiện đại thường *giả định UB không bao giờ xảy ra* và tối ưu mã dựa trên giả định đó – dẫn đến những kết quả gây bất ngờ, đôi khi loại bỏ hẳn các kiểm tra bảo mật mà lập trình viên đã viết.

```c
/* Gia dinh: kiem tra tran so nguyen truoc khi cong */
int add_safe(int a, int b) {
    if (a + b < a) {          /* UB: tran so int co dau la undefined behavior */
        /* trinh bien dich CO THE loai bo nhanh nay hoan toan, */
```

```
        /* vi no "gia dinh" a+b khong bao gio tran               */
        return -1;
    }
    return a + b;
}
/* Ban sua dung: kiem tra TRUOC khi cong, dung phep tru */
int add_safe_fixed(int a, int b) {
    if (b > 0 && a > INT_MAX - b) return -1;
    if (b < 0 && a < INT_MIN - b) return -1;
    return a + b;
}
```

Listing 2.3: UB khiến trình biên dịch loại bỏ kiểm tra bảo mật

Bảng 2.3: Một số Undefined Behavior phổ biến nhất trong C

UB	**Mô tả**
Tràn số nguyên có dấu (signed overflow)	`INT_MAX + 1` là UB (khác với unsigned overflow – có định nghĩa rõ ràng là wrap-around)
Truy cập ngoài biên mảng	Đọc/ghi `arr[i]` với `i` nằm ngoài `[0, size)`
Dereference con trỏ NULL	`*ptr` khi `ptr == NULL`
Sử dụng biến chưa khởi tạo	Đọc giá trị của biến local trước khi gán
Dịch bit vượt quá độ rộng kiểu	`1 << 32` trên kiểu 32-bit
Vi phạm strict aliasing	Truy cập cùng vùng nhớ qua hai con trỏ kiểu không tương thích

2.4.2 Các hàm chuỗi an toàn

Bảng 2.4: Hàm nguy hiểm và hàm thay thế an toàn hơn

Hàm nguy hiểm	Thay thế	Ghi chú
`strcpy`	`strncpy` / `strlcpy`	`strncpy` không đảm bảo null-terminate nếu nguồn dài bằng đúng kích thước đích
`strcat`	`strncat` / `strlcat`	Vẫn cần tính toán đúng kích thước còn lại
`sprintf`	`snprintf`	Luôn truyền đúng kích thước buffer đích
`gets`	`fgets`	`gets` đã bị loại bỏ khỏi chuẩn C11
`scanf("%s", ...)`	`scanf("%Ns", ...)` có giới hạn độ dài	Luôn giới hạn độ dài trường `%s`

```c
int safe_string_copy(char *dst, size_t dst_size, const char *src) {
    if (dst == NULL || src == NULL || dst_size == 0) {
        return -1;
    }
    size_t src_len = strnlen(src, dst_size);   /* gioi han quet */
    if (src_len >= dst_size) {
        return -1;                              /* nguon qua dai, tu choi */
    }
    memcpy(dst, src, src_len);
    dst[src_len] = '\0';
    return 0;
}
```

Listing 2.4: Checklist review áp dụng thực tế: hàm copy chuỗi an toàn

Hoạt động trên lớp

Checklist review mã C an toàn (áp dụng trực tiếp lên bài tập lớn của học phần):

1. Mọi lời gọi `malloc/calloc/realloc` có kiểm tra giá trị trả về `NULL` không?
2. Mọi con trỏ sau `free()` có được gán lại `NULL` không?
3. Mọi thao tác copy chuỗi/mảng có kiểm tra kích thước đích trước khi

ghi không?

4. Mọi phép toán số học trên chỉ số mảng/độ dài có được kiểm tra tràn số trước không?
5. Mọi vòng lặp xử lý mảng có điều kiện dừng đúng `(i < size)` thay vì `(i <= size)` không?

2.5 Sắp xếp cấu trúc dữ liệu (Data Structure Alignment)

2.5.1 Vì sao CPU yêu cầu dữ liệu được căn chỉnh

CPU đọc/ghi bộ nhớ theo từ (word) có kích thước cố định (4 byte, 8 byte...), bắt đầu tại địa chỉ chia hết cho kích thước đó. Truy cập dữ liệu không căn chỉnh (*misaligned*) trên một số kiến trúc có thể: chậm hơn (cần nhiều chu kỳ bus hơn để đọc 2 word rồi ghép lại), hoặc trên một số kiến trúc (ví dụ SPARC cũ, một số ARM) gây lỗi `SIGBUS` trực tiếp.

2.5.2 Padding và ảnh hưởng bảo mật của Structure Layout

```
struct Example {
    char  a;   /* offset 0, 1 byte */
    /* 3 byte padding an de int b canh chinh tai offset 4 */
    int   b;   /* offset 4, 4 byte */
    char  c;   /* offset 8, 1 byte */
    /* 3 byte padding cuoi struct de mang struct nay co the align dung */
};
/* sizeof(struct Example) == 12, khong phai 1+4+1=6 */
```

Listing 2.5: Padding ẩn trong struct: bẫy bảo mật khi serialize

</lstlisting>

Lưu ý / Cảnh báo

Các byte padding *không được khởi tạo* bởi trình biên dịch. Nếu một struct như trên được ghi trực tiếp ra file hoặc gửi qua mạng bằng `memcpy`/`write` (thay vì serialize từng trường), các byte padding sẽ chứa *rác từ stack/heap trước đó* – có thể vô tình rò rỉ dữ liệu nhạy cảm (địa chỉ con trỏ, mảnh dữ liệu cũ) ra ngoài. Đây là một lớp lỗ hổng ***information disclosure*** tinh vi, thường bị bỏ sót khi review vì "trông không giống" lỗi bảo mật thông thường.

Ví dụ

Sắp xếp lại thứ tự trường theo kích thước giảm dần giúp giảm padding:

```
struct Optimized {
```

```
    int  b;   /* offset 0, 4 byte */
    char a;   /* offset 4, 1 byte */
    char c;   /* offset 5, 1 byte */
    /* chi con 2 byte padding cuoi, sizeof = 8 thay vi 12 */
};
```

2.6 Công cụ phân tích tĩnh tìm lỗi bảo mật

2.6.1 Phân loại công cụ

Bảng 2.5: Công cụ phân tích mã C thực tế theo lớp kỹ thuật

Công cụ	Loại	Thời điểm
`cppcheck`	Static analysis	Compile-ti
`clang-tidy`	Static analysis	Compile-ti
`clang -fanalyze`	Symbolic execution nhẹ	Compile-ti

```
# Phan tich tinh voi cppcheck, bat tat ca canh bao
cppcheck --enable=all --inconclusive --std=c11 vulnerable.c

# Bien dich voi AddressSanitizer de phat hien loi runtime
gcc -fsanitize=address -g -O0 vulnerable.c -o vulnerable_asan
./vulnerable_asan

# Valgrind Memcheck (khong can bien dich lai)
gcc -g -O0 vulnerable.c -o vulnerable
valgrind --leak-check=full --show-leak-kinds=all ./vulnerable
```

Listing 2.6: Sử dụng cppcheck và ASan trong thực tế

2.6.2 Module `softsec.ch2_memsafe`: quét mẫu lỗi bằng Python

Để minh họa nguyên lý hoạt động của một static analyzer đơn giản (dựa trên pattern matching, khác với phân tích luồng dữ liệu đầy đủ của cppcheck), module `ch2_memsafe.pattern_scanner` quét mã nguồn C tìm các mẫu lời gọi hàm nguy hiểm bằng biểu thức chính quy và heuristic đơn giản:

```
import re
from dataclasses import dataclass

DANGEROUS_FUNCTIONS = {
    "strcpy":  "Khong kiem tra do dai dich; dung strncpy/strlcpy",
    "strcat":  "Khong kiem tra do dai dich; dung strncat/strlcat",
    "sprintf": "Khong gioi han do dai buffer dich; dung snprintf",
    "gets":    "Da bi loai bo khoi C11; dung fgets",
}

@dataclass
class Finding:
    line_no: int
    function: str
    reason: str
    snippet: str

def scan_source(source: str) -> list[Finding]:
    findings = []
    for i, line in enumerate(source.splitlines(), start=1):
        for func, reason in DANGEROUS_FUNCTIONS.items():
            if re.search(rf"\b{func}\s*\(", line):
                findings.append(Finding(i, func, reason, line.strip()))
    return findings
```

Listing 2.7: Bộ quét mẫu lỗi bộ nhớ – trích `ch2_memsafe/pattern_scanner.py`

Kiểm soát chất lượng

Bộ quét trên minh họa đúng *giới hạn cơ bản* của phân tích tĩnh dựa trên pattern matching: nó không hiểu ngữ cảnh (một lời gọi `strcpy` trong comment hoặc string literal vẫn bị báo lỗi – *false positive*), và không phát hiện được lỗi tương đương về mặt ngữ nghĩa nhưng khác cú pháp (ví dụ tự viết vòng lặp copy byte-by-byte không kiểm tra biên – *false negative*). Đây chính là động lực dẫn tới Chương 3, nơi ta xây dựng các công cụ phân tích dựa trên *ngữ nghĩa hình thức* (SAT/SMT, bounded model checking) thay vì chỉ dựa trên cú pháp bề mặt.

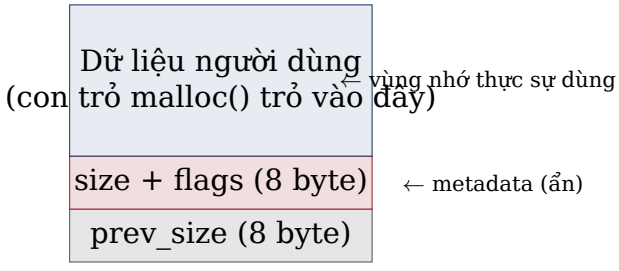
Case Study: CVE-2021-3156 (Sudo Baron Samedit) – Heap buffer overflow

Một lỗi heap buffer overflow tồn tại gần 10 năm trong lệnh `sudo` (chương trình có mặt trên hầu hết hệ thống Unix/Linux), được phát hiện năm 2021. Lỗi nằm ở cách `sudo` xử lý ký tự escape ngược (`\`) khi phân tích tham số dòng lệnh ở chế độ *shell mode*: khi độ dài chuỗi sau khi loại bỏ ký tự escape ngắn hơn buffer đã cấp phát dựa trên độ dài *trước khi* loại bỏ, nhưng một nhánh mã khác lại tính toán độ dài ghi dựa trên chuỗi gốc – gây ra ghi tràn heap có thể kiểm soát được, dẫn tới leo thang đặc quyền lên `root` cho bất kỳ người dùng cục bộ nào, kể cả không có trong `sudoers`. Bài học: lỗi tính toán độ dài buffer (*length calculation bugs*) là một dạng biến thể tinh vi của buffer overflow, khó phát hiện hơn nhiều so với lời gọi hàm nguy hiểm trực tiếp như `strcpy`, và thường *lọt qua* các công cụ quét mẫu đơn giản – minh chứng thực tế cho hạn chế đã nêu ở trên.

2.7 Cơ chế cấp phát bộ nhớ động (Heap Allocator Internals)

2.7.1 Cấu trúc chunk trong glibc malloc

Để hiểu sâu tại sao heap buffer overflow và double-free lại nguy hiểm, cần hiểu allocator quản lý bộ nhớ như thế nào. Trong glibc (ptmalloc2), mỗi vùng nhớ cấp phát được bao bọc bởi một cấu trúc ***chunk*** chứa metadata ngay *trước* vùng dữ liệu trả về cho lập trình viên:



Hình 2.2: Cấu trúc chunk trong glibc malloc (đơn giản hóa)

Lưu ý / Cảnh báo

Ghi tràn ra ngoài vùng dữ liệu (heap buffer overflow) sẽ ghi đè trực tiếp lên trường `size` của chunk *kế tiếp* trong bộ nhớ. Khi chunk đó sau này được `free()`, allocator đọc trường `size` đã bị sửa để quyết định cách hợp nhất (coalesce) với chunk lân cận – kẻ tấn công có thể lợi dụng để ghi giá trị con trỏ tùy ý vào một vị trí bất kỳ trong bộ nhớ (kỹ thuật *unlink exploitation* kinh điển). Đây là lý do các allocator hiện đại (glibc từ 2.3.4, tcmalloc, PartitionAlloc) đều bổ sung các cơ chế kiểm tra tính toàn vẹn metadata (chunk consistency checks) khi thực hiện free/coalesce, làm chậm nhưng không loại bỏ hoàn toàn lớp tấn công này.

2.7.2 Các cơ chế giảm nhẹ ở tầng hệ điều hành/trình biên dịch

Ngoài việc viết mã đúng, các lớp phòng thủ bổ sung (*defense in depth*) giúp giảm mức độ nghiêm trọng ngay cả khi lỗi bộ nhớ vẫn tồn tại:

Bảng 2.6: Các cơ chế giảm nhẹ (mitigation) tầng hệ điều hành/trình biên dịch

Cơ chế	Nguyên lý
Stack Canary	Chèn một giá trị ngẫu nhiên (canary) giữa biến local và return address; kiểm tra giá trị này trước khi hàm return – nếu bị ghi đè bởi buffer overflow, chương trình dừng ngay thay vì thực thi mã độc
ASLR (Address Space Layout Randomization)	Ngẫu nhiên hóa địa chỉ cơ sở của stack, heap, thư viện chia sẻ mỗi lần chạy – khiến kẻ tấn công khó đoán địa chỉ chính xác để nhảy tới (ví dụ trong ROP)
DEP/NX (Data Execution Prevention / No-eXecute)	Đánh dấu các trang bộ nhớ dữ liệu (stack, heap) là không thể thực thi – ngăn kỹ thuật shellcode injection cổ điển (dù không ngăn được ROP vì ROP tái sử dụng mã đã có sẵn)
RELRO (Relocation Read-Only)	Đánh dấu bảng GOT (Global Offset Table) là chỉ đọc sau khi liên kết – ngăn kỹ thuật ghi đè GOT để chiếm quyền điều khiển
Fortify Source (`_FORTIFY_SOURCE`)	Trình biên dịch tự động thay thế các hàm nguy hiểm (`strcpy`, `sprintf`) bằng phiên bản kiểm tra biên khi kích thước buffer đích xác định được tại compile-time

Kiểm soát chất lượng

Các cơ chế trên đều thuộc lớp *giảm nhẹ* (mitigation), không phải lớp *sửa lỗi* (fix) – chúng làm cho việc khai thác một lỗ hổng đã tồn tại trở nên khó khăn/tốn kém hơn, nhưng *không loại bỏ* bản thân lỗ hổng. Chuẩn thực hành công nghiệp (ví dụ hướng dẫn của CISA) luôn nhấn mạnh: mitigation là lớp phòng thủ bổ sung (defense in depth), không thay thế cho việc viết mã đúng ngay từ đầu và kiểm chứng bằng các công cụ đã học ở mục 2.6 và toàn bộ Chương 3.

2.8 Lỗi định dạng chuỗi (Format String Vulnerabilities

Format String Vulnerability

Xảy ra khi chuỗi định dạng (format string) của các hàm họ `printf` được lấy trực tiếp từ dữ liệu do người dùng cung cấp, thay vì một hằng chuỗi cố định. Kẻ tấn công có thể chèn các specifier như `%x` (đọc giá trị từ stack, rò rỉ thông tin) hoặc `%n` (ghi số byte đã in ra vào một địa chỉ con trỏ trên stack – biến đọc thành *ghi* bộ nhớ tùy ý).

```
void log_message(const char *user_input) {
    printf(user_input);        /* SAI: user_input duoc dung lam format string */
}
void log_message_fixed(const char *user_input) {
    printf("%s", user_input);  /* DUNG: user_input luon la DU LIEU, khong phai
        format */
}
```

Listing 2.8: Lỗi format string kinh điển và cách sửa

2.9 Tràn số nguyên (Integer Overflow) chuyên sâu

Tràn số nguyên thường không tự nó gây hại trực tiếp, nhưng lại là *nguyên nhân gốc* dẫn tới buffer overflow khi giá trị đã tràn được dùng làm kích thước cấp phát hoặc chỉ số mảng – lớp lỗi này được gọi là ***Integer Overflow to Buffer Overflow (CWE-190 → CWE-787)***.

```
void process_records(int count, size_t record_size) {
    /* count la int32 do nguoi dung cung cap qua giao thuc mang */
    size_t total = count * record_size;  /* CO THE TRAN neu count, record_size
        lon */
    char *buffer = malloc(total);        /* malloc() voi gia tri da tran, nho hon
         du kien */
    for (int i = 0; i < count; i++) {
        memcpy(buffer + i * record_size, get_record(i), record_size);
        /* GHI VUOT QUA buffer thuc te da cap phat -> heap overflow */
```

```
    }
}
/* Sua dung: kiem tra tran TRUOC khi nhan */
void process_records_fixed(int count, size_t record_size) {
    if (record_size != 0 && (size_t)count > SIZE_MAX / record_size) {
        return;  /* tu choi: phep nhan se tran */
    }
    size_t total = (size_t)count * record_size;
    char *buffer = malloc(total);
    if (!buffer) return;
    /* ... */
}
```

Listing 2.9: Integer overflow dẫn tới heap overflow

2.10 Cấu trúc dữ liệu động nâng cao: Hash Table và Cây

```
#define TABLE_SIZE 256

typedef struct Entry {
    char *key;              /* so huu rieng, duoc strdup() khi insert */
    void *value;
    struct Entry *next;     /* xu ly va cham (collision) bang chaining */
} Entry;

typedef struct {
    Entry *buckets[TABLE_SIZE];
} HashTable;

static unsigned long hash_key(const char *key) {
    unsigned long h = 5381;
    for (; *key; key++) h = ((h << 5) + h) + (unsigned char)*key;
    return h % TABLE_SIZE;
}

int ht_insert(HashTable *ht, const char *key, void *value) {
    unsigned long idx = hash_key(key);
    Entry *e = malloc(sizeof(Entry));
    if (!e) return -1;
    e->key = strdup(key);           /* copy rieng: HashTable so huu key nay */
    if (!e->key) { free(e); return -1; }
    e->value = value;
    e->next = ht->buckets[idx];
    ht->buckets[idx] = e;
    return 0;
}

void ht_free(HashTable *ht) {
```

```
    for (int i = 0; i < TABLE_SIZE; i++) {
        Entry *e = ht->buckets[i];
        while (e) {
            Entry *next = e->next;
            free(e->key);           /* giai phong dung thu tu so huu */
            free(e);
            e = next;
        }
        ht->buckets[i] = NULL;
    }
}
```

Listing 2.10: Bảng băm đơn giản với quản lý ownership rõ ràng

Kiểm soát chất lượng

Ví dụ trên minh họa một nguyên tắc thiết kế quan trọng trong quản lý ownership thủ công: ***"mỗi hàm `xxx_create/xxx_insert` sở hữu một hàm `xxx_free/xxx_destroy` đối xứng"***. Đây là tiền thân trực tiếp của khái niệm *RAII* (Resource Acquisition Is Initialization) trong C++/Rust, nơi trình biên dịch tự động hóa việc gọi hàm hủy đối xứng – điều C *không* làm được, buộc lập trình viên phải tự kỷ luật duy trì tính đối xứng đó trong toàn bộ codebase.

2.11 Case Study lịch sử: Morris Worm (1988) và `gets()`

Case Study: Morris Worm – sâu máy tính đầu tiên lan truyền qua buffer overflow

Năm 1988, Robert Tappan Morris phát tán một chương trình sâu (worm) khai thác lỗ hổng buffer overflow trong daemon `fingerd` trên hệ thống Unix BSD, sử dụng chính xác kỹ thuật ghi tràn stack buffer bằng hàm `gets()` không giới hạn độ dài đầu vào – lỗ hổng đã trình bày ở Bảng hàm nguy hiểm (mục 2.4). Sâu này lây lan tới ước tính 10% số máy chủ kết nối Internet thời điểm đó (khoảng 6000 máy) chỉ trong vài giờ, gây ra sự cố Internet quy mô lớn đầu tiên trong lịch sử. Sự kiện này trực tiếp dẫn tới việc thành lập CERT/CC (Computer Emergency Response Team Coordination Center) tại Đại học Carnegie Mellon – tổ chức ứng phó sự cố an ninh mạng đầu tiên trên thế giới, và là một trong những lý do chính khiến hàm `gets()` chính thức bị loại bỏ khỏi chuẩn ngôn ngữ C (C11), hơn 20 năm sau sự cố.

2.12 Thực hành từng bước: đọc thông báo lỗi AddressSanitizer

Để kết nối lý thuyết heap layout (mục trước) với thực hành, phần này phân tích từng dòng thông báo lỗi thực tế mà ASan in ra khi phát hiện heap buffer overflow.

```c
#include <stdlib.h>
#include <string.h>
int main(void) {
    char *buf = malloc(10);
    strcpy(buf, "0123456789ABCDEF");  /* ghi 17 byte + null vao 10 byte */
    free(buf);
    return 0;
}
```

Listing 2.11: Chương trình gây heap-buffer-overflow để minh họa

```
==12345==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x...
WRITE of size 18 at 0x... thread T0
    #0 in strcpy
    #1 in main example.c:5

0x... is located 0 bytes to the right of 10-byte region
allocated by thread T0 here:
    #0 in malloc
    #1 in main example.c:4
```

Listing 2.12: Thông báo lỗi ASan điển hình (rút gọn) và cách đọc

Kiểm soát chất lượng

Cách đọc thông báo trên theo đúng thứ tự: (1) *loại lỗi* `heap-buffer-overflow` và *thao tác* gây lỗi (`WRITE of size 18`) – khớp chính xác với độ dài chuỗi ”0123456789ABCDEF” cộng byte null-terminator; (2) *stack trace tại thời điểm lỗi* – hàm `strcpy` được gọi từ dòng 5 của `main`; (3) *vị trí tương đối so với vùng nhớ đã cấp phát* – ”0 bytes to the right of 10-byte region” xác nhận chính xác đây là ghi tràn *ngay sau* vùng 10 byte đã `malloc`; (4) *stack trace tại thời điểm cấp phát* – dòng 4, giúp xác định ngay vùng nhớ nào bị tràn mà không cần đoán mò. Ba mảnh thông tin này (loại lỗi, nơi xảy ra, nơi cấp phát) là đủ để sửa lỗi trong hầu hết trường hợp thực tế mà không cần chạy debugger thủ công.

2.13 Tổng kết chương

Tổng kết chương

- Ngôn ngữ C đánh đổi hiệu năng và kiểm soát phần cứng trực tiếp lấy việc không có bảo vệ bộ nhớ tự động; hiểu rõ mô hình bộ nhớ tiến trình (stack/heap/data/text) là nền tảng để phân tích rủi ro.
- Ownership vùng nhớ động là một kỷ luật lập trình chứ không phải cấu trúc ngôn ngữ; vi phạm ownership (double free, leak, dangling pointer) là nguồn gốc của nhiều lỗ hổng nghiêm trọng nhất trong thực tế (CVE-2021-3156, UAF trong trình duyệt).
- Undefined Behavior không chỉ là ”lỗi runtime” mà còn có thể khiến trình biên dịch tối ưu hóa loại bỏ chính các kiểm tra bảo mật mà lập trình viên đã viết – một trong những cạm bẫy nguy hiểm và phản trực giác nhất của C.
- Data structure alignment ảnh hưởng không chỉ tới hiệu năng mà còn có thể gây rò rỉ thông tin qua padding không khởi tạo khi serialize struct trực tiếp.
- Công cụ phân tích tĩnh (cppcheck, clang-tidy) và động (Valgrind, ASan) bổ trợ lẫn nhau: tĩnh phát hiện sớm nhưng có false positive/negative; động chính xác hơn nhưng chỉ phát hiện lỗi trên đường thực thi đã được test tới.

Xu hướng hiện đại

Xu hướng công nghiệp rõ rệt nhất hiện nay là dịch chuyển dần khỏi C/C++ cho các thành phần mới trong hệ thống an toàn cao: Google, Microsoft và nhiều tổ chức chính phủ (CISA – Cybersecurity and Infrastructure Security Agency, Hoa Kỳ) đã công khai khuyến nghị chuyển sang các ngôn ngữ *memory-safe* (Rust, Go, Swift) cho mã mới, viện dẫn thống kê nội bộ cho thấy 60–70% lỗ hổng bảo mật nghiêm trọng trong các codebase C/C++ lớn thuộc lớp lỗi bộ nhớ. Song song đó, các công cụ phân tích tĩnh dựa trên học máy (ví dụ mô hình được huấn luyện trên hàng triệu commit vá lỗi bảo mật) đang được tích hợp vào IDE và CI/CD pipeline để gợi ý sửa lỗi bộ nhớ theo thời gian thực, kết hợp giữa pattern matching cổ điển (như bộ quét trong mục 2.6) và mô hình ngôn ngữ lớn để giảm false positive.

Bài tập

Bài tập tự luận.

1. Vẽ mô hình bộ nhớ tiến trình và chỉ ra chính xác vị trí bị ghi đè trong ví dụ Listing 1.2 nếu `argv[1]` dài 40 ký tự.
2. Giải thích vì sao `unsigned int` overflow *không* phải là undefined behavior trong khi `signed int` overflow là UB. Nêu một hệ quả bảo

mật của sự khác biệt này.

3. Tính `sizeof` của struct sau trên kiến trúc 64-bit và đề xuất cách sắp xếp lại để tối ưu:

```
struct S { char a; double b; char c; short d; };
```

Bài tập lập trình.

4. Mở rộng `DANGEROUS_FUNCTIONS` trong `pattern_scanner.py` thêm ít nhất 5 hàm nguy hiểm khác (tra cứu CWE liên quan), viết test bằng `pytest` với ít nhất một trường hợp false positive đã biết (ví dụ `strcpy` xuất hiện trong comment).
5. Viết một chương trình C có chứa lỗi *use-after-free* có chủ đích, biên dịch với `-fsanitize=address`, và giải thích thông báo lỗi mà AddressSanitizer đưa ra.

CHƯƠNG 3

Phát hiện lỗ hổng phần mềm: Phân tích tĩnh

Mục tiêu chương

Sau khi hoàn thành chương này, sinh viên có khả năng:

- Phân biệt được kiểm chứng (verification) và thẩm định (validation) phần mềm ở mức độ hình thức, và định nghĩa được soundness và completeness của một kỹ thuật phát hiện lỗi.
- Phân biệt được phân tích tĩnh, mô phỏng kiểm thử (testing) và gỡ lỗi (debugging) theo các tiêu chí: chi phí, độ phủ, và loại lỗi phát hiện được.
- Giải bài toán thỏa mãn Boolean (SAT) bằng thuật toán DPLL, và mở rộng sang SMT (Satisfiability Modulo Theories) với lý thuyết bit-vector và mảng.
- Xây dựng và áp dụng Bounded Model Checking (BMC) để kiểm chứng một chương trình C đơn giản bằng cách ”trải phẳng” (unroll) chương trình thành công thức SAT/SMT.
- Giải thích mô hình bộ nhớ (memory model) dùng trong BMC và ảnh hưởng của alignment tới việc mô hình hóa mảng/con trỏ.

3.1 Giới thiệu về kiểm chứng và thẩm định phần mềm

Chương 3 xây dựng nền tảng lý thuyết đứng sau các công cụ đã sử dụng ở Chương 2 (cppcheck, clang-tidy): *làm sao một công cụ tự động có thể ”biết” một chương trình có lỗi hay không, mà không cần chạy nó?* Đây chính là lĩnh vực ***phân tích tĩnh hình thức*** (formal static analysis), khác biệt về bản chất với phân tích tĩnh dựa trên pattern matching đã trình bày ở mục 2.6.

Formal Verification

Kiểm chứng hình thức (formal verification) là quá trình chứng minh hoặc bác bỏ *một cách toán học* rằng một hệ thống thỏa mãn một đặc tả (specification) cho trước, dựa trên một mô hình hình thức của hệ thống đó (ví dụ Kripke Structure ở Chương 1) và logic hình thức để biểu diễn đặc tả (ví dụ LTL/CTL).

3.2 Soundness và Completeness

Đây là hai tiêu chí lý thuyết trung tâm để đánh giá *chất lượng* của bất kỳ kỹ thuật phát hiện lỗi nào – static analyzer, type checker, hay model checker.

Soundness (Tính đúng đắn/chính xác)

Một kỹ thuật là ***sound*** nếu: khi nó báo cáo ”không có lỗi”, thì *chắc chắn* chương trình không có lỗi đó (không bỏ sót – *no false negatives*). Một công cụ sound có thể vẫn báo động giả (false positive), nhưng không bao giờ ”im lặng” khi thực sự có lỗi.

Completeness (Tính đầy đủ)

Một kỹ thuật là ***complete*** nếu: khi nó báo cáo ”có lỗi”, thì *chắc chắn* chương trình thực sự có lỗi đó (không báo động giả – *no false positives*). Một công cụ complete có thể vẫn bỏ sót một số lỗi (false negative), nhưng mọi cảnh báo nó đưa ra đều đúng.

Bảng 3.1: Ma trận Soundness/Completeness và các loại công cụ thực tế

Loại công cụ	Đặc điểm
Sound nhưng không complete	Static analyzer bảo thủ (conservative): không bỏ sót lỗi thật, nhưng có nhiều false positive. Ví dụ: các công cụ phân tích dựa trên *abstract interpretation* chặt.
Complete nhưng không sound	Test-based (kiểm thử động): mọi lỗi tìm được là lỗi thật (chương trình thực sự crash), nhưng không đảm bảo phủ hết mọi trường hợp – không có crash không có nghĩa là không có lỗi.
Không sound, không complete	Phần lớn công cụ heuristic thực tế (cppcheck ở chế độ mặc định): cân bằng giữa hai yếu tố để giữ tốc độ và usability, chấp nhận cả false positive lẫn false negative ở mức độ vừa phải.
Sound và complete	Chỉ khả thi với bài toán *quyết định được* (decidable) và không gian trạng thái *hữu hạn*, ví dụ Bounded Model Checking trong một giới hạn độ sâu k cố định (xem mục 3.5).

Ví dụ

Định lý bất khả quyết của Rice (Rice’s Theorem) chứng minh rằng *không tồn tại* thuật toán nào vừa sound vừa complete cho *mọi* thuộc tính ngữ

nghĩa không tầm thường của chương trình tùy ý (bài toán tổng quát tương đương bài toán dừng – Halting Problem). Đây là lý do căn bản khiến mọi công cụ phân tích tĩnh thực tế đều phải đánh đổi: hoặc chấp nhận false positive (giữ soundness), hoặc chấp nhận false negative (giữ completeness), hoặc giới hạn phạm vi bài toán (bounded, như BMC).

3.3 Sự khác biệt giữa phân tích tĩnh, kiểm thử và gỡ lỗi

Bảng 3.2: So sánh Static Analysis, Testing, Debugging

Tiêu chí	**Static Analysis**	**Testing (Dynamic)**	**Debugging**
Thực thi chương trình?	Không	Có	Có
Thời điểm áp dụng	Trước khi chạy, ngay cả trước khi biên dịch	Sau khi build, cần môi trường chạy	Sau khi phát hiện lỗi (qua testing/production)
Mục tiêu	Tìm lỗi tiềm ẩn trên *mọi* đường thực thi có thể	Xác nhận hành vi đúng/sai trên các input *cụ thể* đã chọn	Xác định *nguyên nhân gốc* của một lỗi đã biết
Độ phủ lý thuyết	Có thể phủ toàn bộ không gian trạng thái (nếu sound)	Giới hạn bởi số test case thực thi	Chỉ xét đúng luồng thực thi gây lỗi đang điều tra
Chi phí	Thấp (tự động, nhanh, không cần input)	Trung bình (cần thiết kế test case, môi trường chạy)	Cao (cần công sức con người, thường thủ công)

Kiểm soát chất lượng

Trong quy trình DevSecOps hiện đại, ba kỹ thuật này được xếp vào các tầng khác nhau của ”Shift-Left Security”: static analysis chạy ngay trong IDE và tại mỗi commit (pre-commit hook, CI pipeline sớm nhất); dynamic testing (bao gồm fuzzing ở Chương 4) chạy ở tầng CI sau khi build thành công; debugging chỉ xảy ra khi có sự cố thực tế (production incident) hoặc khi điều tra kết quả từ hai tầng trên. Chi phí sửa một lỗi tăng theo cấp số nhân qua từng tầng – đây là lý do các tổ chức đầu tư mạnh vào static analysis dù biết trước nó sẽ tạo ra false positive.

3.4 SAT và SMT Solving

3.4.1 Bài toán SAT

Boolean Satisfiability Problem (SAT)

Cho một công thức logic mệnh đề (Boolean formula) φ ở dạng chuẩn hội (CNF – Conjunctive Normal Form), bài toán SAT hỏi: có tồn tại một phép gán giá trị chân lý (True/False) cho các biến sao cho φ được thỏa mãn (đúng) hay không? SAT là bài toán ***NP-complete*** đầu tiên được chứng minh (Định lý Cook-Levin, 1971), nhưng các SAT solver hiện đại giải được các instance thực tế có *hàng triệu* biến nhờ các heuristic mạnh.

Ví dụ

Công thức CNF: $\varphi = (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (\neg x_3)$. Phép gán $x_1 = \text{False}, x_2 = \text{False}, x_3 = \text{False}$ thỏa mãn φ (kiểm tra: mệnh đề 1 đúng vì $\neg x_2$ đúng; mệnh đề 2 đúng vì $\neg x_1$ đúng; mệnh đề 3 đúng vì $\neg x_3$ đúng). Vậy φ là *satisfiable (SAT)*.

3.4.2 Thuật toán DPLL

Thuật toán ***DPLL*** (Davis-Putnam-Logemann-Loveland) là nền tảng của hầu hết SAT solver hiện đại, dựa trên tìm kiếm quay lui (backtracking) kết hợp hai kỹ thuật cắt tỉa:

- ***Unit propagation***: nếu một mệnh đề chỉ còn đúng một literal chưa gán giá trị, gán giá trị đó ngay để mệnh đề đúng (lan truyền dây chuyền).
- ***Pure literal elimination***: nếu một biến chỉ xuất hiện dưới một dạng (luôn dương hoặc luôn âm) trong toàn bộ công thức, gán giá trị làm nó luôn đúng.

Algorithm 2: DPLL(formula φ, assignment α)

Đầu vào: Công thức CNF φ, phép gán bộ phận α
Đầu ra: SAT (kèm phép gán) hoặc UNSAT

if *φ dưới α rỗng (mọi mệnh đề đã thỏa mãn)* **then**
  return SAT với α
if *φ dưới α chứa mệnh đề rỗng (mâu thuẫn)* **then**
  return UNSAT
while *tồn tại unit clause hoặc pure literal* **do**
  áp dụng unit propagation / pure literal elimination, cập nhật α;
chọn biến x chưa gán (theo heuristic, ví dụ VSIDS);
if *DPLL(φ, $\alpha \cup \{x = True\}$) = SAT* **then**
  return SAT
return DPLL(φ, $\alpha \cup \{x = \text{False}\}$);

```
Clause = list[int]      # vi du [1, -2, 3] bieu dien (x1 OR NOT x2 OR x3)
```

```python
Formula = list[Clause]

def unit_propagate(formula: Formula, assign: dict[int, bool]):
    changed = True
    while changed:
        changed = False
        for clause in formula:
            unassigned = [l for l in clause
                          if abs(l) not in assign]
            satisfied = any(
                (l > 0 and assign.get(abs(l))) or
                (l < 0 and assign.get(abs(l)) is False)
                for l in clause if abs(l) in assign
            )
            if satisfied:
                continue
            if len(unassigned) == 0:
                return None            # mau thuan: mot clause rong
            if len(unassigned) == 1:
                lit = unassigned[0]
                assign[abs(lit)] = (lit > 0)
                changed = True
    return assign

def dpll(formula: Formula, assign: dict[int, bool] | None = None):
    assign = dict(assign) if assign else {}
    result = unit_propagate(formula, assign)
    if result is None:
        return None                    # UNSAT tren nhanh nay
    assign = result
    all_vars = {abs(l) for clause in formula for l in clause}
    unassigned = [v for v in all_vars if v not in assign]
    if not unassigned:
        return assign                  # SAT: da gan het bien
    var = unassigned[0]
    for value in (True, False):
        trial = dict(assign); trial[var] = value
        sat = dpll(formula, trial)
        if sat is not None:
            return sat
    return None                        # UNSAT ca hai nhanh
```

Listing 3.1: SAT solver DPLL/CDCL đơn giản – trích `ch3_staticsat/sat_solver.py`

Kiểm soát chất lượng

SAT solver công nghiệp hiện đại (MiniSat, Glucose, CaDiCaL) mở rộng DPLL thành ***CDCL*** (Conflict-Driven Clause Learning): khi gặp mâu thuẫn, thay vì chỉ quay lui đơn giản, solver *phân tích nguyên nhân* mâu thuẫn và học một *mệnh đề mới* (learned clause) để tránh lặp lại chính xác nguyên nhân đó trong tương lai, kết hợp với *non-chronological*

backtracking (nhảy thẳng về điểm quyết định gây mâu thuẫn thay vì lùi từng bước). Đây là lý do CDCL solver hiện đại giải được các instance công nghiệp hàng triệu biến trong khi DPLL thuần túy chỉ khả thi với vài nghìn biến.

3.4.3 Từ SAT đến SMT

Satisfiability Modulo Theories (SMT)

SMT mở rộng SAT bằng cách cho phép các literal không chỉ là biến Boolean mà là các *ràng buộc trong một lý thuyết nền* (background theory) – ví dụ số nguyên tuyến tính, bit-vector có độ rộng cố định (mô hình chính xác kiểu `int`/`long` trong C), mảng (array theory, mô hình hóa bộ nhớ), hoặc uninterpreted functions. SMT solver (Z3, CVC5) kết hợp một SAT solver CDCL ở tầng Boolean với các *theory solver* chuyên biệt (kiến trúc *DPLL(T)*) để quyết định tính thỏa mãn của các ràng buộc lý thuyết.

Ví dụ

Ràng buộc SMT trên lý thuyết bit-vector 8-bit: $x + y = 300 \wedge x < 10$. Với số nguyên toán học thuần túy, đây là bài toán tầm thường (vô số nghiệm với y đủ lớn). Nhưng nếu x, y là `unsigned char` (bit-vector 8-bit, phạm vi $[0, 255]$ với phép cộng *wrap-around modulo 256*), thì $x + y = 300 \equiv x + y = 44 \pmod{256}$ – solver bit-vector chính xác mô hình hóa được hành vi tràn số này, trong khi lý thuyết số nguyên tuyến tính thông thường sẽ cho kết quả sai. Đây chính là lý do BMC cho mã C *bắt buộc* phải dùng lý thuyết bit-vector, không phải số nguyên toán học trừu tượng.

```
"""
Lop wrapper mong, minh hoa cach mot cong cu BMC thuc te (CBMC, ESBMC)
sinh cong thuc SMT tu ma C va goi solver ben ngoai (o day la Z3
qua thu vien z3-solver, neu duoc cai dat).
"""
def check_overflow_smt(bit_width: int, a_val, b_val):
    """
    Sinh dieu kien: co ton tai a, b trong [0, 2^bit_width) sao cho
    a + b (theo phep cong bit-vector) < a  (tran so khong dau)?
    Tra ve (True, model) neu SAT (tim thay truong hop loi),
    hoac (False, None) neu UNSAT (khong the tran so).
    """
    try:
        import z3
    except ImportError:
        raise RuntimeError("Can cai dat goi 'z3-solver' de chay ham nay")
    a = z3.BitVec("a", bit_width)
    b = z3.BitVec("b", bit_width)
    s = z3.Solver()
    s.add(z3.ULT(a + b, a))     # dieu kien tran so khong dau (unsigned)
```

```
    if s.check() == z3.sat:
        m = s.model()
        return True, {"a": m[a].as_long(), "b": m[b].as_long()}
    return False, None
```

Listing 3.2: Wrapper gọi solver SMT thực tế (Z3) – `ch3_staticsat/smt_wrapper.py`

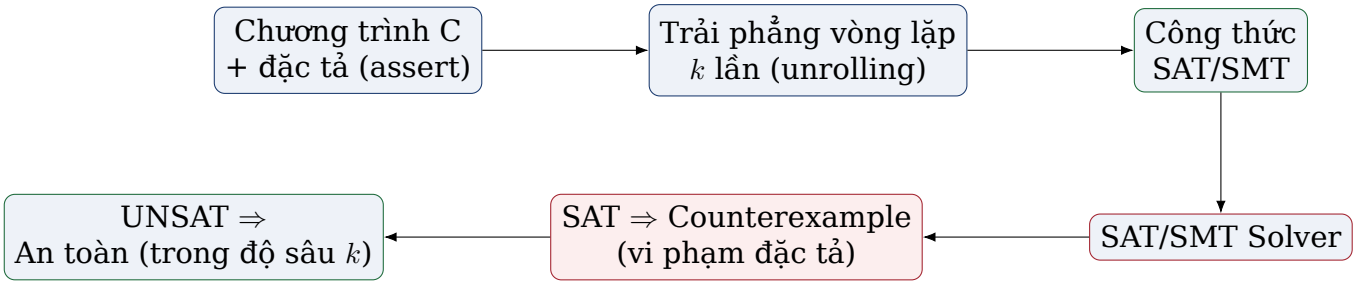
3.5 Kiểm chứng mô hình phần mềm (Bounded Model Checking)

3.5.1 Giới hạn của Model Checking đầy đủ

Model checking đầy đủ (Chương 1) duyệt toàn bộ không gian trạng thái của Kripke Structure để kiểm tra $K \models \varphi$. Giới hạn cơ bản: ***state explosion*** – với chương trình thực tế, không gian trạng thái tăng theo hàm mũ với số biến (mỗi biến 32-bit thêm vào làm không gian trạng thái nhân lên 2^{32} lần), khiến việc duyệt tường minh (explicit-state) hoặc thậm chí duyệt ký hiệu bằng BDD (symbolic model checking cổ điển) không khả thi cho chương trình vừa và lớn.

Bounded Model Checking (BMC)

BMC giới hạn việc tìm kiếm counterexample trong các path có độ dài *tối đa* k (bounded), thay vì tìm trên mọi path có thể (có thể vô hạn). Ý tưởng cốt lõi: ”trải phẳng” (unroll) chương trình k lần thành một công thức logic mệnh đề/SMT duy nhất, sao cho công thức đó ***thỏa mãn được*** *khi và chỉ khi* tồn tại một counterexample có độ dài $\leq k$ vi phạm thuộc tính cần kiểm chứng. Bài toán model checking được quy về bài toán SAT/SMT solving.



Hình 3.1: Quy trình Bounded Model Checking

3.5.2 Ví dụ trải phẳng chương trình

```
int arr[5] = {1, 2, 3, 4, 5};
int access_element(int i) {
    return arr[i];   /* assert ngam dinh: 0 <= i < 5 */
}
```

Listing 3.3: Chương trình C cần kiểm chứng: không được tràn mảng

Với $k = 1$ (một lần gọi hàm), BMC sinh công thức SMT tương đương:

$$\varphi_{\text{BMC}} \equiv (\, i \geq 0 \ \wedge \ i < 5 \,)^{c} \quad \text{(dạng phủ định để tìm vi phạm)}$$

tức là tìm i sao cho $\neg(0 \leq i < 5)$, nghĩa là $i < 0 \vee i \geq 5$. Nếu solver trả về SAT với model $i = 5$ (hoặc $i = -1$), đó chính là ***counterexample*** cụ thể chứng minh chương trình có lỗi tràn mảng. Nếu công thức UNSAT, chương trình *an toàn trong giới hạn được kiểm tra* – lưu ý BMC chỉ đảm bảo *sound trong giới hạn* k, không đảm bảo cho $k' > k$ (đây là điểm khác biệt căn bản với model checking đầy đủ).

```python
def check_array_bounds(index_range: range, array_size: int) -> tuple[bool, int | None]:
    """
    Mo phong BMC don gian: duyet moi gia tri co the co cua index
    (thay the cho viec sinh cong thuc SMT day du) va tra ve
    counterexample dau tien vi pham 0 <= i < array_size.
    Trong cong cu thuc te (CBMC, ESBMC), buoc nay duoc thuc hien
    bang mot SMT solver thay vi vet can bang tay, de mo rong duoc
    voi index la bieu thuc phuc tap thay vi hang so.
    """
    for i in index_range:
        if not (0 <= i < array_size):
            return True, i          # tim thay counterexample
    return False, None              # UNSAT trong pham vi da xet: an toan
```

Listing 3.4: Bounded Model Checker thu nhỏ dùng SAT solver – trích `ch3_staticsat/bounded_checker.py`

Case Study: CBMC và ESBMC trong công nghiệp

CBMC (C Bounded Model Checker, Đại học Oxford/Diffblue) và **ESBMC** (Efficient SMT-based BMC, Đại học Manchester/Federal University Bahia) là hai công cụ BMC mã nguồn mở được dùng rộng rãi để kiểm chứng mã C/C++ trong công nghiệp, đặc biệt trong lĩnh vực hệ thống nhúng an toàn cao (automotive theo ISO 26262, hàng không theo DO-178C). Cả hai đều dịch chương trình C thành công thức SMT (dùng backend Z3, Boolector, hoặc MathSAT), tự động kiểm tra các thuộc tính ngầm định (buffer overflow, division by zero, integer overflow, assertion vi phạm, pointer safety) mà không cần lập trình viên viết đặc tả thủ công cho từng lỗi cơ bản. Amazon Web Services cũng phát triển công cụ tương tự để kiểm chứng hình thức các thành phần lõi của hệ thống ảo hóa Firecracker, chứng minh không có buffer overflow trong toàn bộ codebase C quan trọng trước khi release.

3.6 Mô hình bộ nhớ (Memory Model)

Để BMC có thể suy luận chính xác về con trỏ và mảng, chương trình C cần được mô hình hóa dưới dạng công thức mà một SMT solver hiểu được.

Cách tiếp cận phổ biến nhất là ***lý thuyết mảng*** (Theory of Arrays) kết hợp bit-vector.

Theory of Arrays trong SMT

Bộ nhớ được mô hình hóa như một hàm (mảng logic) từ địa chỉ (bit-vector) tới giá trị (bit-vector), với hai phép toán tiên đề hóa: `select(mem, addr)` (đọc giá trị tại địa chỉ) và `store(mem, addr, val)` (ghi giá trị, trả về một mảng *mới* — do SMT theory of arrays có ngữ nghĩa *functional/immutable*). Tiên đề cốt lõi: select(store$(mem, a, v), a) = v$ (đọc lại đúng vị trí vừa ghi cho ra đúng giá trị vừa ghi).

3.6.1 Sắp xếp mô hình bộ nhớ (Aligned Memory Model)

Khi mô hình hóa một struct có padding (mục 2.6 Chương 2) trong SMT, BMC cần biểu diễn *chính xác* offset thực tế của từng trường sau khi trình biên dịch chèn padding, *không phải* tổng đơn giản kích thước từng trường. Nếu công cụ BMC dùng sai offset (bỏ qua alignment), nó có thể bỏ sót lỗi thật (đọc nhầm sang trường khác) hoặc báo động giả trên một chương trình thực chất an toàn – đây là lý do các công cụ BMC công nghiệp (CBMC) phải mô phỏng *chính xác* ABI (Application Binary Interface) của trình biên dịch/kiến trúc đích, bao gồm quy tắc alignment và padding, chứ không chỉ mô hình hóa ngữ nghĩa C trừu tượng.

Kiểm soát chất lượng

Tiêu chuẩn công nghiệp cho kiểm chứng hình thức trong các lĩnh vực an toàn cao (DO-178C mức A cho hàng không, ISO 26262 ASIL-D cho ô tô, IEC 61508 cho hệ thống công nghiệp) đều liệt kê *formal methods / model checking* như một kỹ thuật được khuyến nghị mạnh (*highly recommended*) cho các thành phần an toàn cao nhất, đặc biệt để chứng minh *không có* các lớp lỗi runtime cụ thể (buffer overflow, division by zero, integer overflow) – chính xác là những gì BMC được thiết kế để làm.

3.7 Symbolic Model Checking bằng BDD

Trước khi BMC (dựa trên SAT/SMT) trở nên phổ biến từ cuối thập niên 1990, kỹ thuật ***symbolic model checking*** cổ điển (Ken McMillan, đầu thập niên 1990) đã giải quyết một phần vấn đề state explosion bằng cách biểu diễn *tập hợp* trạng thái (thay vì từng trạng thái riêng lẻ) dưới dạng ***Binary Decision Diagram (BDD)*** – một cấu trúc đồ thị có hướng không chu trình biểu diễn hàm Boolean một cách nén (canonical, chia sẻ các nhánh con giống nhau).

Binary Decision Diagram (BDD)

BDD biểu diễn một hàm Boolean $f : \{0,1\}^n \to \{0,1\}$ dưới dạng cây quyết định nhị phân đã được *rút gọn* (loại bỏ node dư thừa, chia sẻ node con

giống nhau, cố định thứ tự biến), cho một dạng biểu diễn *canonical* (duy nhất) với thứ tự biến cho trước – nghĩa là hai hàm Boolean tương đương logic luôn có cùng một BDD, giúp việc kiểm tra tương đương trở thành so sánh con trỏ.

Bảng 3.3: So sánh Symbolic Model Checking (BDD) và Bounded Model Checking (SAT/SMT)

Tiêu chí	**BDD-based (cổ điển)**	**SAT/SMT-based BMC**
Độ đầy đủ	Đầy đủ (unbounded): có thể chứng minh đúng cho mọi độ dài path	Chỉ đúng trong giới hạn k (trừ khi kết hợp k-induction)
Độ nhạy với thứ tự biến	Rất nhạy: thứ tự biến kém có thể khiến BDD bùng nổ kích thước theo hàm mũ	Không có khái niệm thứ tự biến tương tự
Khả năng mở rộng thực tế	Tốt cho mạch phần cứng (hardware), kém hơn cho phần mềm có số nguyên/con trỏ phức tạp	Tốt hơn cho phần mềm nhờ lý thuyết bit-vector/mảng mô hình hóa trực tiếp ngữ nghĩa C
Công cụ tiêu biểu	SMV, NuSMV	CBMC, ESBMC, SAGE (dynamic symbolic)

3.8 Trừu tượng hóa (Abstract Interpretation)

Abstract Interpretation

Một khung lý thuyết (Patrick và Radhia Cousot, 1977) cho phép xây dựng các static analyzer *sound theo thiết kế* bằng cách thực thi chương trình trên một ***miền trừu tượng*** (abstract domain) thay vì miền giá trị cụ thể – ví dụ thay vì theo dõi giá trị chính xác của một biến nguyên ($x = 7$), chỉ theo dõi *khoảng giá trị có thể* ($x \in [0, 100]$) hoặc dấu của nó ($x > 0$). Mọi phép toán trên chương trình gốc được ánh xạ sang một phép toán tương ứng trên miền trừu tượng, đảm bảo tính chất *sound*: nếu phân tích trừu tượng kết luận ”không thể xảy ra lỗi”, điều đó đúng cho *mọi* thực thi cụ thể có thể có.

Ví dụ

Miền trừu tượng *Interval* theo dõi $x \in [lo, hi]$. Với đoạn mã `y = x + 5;` nếu $x \in [0, 10]$ thì phép cộng trừu tượng suy ra ngay $y \in [5, 15]$ mà không cần liệt kê từng giá trị cụ thể của x. Công cụ công nghiệp **Astrée** (dùng miền trừu tượng phức tạp hơn nhiều: octagon, polyhedra) đã được dùng để chứng minh *không có* runtime error (bao gồm mọi buffer overflow, division by zero) trong toàn bộ phần mềm điều khiển bay Airbus A380 – một trong những thành tựu ứng dụng thực tế lớn nhất của abstract interpretation trong công nghiệp hàng không.

3.9 K-Induction: mở rộng BMC thành chứng minh không giới hạn

Một hạn chế cố hữu của BMC "thuần túy" là kết quả UNSAT tại độ sâu k chỉ chứng minh *an toàn trong k bước*, không chứng minh an toàn mãi mãi. ***K-induction*** là kỹ thuật kết hợp BMC với nguyên lý quy nạp toán học để đạt được chứng minh *unbounded* (không giới hạn) mà vẫn dùng nền tảng SAT/SMT solving.

K-Induction

Gồm hai bước:

1. **Base case**: dùng BMC thông thường để chứng minh thuộc tính φ đúng cho k bước đầu tiên từ trạng thái khởi tạo.
2. **Inductive step**: chứng minh rằng *nếu* φ đúng tại k bước liên tiếp bất kỳ (không nhất thiết từ trạng thái khởi tạo), *thì* φ cũng đúng ở bước thứ $k+1$ tiếp theo – bằng cách hỏi SAT solver: "có tồn tại một dãy $k+1$ trạng thái sao cho φ đúng ở k trạng thái đầu nhưng sai ở trạng thái cuối không?" Nếu UNSAT, quy nạp thành công, và φ được chứng minh đúng *mãi mãi* (unbounded).

Kiểm soát chất lượng

Công cụ công nghiệp như CBMC hỗ trợ trực tiếp cờ `--k-induction`, tự động tăng dần k cho tới khi tìm được một k đủ lớn để bước quy nạp thành công (nếu thuộc tính thực sự đúng), hoặc tìm ra counterexample cụ thể (nếu thuộc tính sai) – kết hợp được cả tốc độ thực dụng của BMC lẫn độ tin cậy toán học của model checking đầy đủ, mà không cần xây dựng BDD tốn kém như phương pháp cổ điển.

3.10 Thực hành SMT solving nâng cao với Z3

```
import z3

def verify_no_overflow_after_write(arr_size: int, write_idx_range: range):
```

```
    """
    Minh hoa Theory of Arrays: mo hinh mot mang co kich thuoc co dinh,
    kiem tra co ton tai chi so ghi nam ngoai bien khong.
    """
    idx = z3.Int("idx")
    s = z3.Solver()
    s.add(idx >= write_idx_range.start, idx < write_idx_range.stop)
    s.add(z3.Or(idx < 0, idx >= arr_size))   # dieu kien VI PHAM bien
    if s.check() == z3.sat:
        m = s.model()
        return True, m[idx].as_long()        # tim thay chi so vi pham
    return False, None

# Vi du su dung: mang kich thuoc 10, chi so du kien trong [5, 15)
found, bad_idx = verify_no_overflow_after_write(10, range(5, 15))
# found = True, bad_idx co the la mot gia tri >= 10 (vi du 10, 11, 12, 13, 14)
```

Listing 3.5: Mô hình hóa bộ nhớ mảng bằng Theory of Arrays trong Z3

3.11 Case Study: Kiểm chứng hình thức hạt nhân hệ điều hành seL4

Case Study: seL4 – hạt nhân hệ điều hành được chứng minh hình thức đầy đủ

seL4 (một biến thể của họ vi nhân L4) là hạt nhân hệ điều hành đầu tiên trên thế giới được chứng minh hình thức *đầy đủ* (end-to-end formal proof) rằng cài đặt bằng mã C của nó *tuân thủ chính xác* đặc tả trừu tượng cấp cao, và đặc tả đó *đảm bảo* các thuộc tính an toàn bộ nhớ (không có buffer overflow, không có null-pointer dereference, không có undefined behavior) cũng như các thuộc tính bảo mật cấp cao (tính toàn vẹn, cách ly tiến trình). Dự án do nhóm nghiên cứu NICTA/Data61 (Úc) thực hiện, sử dụng trợ lý chứng minh Isabelle/HOL, với tổng cộng khoảng 200.000 dòng chứng minh hình thức cho khoảng 10.000 dòng mã C – tỷ lệ minh chứng cho chi phí cực lớn của kiểm chứng hình thức đầy đủ so với BMC (vốn chỉ kiểm tra trong giới hạn k nhưng rẻ hơn nhiều bậc). seL4 hiện được sử dụng trong các hệ thống an toàn cao thực tế: hệ thống tránh va chạm máy bay không người lái của Boeing, và các thiết bị quân sự yêu cầu đảm bảo an toàn tuyệt đối. Đây là minh chứng cho việc lựa chọn kỹ thuật kiểm chứng (BMC giới hạn hay full formal proof) luôn là một bài toán đánh đổi giữa chi phí và mức độ đảm bảo cần thiết cho từng loại hệ thống.

3.12 Thực hành từng bước: giải DPLL bằng tay với truy vết đầy đủ

Phần này truy vết chi tiết thuật toán DPLL (Listing 3.1) trên công thức $\varphi = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3)$, bổ sung cho phần lý thuyết ở mục 3.4.

Bảng 3.4: Truy vết DPLL từng bước trên φ

Bước	Hành động	Phép gán hiện tại	Trạng thái mệnh đề
1	Không có unit clause; chọn nhánh $x_1{=}True$	$x_1{=}T$	Mệnh đề 1 thỏa (do x_1); mệnh đề 2 rút gọn còn (x_3) – **unit clause mới**
2	Unit propagation: $(x_3) \Rightarrow x_3{=}True$	$x_1{=}T, x_3{=}T$	Mệnh đề 3 rút gọn còn $(\neg x_2)$ – **unit clause mới**
3	Unit propagation: $(\neg x_2) \Rightarrow x_2{=}False$	$x_1{=}T, x_2{=}F, x_3{=}T$	Mọi mệnh đề đã thỏa mãn
4	Không còn biến chưa gán	–	**SAT**: trả về $\{x_1{=}T, x_2{=}F, x_3{=}T\}$

Điểm mấu chốt cần quan sát: quyết định thử $x_1{=}True$ ở bước 1 *lập tức kích hoạt* một chuỗi unit propagation liên tiếp (bước 2, 3) mà không cần thêm bất kỳ quyết định (backtracking point) nào khác – đây chính là tình huống lý tưởng nhất cho DPLL, khi *constraint propagation* một mình giải quyết được toàn bộ công thức sau một quyết định duy nhất. Với công thức lớn hơn trong thực tế công nghiệp (hàng triệu biến), CDCL đạt hiệu quả bằng cách tối đa hóa số lần ”chuỗi domino” như thế này xảy ra thông qua heuristic chọn biến quyết định thông minh (VSIDS – Variable State Independent Decaying Sum, ưu tiên biến xuất hiện trong các mệnh đề xung đột gần đây).

Hoạt động trên lớp

Lặp lại truy vết trên cho công thức $\varphi' = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_3)$ (thêm một mệnh đề mới). Xác định tại bước nào xảy ra mâu thuẫn (clause rỗng) buộc thuật toán phải quay lui (backtrack) sang nhánh $x_1{=}False$, và công thức cuối cùng là SAT hay UNSAT.

3.13 Tổng kết chương

Tổng kết chương

- Soundness (không bỏ sót lỗi thật) và completeness (không báo động giả) là hai tiêu chí lý thuyết đối lập nhau; Định lý Rice chứng minh không công cụ nào đạt được cả hai cho mọi thuộc tính tổng quát – mọi công cụ thực tế đều phải đánh đổi hoặc giới hạn phạm vi bài toán.
- Phân tích tĩnh, kiểm thử động và debugging là ba kỹ thuật bổ trợ, không thay thế lẫn nhau, và nên được áp dụng ở các tầng khác nhau của quy trình phát triển (shift-left security).
- SAT là bài toán NP-complete nền tảng; thuật toán DPLL (unit propagation + pure literal + backtracking) và mở rộng CDCL (conflict-driven clause learning) là cơ sở của mọi SAT solver hiện đại.
- SMT mở rộng SAT với các lý thuyết nền (bit-vector, array), cho phép mô hình hóa chính xác ngữ nghĩa số học và bộ nhớ của ngôn ngữ C, đặc biệt là hành vi tràn số kiểu wrap-around.
- Bounded Model Checking quy bài toán kiểm chứng chương trình về bài toán SAT/SMT bằng cách trải phẳng (unroll) vòng lặp/đệ quy tới độ sâu k; đây là kỹ thuật thực tế mạnh mẽ nhất được dùng trong các công cụ công nghiệp (CBMC, ESBMC) để chứng minh không có buffer overflow, division by zero, hay integer overflow trong mã C/C++.

Xu hướng hiện đại

SMT solver hiện đại (Z3 của Microsoft Research, CVC5) ngày càng được tích hợp sâu vào công cụ phát triển phần mềm chủ đạo: từ kiểm tra tính nhất quán của cấu hình hạ tầng dạng mã (Infrastructure as Code), xác minh smart contract (công cụ dùng symbolic execution kết hợp SMT để tìm lỗi reentrancy), đến kiểm chứng thuật toán mật mã học của các thư viện mã nguồn mở lớn. Một xu hướng nghiên cứu mới nổi là kết hợp mô hình ngôn ngữ lớn (LLM) với SMT solver: LLM được dùng để tự động sinh *giả thuyết bất biến* (loop invariants) hoặc đặc tả (assertions) từ mã nguồn, sau đó SMT solver xác minh hình thức tính đúng đắn của giả thuyết đó – kết hợp khả năng ”hiểu ý định” của LLM với độ chính xác

toán học tuyệt đối của SMT.

Bài tập

Bài tập tự luận.

1. Cho công thức CNF $(x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3) \wedge (x_1 \vee \neg x_3)$. Chạy thuật toán DPLL bằng tay (vẽ cây quyết định), xác định công thức là SAT hay UNSAT.
2. Giải thích vì sao lý thuyết số nguyên tuyến tính (Linear Integer Arithmetic) không đủ để mô hình hóa chính xác phép cộng `uint8_t` trong C; đề xuất lý thuyết SMT phù hợp hơn.
3. Với chương trình ở Listing 3.3, viết công thức SMT đầy đủ (dùng ký hiệu toán học) cho trường hợp $k = 2$ (hai lần gọi `access_element` với hai giá trị `i1`, `i2` độc lập).

Bài tập lập trình.

4. Cài đặt CDCL cơ bản (thêm clause learning đơn giản) mở rộng từ `dpll()` ở Listing 3.1; so sánh số lần backtrack với DPLL thuần túy trên cùng một bộ test.
5. Sử dụng `check_array_bounds` ở Listing 3.4 để kiểm tra một mảng kích thước 10 với index được tính từ biểu thức `i = (a * 3 + b) % 12`; xác định counterexample nếu có.

CHƯƠNG 4

Phát hiện lỗ hổng phần mềm: Phân tích động

Mục tiêu chương

Sau khi hoàn thành chương này, sinh viên có khả năng:

- Phân biệt được kiểm thử bảo mật (security testing) với kiểm thử chức năng (functional testing) về mục tiêu, oracle, và phương pháp.
- Đo lường và diễn giải được các chỉ số code coverage (statement, branch, path coverage) và hiểu giới hạn *path explosion*.
- Cài đặt được một bộ theo dõi thực thi chương trình (execution tracing) đơn giản và giải thích khái niệm taint tracking.
- Xây dựng được một black-box fuzzer coverage-guided cơ bản và giải thích ”magic byte problem”.
- Xây dựng được một white-box fuzzer sử dụng symbolic/concolic execution kết hợp SMT solving (Chương 3) để giải quyết magic byte problem.

4.1 Giới thiệu về kiểm thử bảo mật

4.1.1 Security Testing khác Functional Testing ở đâu?

Bảng 4.1: Kiểm thử chức năng vs. Kiểm thử bảo mật

Tiêu chí	Testing chức năng	Security Testing
Mục tiêu	Xác nhận hành vi *đúng theo đặc tả*	Tìm hành vi *ngoài đặc tả* có thể bị khai thác
Oracle (tiêu chí đánh giá)	So sánh output thực tế với output mong đợi đã biết trước	Thường chỉ có oracle ”yếu”: crash/không crash, vi phạm bất biến bộ nhớ (qua sanitizer)
Input được thiết kế bởi	Người thiết kế test dựa trên yêu cầu nghiệp vụ	Kẻ tấn công tiềm năng – bao gồm cả input ”không ai nghĩ tới”
Độ phủ mong muốn	Phủ các luồng nghiệp vụ chính (happy path + edge case đã biết)	Phủ *toàn bộ không gian input có thể*, kể cả input dị dạng

Fuzzing (Kiểm thử mờ)

Fuzzing là kỹ thuật kiểm thử động tự động sinh (thường là ngẫu nhiên hoặc có hướng dẫn) một lượng lớn input, đưa vào chương trình mục tiêu, và quan sát các dấu hiệu bất thường (crash, treo, vi phạm assertion, vi phạm bất biến bộ nhớ phát hiện bởi sanitizer) mà *không* cần biết trước output mong đợi chính xác cho từng input – oracle đơn giản nhất chỉ là ”chương trình có crash hay không”.

4.2 Phủ mã nguồn (Code Coverage)

4.2.1 Các mức độ coverage

Bảng 4.2: Các mức độ code coverage

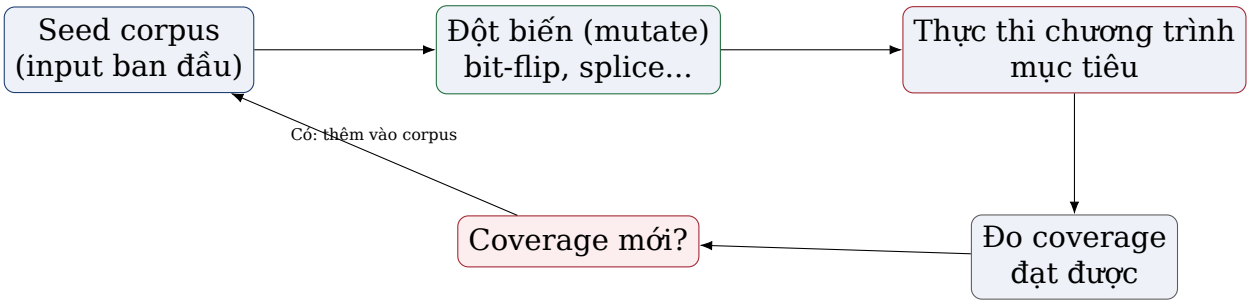
Loại coverage	Định nghĩa
Statement/Line coverage	Tỷ lệ dòng lệnh đã được thực thi ít nhất một lần
Branch coverage	Tỷ lệ nhánh rẽ (true/false của mỗi điều kiện) đã được thực thi cả hai phía
Path coverage	Tỷ lệ *đường thực thi hoàn chỉnh* (tổ hợp các nhánh) đã được thực thi

Lưu ý / Cảnh báo

Path coverage: bao phủ mọi đường thực thi có thể có – lý tưởng nhưng thường ***bất khả thi*** (*path explosion*): với n điều kiện rẽ nhánh độc lập tuần tự, số đường thực thi có thể lên tới 2^n. Một hàm chỉ 20 câu lệnh `if` độc lập đã có hơn 1 triệu đường thực thi khác nhau – vượt xa khả năng liệt kê tường minh trong thời gian hợp lý.

4.2.2 Coverage-guided Fuzzing

Ý tưởng đột phá của các fuzzer hiện đại (AFL, AFL++, libFuzzer) là dùng *code coverage làm tín hiệu phản hồi (feedback)* để hướng dẫn quá trình sinh input, thay vì sinh input hoàn toàn ngẫu nhiên (*dumb fuzzing*):



Hình 4.1: Vòng lặp coverage-guided fuzzing

Nếu một input đột biến đạt được coverage *mới* (đi qua một nhánh mã chưa từng thấy trước đây), nó được thêm vào ***seed corpus*** để tiếp tục đột biến từ đó – cơ chế này giúp fuzzer ”khám phá” dần các phần sâu của chương trình một cách có định hướng, hiệu quả hơn hẳn random fuzzing thuần túy.

```python
class CoverageTracker:
    """Theo dõi tap cac 'edge' (cap lenh lien tiep) da duoc thuc thi,
    mo phong co che edge coverage cua AFL o muc do don gian hoa."""

    def __init__(self):
        self.visited_edges: set[tuple[int, int]] = set()
        self.last_location = 0

    def hit(self, location: int) -> bool:
        """Goi tai moi 'basic block'; tra ve True neu day la edge MOI."""
        edge = (self.last_location, location)
        self.last_location = location
        is_new = edge not in self.visited_edges
        self.visited_edges.add(edge)
        return is_new

    def coverage_count(self) -> int:
        return len(self.visited_edges)

    def merge(self, other: "CoverageTracker") -> int:
        """Hop nhat coverage, tra ve so edge moi duoc them vao."""
        before = len(self.visited_edges)
        self.visited_edges |= other.visited_edges
```

```
24          return len(self.visited_edges) - before
```

Listing 4.1: Bộ theo dõi coverage đơn giản – trích `ch4_dynafuzz/coverage_tracker.py`

4.3 Theo dõi thực thi chương trình

4.3.1 Execution Tracing và Taint Tracking

Taint Tracking (Theo dõi lan truyền dữ liệu bẩn)

Taint tracking là kỹ thuật đánh dấu (tag) dữ liệu bắt nguồn từ nguồn không tin cậy (input người dùng, mạng, file) là "bẩn" (*tainted*), sau đó theo dõi sự lan truyền của nhãn "bẩn" đó qua các phép toán và gán biến trong suốt quá trình thực thi. Nếu dữ liệu bẩn chảy tới một điểm nhạy cảm (*sink*) mà không qua kiểm tra/thanh lọc (*sanitization*) – ví dụ chảy trực tiếp vào một câu lệnh SQL hoặc một chỉ số mảng – đó là dấu hiệu của một lỗ hổng injection hoặc buffer overflow tiềm ẩn.

Ví dụ

Luồng dữ liệu: `input_data` (nguồn: đọc từ socket, *tainted*) $\rightarrow$ `parsed_length = atoi(input_data)` (vẫn tainted, vì phép toán trên dữ liệu bẩn tạo ra dữ liệu bẩn) $\rightarrow$ `memcpy(buffer, data, parsed_length)` (*sink*: `parsed_length` bẩn được dùng trực tiếp làm kích thước copy mà không kiểm tra biên – đây chính là mẫu lỗi gốc rễ của Heartbleed đã trình bày ở Chương 1).

4.3.2 Magic Byte Problem

Lưu ý / Cảnh báo

Magic byte problem: nhiều định dạng file/giao thức yêu cầu một chuỗi byte "ma thuật" cố định ở đầu (ví dụ header `PNG` bắt đầu bằng 8 byte cố định, hoặc một checksum/magic number để xác thực định dạng). Random fuzzing gần như *không bao giờ* tự sinh ngẫu nhiên đúng chuỗi byte đó (xác suất $1/256^n$ với n byte magic), nên fuzzer bị "mắc kẹt" mãi mãi ở bước kiểm tra magic byte đầu tiên, không bao giờ đi sâu vào logic parser thực sự bên trong – nơi phần lớn lỗ hổng thực sự tồn tại.

4.4 Tạo ra các đường thực thi của chương trình

Để giải quyết magic byte problem và khám phá sâu hơn không gian đường thực thi, hai chiến lược chính được sử dụng, tương ứng với hai mục tiếp theo: black-box fuzzing thông minh hóa bằng coverage feedback (đã nêu ở mục 4.1, không cần hiểu logic bên trong) và white-box fuzzing dùng phân tích symbolic để giải ngược điều kiện nhánh (hiểu logic bên trong ở mức công thức toán học).

4.5 Kiểm thử hộp đen (Black-box Fuzzing)

Black-box Fuzzing

Fuzzer coi chương trình mục tiêu là một ”hộp đen”: chỉ quan sát input/output (và coverage nếu có instrumentation nhẹ, như AFL), *không* phân tích mã nguồn hay logic bên trong để quyết định cách sinh input tiếp theo – việc sinh input dựa hoàn toàn trên đột biến (mutation) từ seed corpus và tín hiệu coverage phản hồi.

```python
import random

def mutate(data: bytes) -> bytes:
    """Dot bien don gian: flip 1-4 bit ngau nhien trong buffer."""
    data = bytearray(data)
    if not data:
        return bytes(data)
    n_flips = random.randint(1, min(4, len(data)))
    for _ in range(n_flips):
        idx = random.randrange(len(data))
        bit = 1 << random.randrange(8)
        data[idx] ^= bit
    return bytes(data)

def fuzz_loop(target_fn, seed_corpus: list[bytes], iterations: int,
              tracker_factory):
    """
    target_fn(data, tracker) -> None; co the nem CrashError.
    Tra ve (corpus_moi, danh_sach_crash).
    """
    corpus = list(seed_corpus)
    crashes = []
    global_tracker = tracker_factory()
    for _ in range(iterations):
        base = random.choice(corpus)
        candidate = mutate(base)
        local_tracker = tracker_factory()
        try:
            target_fn(candidate, local_tracker)
        except Exception as exc:                    # "crash" duoc bat lai
            crashes.append((candidate, str(exc)))
            continue
        if global_tracker.merge(local_tracker) > 0:
            corpus.append(candidate)                # coverage moi: giu lai
    return corpus, crashes
```

Listing 4.2: Black-box fuzzer coverage-guided đơn giản – trích `ch4_dynafuzz/blackbox_fuzzer.py`

Case Study: AFL và Heartbleed

American Fuzzy Lop (AFL), công cụ coverage-guided fuzzer nổi tiếng của Michal Zalewski, đã chứng minh khả năng tự động tìm lại được lỗi tương tự lớp Heartbleed trên nhiều thư viện phân tích định dạng nhị phân chỉ trong vài giờ chạy trên phần cứng thông thường, mà không cần biết trước bất kỳ thông tin gì về cấu trúc lỗi – chỉ cần một bộ seed input hợp lệ ban đầu (ví dụ vài gói tin TLS hợp lệ) và một bản build có instrumentation coverage. Thành công này đã khởi động làn sóng đầu tư "fuzzing liên tục" (*continuous fuzzing*) trong công nghiệp, tiêu biểu là dự án OSS-Fuzz của Google, chạy fuzzer 24/7 trên hàng nghìn dự án mã nguồn mở quan trọng và đã phát hiện hàng chục nghìn lỗi kể từ khi ra mắt năm 2016.

4.6 Kiểm thử hộp trắng (White-box Fuzzing)

4.6.1 Symbolic Execution và Concolic Execution

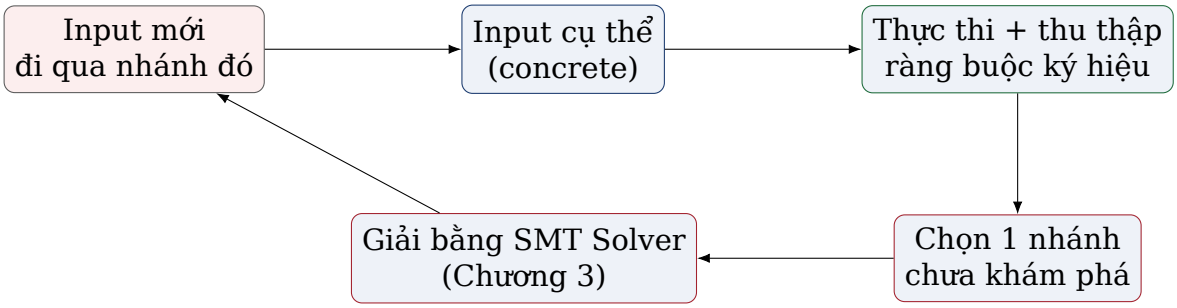
Symbolic Execution

Thay vì thực thi chương trình với giá trị input cụ thể, symbolic execution thực thi chương trình với các *biến ký hiệu* (symbolic variables) đại diện cho input chưa biết. Tại mỗi điểm rẽ nhánh điều kiện (`if`), công cụ ghi lại ràng buộc (constraint) trên các biến ký hiệu cho cả hai nhánh, và dùng SMT solver (Chương 3) để kiểm tra ràng buộc mỗi nhánh có thỏa mãn được không – nếu có, sinh ra một giá trị input cụ thể (*concrete input*) sẽ đi theo đúng nhánh đó.

Concolic Execution (CONCrete + symbOLIC)

Kết hợp thực thi cụ thể (concrete, với giá trị input thật) *song song* với thu thập ràng buộc ký hiệu dọc theo đúng đường thực thi cụ thể đó. Ưu điểm so với symbolic execution thuần túy: tránh được vấn đề *path explosion* bùng nổ tổ hợp vì chỉ theo dõi *một* đường thực thi tại một thời điểm, đồng thời tránh được giới hạn của symbolic execution thuần túy khi gặp lời gọi thư viện ngoài không thể mô hình hóa ký hiệu (dùng luôn giá trị cụ thể trong trường hợp đó).

4.6.2 Giải quyết Magic Byte Problem bằng White-box Fuzzing



Hình 4.2: Vòng lặp concolic execution giải quyết magic byte problem

Với magic byte problem (ví dụ điều kiện `if (header[0]==0x89 && header[1]=='P' && ...)`), symbolic/concolic execution thu thập chính xác ràng buộc đẳng thức đó, đưa cho SMT solver, và solver trả về *ngay lập tức* giá trị byte chính xác thỏa mãn – thay vì phải chờ random mutation ”may mắn” trúng đúng chuỗi byte đó.

```python
def solve_magic_bytes(magic: bytes) -> bytes:
    """
    Minh hoa nguyen ly cot loi cua white-box fuzzing (vi du KLEE,
    Driller, SAGE): dung SMT solver de tim input thoa man dieu kien
    nhanh re, thay vi doan ngau nhien.
    """
    import z3
    n = len(magic)
    input_bytes = [z3.BitVec(f"b{i}", 8) for i in range(n)]
    s = z3.Solver()
    for sym_byte, concrete_byte in zip(input_bytes, magic):
        s.add(sym_byte == concrete_byte)     # rang buoc thu thap tu nhanh if
    assert s.check() == z3.sat
    m = s.model()
    return bytes(m[b].as_long() for b in input_bytes)
```

Listing 4.3: White-box fuzzer tối giản dùng Z3 để vượt qua magic byte check

Kiểm soát chất lượng

Công cụ white-box fuzzing công nghiệp tiêu biểu: **KLEE** (LLVM-based symbolic execution, Đại học Stanford/Imperial College), **Driller** (kết hợp AFL với symbolic execution chỉ khi black-box fuzzer bị ”mắc kẹt”, giành giải DARPA Cyber Grand Challenge 2016), và **SAGE** (Microsoft, whitebox fuzzing quy mô lớn dùng nội bộ để kiểm thử hàng trăm sản phẩm Windows/Office trước khi phát hành, được công bố đã tiết kiệm hàng triệu đô la chi phí vá lỗi sau phát hành). Xu hướng chung: *kết hợp* black-box (nhanh, rẻ, phủ rộng) với white-box (chậm, đắt, nhưng vượt qua được các rào cản như magic byte) theo tỷ lệ linh hoạt – thay vì chọn một trong hai một cách tuyệt đối.

4.7 Sanitizer: cầu nối giữa Static và Dynamic Analysis

4.7.1 AddressSanitizer, UndefinedBehaviorSanitizer, MemorySa

Đã giới thiệu sơ lược ở Chương 2, các ***sanitizer*** là công cụ được tích hợp vào trình biên dịch (GCC/Clang) để chèn thêm mã kiểm tra (instrumentation) tại compile-time, phát hiện lỗi tại runtime với overhead thấp hơn nhiều so với Valgrind – đây chính là ”oracle” quan trọng nhất làm cho fuzzing hiện đại phát hiện được nhiều lớp lỗi hơn chỉ crash đơn thuần.

Bảng 4.3: Họ Sanitizer của LLVM/GCC

Sanitizer	Phát hiện	Overhead
AddressSanitizer (ASan)	Buffer overflow (stack/heap/global), use-after- free, use-after-return, double-free	∼2–3x
UndefinedBehaviorSanitizer (UBSan)	Tràn số nguyên có dấu, dịch bit vượt độ rộng kiểu, truy cập qua con trỏ NULL, vi phạm strict aliasing	∼1.2–2x
MemorySanitizer (MSan)	Đọc biến chưa khởi tạo (uninitialized memory read)	∼3x
ThreadSanitizer (TSan)	Race condition (data race) giữa các thread	∼5–15x

Shadow Memory – nguyên lý hoạt động của ASan

ASan cấp phát một vùng ”shadow memory” song song, ánh xạ mỗi 8 byte bộ nhớ chương trình thực tế thành 1 byte trạng thái (hợp lệ/không hợp lệ/ ”vùng đệm đỏ” – redzone). Khi chương trình cấp phát một buffer, ASan tự động chèn thêm các redzone (vùng nhớ ”cấm”) ngay trước và sau buffer đó; mọi truy cập đọc/ghi được instrument để kiểm tra shadow memory tương ứng trước khi thực hiện – nếu truy cập rơi vào redzone, ASan phát hiện ngay lập tức và in ra stack trace chính xác vị trí lỗi, thay vì để chương trình tiếp tục chạy với hành vi không xác định (như khi không có sanitizer).

```
# Bien dich muc tieu fuzzing voi ca ASan lan UBSan
clang -fsanitize=address,undefined,fuzzer -g -O1 target.c -o target_fuzzer

# Chay fuzzer (libFuzzer tich hop san trong LLVM, khong can driver rieng)
./target_fuzzer -max_len=4096 -timeout=10 corpus_dir/
```

```

# Toi thieu hoa corpus sau khi fuzzing (loai bo input trung lap coverage)
./target_fuzzer -merge=1 minimized_corpus/ corpus_dir/
```

Listing 4.4: Kết hợp fuzzing với sanitizer trong thực tế (libFuzzer)

4.8 Corpus Minimization và Crash Triage

4.8.1 Tối thiểu hóa corpus

Sau hàng triệu lần thực thi, seed corpus có thể chứa rất nhiều input dư thừa (nhiều input khác nhau nhưng đạt cùng một coverage). ***Corpus minimization*** loại bỏ các input dư thừa, giữ lại một tập nhỏ nhất vẫn bảo toàn toàn bộ coverage đã đạt được – bài toán tương đương *minimum set cover* (NP-hard), thường được giải bằng heuristic tham lam (greedy): lặp lại chọn input đạt được nhiều coverage *mới* nhất cho tới khi phủ hết.

4.8.2 Crash Triage và Deduplication

Khi fuzzer chạy hàng giờ/ngày, số lượng crash thu được có thể lên tới hàng nghìn, nhưng phần lớn thường xuất phát từ *cùng một* lỗi gốc (chỉ khác nhau về input cụ thể kích hoạt). ***Crash triage*** là quy trình phân loại và loại bỏ trùng lặp (deduplication), thường dựa trên chữ ký stack trace tại điểm crash (ví dụ hàm gây lỗi + vài frame gọi hàm phía trên) để nhóm các crash ”giống nhau về nguyên nhân gốc” lại với nhau, giúp kỹ sư bảo mật chỉ cần review một đại diện cho mỗi nhóm thay vì hàng nghìn crash riêng lẻ.

Kiểm soát chất lượng

Trong CI/CD pipeline công nghiệp, một fuzzer thường được cấu hình chạy liên tục (continuous fuzzing) với ngân sách thời gian cố định mỗi build (ví dụ 5 phút/target trong OSS-Fuzz), corpus được lưu trữ và tái sử dụng qua các lần chạy (persistent corpus), và mọi crash mới được tự động tạo issue tracker kèm chữ ký deduplication – biến fuzzing từ một hoạt động nghiên cứu thủ công thành một hạng mục kiểm soát chất lượng tự động hóa hoàn toàn, tương tự như unit test trong CI truyền thống.

4.9 Structure-aware Fuzzing (Kiểm thử mờ có cấu trúc)

4.9.1 Vấn đề với input có cấu trúc phức tạp

Đột biến bit-flip đơn giản (Listing 4.2) hoạt động tốt với định dạng nhị phân ”phẳng”, nhưng kém hiệu quả với các định dạng có cấu trúc lồng nhau phức tạp và ràng buộc ngữ nghĩa chặt (JSON, XML, ngôn ngữ lập trình, giao thức

Protocol Buffers) – một bit-flip ngẫu nhiên trên input JSON hợp lệ gần như chắc chắn phá vỡ cú pháp JSON ngay từ bước parse đầu tiên, không bao giờ chạm tới logic xử lý ngữ nghĩa bên trong.

Grammar-based / Structure-aware Fuzzing

Thay vì đột biến chuỗi byte thô, structure-aware fuzzer đột biến trực tiếp trên *cây cú pháp* (AST) hoặc theo một văn phạm (grammar) hình thức đã định nghĩa trước cho định dạng đầu vào – đảm bảo mọi input sinh ra vẫn *hợp lệ về cú pháp* (syntactically valid), cho phép fuzzer ”vượt qua” tầng parser để tập trung khám phá logic xử lý ngữ nghĩa sâu hơn bên trong, nơi phần lớn lỗi logic thực sự nằm.

Bảng 4.4: Công cụ structure-aware fuzzing tiêu biểu

Công cụ	Cách tiếp cận
libprotobuf-mutator	Đột biến trực tiếp trên cấu trúc Protocol Buffers đã định nghĩa qua file `.proto`, dùng cho fuzzing gRPC API
AFL++ custom mutators	Cho phép plug-in mutator tùy chỉnh hiểu cấu trúc định dạng cụ thể (ví dụ mutator riêng cho PNG, cho JavaScript AST)
Grammarinator/Nautilus	Sinh input hoàn toàn mới từ văn phạm ngữ cảnh phi ngữ cảnh (context-free grammar) đã đặc tả cho ngôn ngữ đích

4.10 Differential Fuzzing

Differential Fuzzing

Thay vì cần một oracle tường minh, differential fuzzing chạy *cùng một* input trên *hai (hoặc nhiều) cài đặt độc lập* của cùng một chức năng (ví dụ hai thư viện parse JSON khác nhau, hoặc phiên bản cũ/mới của cùng một hàm), và so sánh kết quả đầu ra. Sự khác biệt giữa hai kết quả là tín hiệu mạnh cho thấy *ít nhất một trong hai* cài đặt có lỗi – không cần biết trước output ”đúng” là gì.

Ví dụ

Google đã dùng differential fuzzing để so sánh hành vi của nhiều thư viện mã hóa TLS/x.509 khác nhau (OpenSSL, BoringSSL, mbedTLS) trên cùng một tập chứng chỉ số được sinh ngẫu nhiên/đột biến – bất kỳ trường hợp nào một thư viện chấp nhận chứng chỉ trong khi thư viện khác từ chối (hoặc ngược lại) đều là dấu hiệu của một lỗi phân tích cú pháp x.509 tiềm ẩn, có thể dẫn tới lỗ hổng bỏ qua xác thực chứng chỉ (certificate validation bypass) nghiêm trọng.

4.11 Case Study: Syzkaller và Fuzzing Nhân Hệ điều hành

Case Study: Syzkaller – coverage-guided fuzzing cho Linux kernel

Syzkaller (Google, mã nguồn mở) là một hệ thống fuzzing chuyên biệt cho nhân hệ điều hành (kernel fuzzing), đối tượng đặc biệt khó fuzz vì bề mặt tấn công là hàng nghìn system call với tham số phức tạp và trạng thái nội bộ khổng lồ (file descriptor, socket, tiến trình). Syzkaller mô tả syscall bằng một ngôn ngữ đặc tả riêng (giống structure-aware fuzzing ở mục trước) để sinh chuỗi lời gọi syscall hợp lệ về cú pháp, sử dụng coverage-guided feedback thu thập trực tiếp từ bên trong kernel (qua cơ chế `KCOV`). Kể từ khi ra mắt năm 2015, Syzkaller đã phát hiện hơn 5000 lỗi trong nhân Linux (theo dữ liệu công khai trên syzkaller.appspot.com), phần lớn là các lỗi use-after-free và race condition trong driver thiết bị và hệ thống file – những lớp lỗi cực kỳ khó phát hiện bằng code review thủ công do độ phức tạp của tương tác đa luồng trong kernel. Dự án minh chứng rằng dù kernel là mã nguồn đã được hàng nghìn kỹ sư review qua nhiều thập kỷ, fuzzing tự động vẫn liên tục tìm ra lớp lỗi mới – củng cố luận điểm cốt lõi của toàn giáo trình: phân tích tĩnh, phân tích động, và review thủ công là các kỹ thuật *bổ trợ*, không kỹ thuật nào một mình là đủ.

4.12 Thực hành từng bước: một vòng lặp fuzzing hoàn chỉnh

Phần này truy vết một vòng lặp `fuzz_loop` (Listing 4.2) hoàn chỉnh trên một hàm mục tiêu đơn giản để minh họa cụ thể cách coverage feedback dẫn dắt quá trình khám phá.

```
def target(data: bytes, tracker: CoverageTracker):
    tracker.hit(1)
    if len(data) >= 1 and data[0] == 0x41:      # 'A'
        tracker.hit(2)
        if len(data) >= 2 and data[1] == 0x42:  # 'B'
            tracker.hit(3)
            raise ValueError("CRASH: tim thay chuoi 'AB'!")
        else:
            tracker.hit(4)
    else:
        tracker.hit(5)
```

Listing 4.5: Hàm mục tiêu đơn giản để truy vết fuzzing

Bảng 4.5: Truy vết các iteration then chốt của fuzzing loop

Iteration	Input (hex)	Edge mới?	Kết quả
1 (seed)	`00`	{(0,1),(1,5)} mới	Không crash, giữ lại vì coverage ban đầu
∼50	`41 00` (đột biến trúng byte 0x41 ở vị trí 0)	{(1,2),(2,4)} mới	Không crash, nhưng coverage MỚI – thêm vào corpus
∼180	`41 42` (đột biến từ input trên, trúng thêm byte 0x42)	{(2,3)} mới	**CRASH!** Ghi nhận vào danh sách crash

Kiểm soát chất lượng

Quan sát mấu chốt: input gây crash (`41 42`) *không bao giờ* được sinh ra trực tiếp từ input rỗng ban đầu bằng một bước đột biến duy nhất (xác suất ngẫu nhiên trúng đúng 2 byte cụ thể là $1/65536$) – nó được xây dựng *tăng dần qua hai bước*, mỗi bước được ”khóa lại” nhờ tín hiệu coverage mới. Đây chính là lý do coverage-guided fuzzing hiệu quả hơn nhiều bậc so với random fuzzing thuần túy trên các hàm mục tiêu có nhiều điều kiện lồng nhau: mỗi lớp điều kiện được ”bóc tách” độc lập thay vì phải đoán trúng toàn bộ chuỗi điều kiện cùng lúc.

4.13 Tổng kết chương

Tổng kết chương

- Kiểm thử bảo mật khác kiểm thử chức năng ở oracle (crash/vi phạm bất biến thay vì so sánh output mong đợi) và ở nguồn gốc thiết kế input (kẻ tấn công thay vì đặc tả nghiệp vụ).
- Code coverage (statement/branch/path) là thước đo độ phủ kiểm thử; path coverage đầy đủ thường bất khả thi do path explosion, dẫn tới coverage- guided fuzzing như một chiến lược thực dụng thay thế.
- Taint tracking theo dõi lan truyền dữ liệu không tin cậy từ nguồn (source) tới điểm nhạy cảm (sink), là công cụ lý thuyết nền tảng cho việc phát hiện injection và buffer overflow.
- Black-box fuzzing (AFL-style, coverage-guided) hiệu quả và rẻ nhưng bị chặn bởi magic byte problem; white-box fuzzing (symbolic/concolic execution kết hợp SMT solving từ Chương 3) giải quyết được vấn đề này bằng cách giải ngược ràng buộc nhánh điều kiện một cách chính xác.
- Xu hướng công nghiệp là kết hợp cả hai (*hybrid fuzzing*, ví dụ Driller) để tận dụng tốc độ của black-box và khả năng vượt rào cản của white-box.

Xu hướng hiện đại

Fuzzing hiện đại ngày càng tích hợp học máy: các mô hình được huấn luyện để dự đoán *"đột biến nào có khả năng cao mở khóa coverage mới"* thay vì đột biến ngẫu nhiên đều tay (nghiên cứu như NEUZZ, MOPT sử dụng mạng neural để học cấu trúc gradient giữa input và coverage). Mô hình ngôn ngữ lớn (LLM) cũng đang được ứng dụng để tự động sinh *seed corpus có cấu trúc hợp lệ ban đầu* (ví dụ sinh file PDF/JSON hợp lệ đa dạng để làm điểm khởi đầu cho fuzzer, giải quyết một phần magic byte problem mà không cần symbolic execution tốn kém), cũng như tự động viết harness fuzzing (*fuzz driver*) cho các hàm API phức tạp – công việc trước đây đòi hỏi kỹ sư bảo mật có kinh nghiệm để viết thủ công. Google đã công bố dự án kết hợp LLM với OSS-Fuzz (dự án *OSS-Fuzz gen*) để tự động sinh và cải thiện fuzz harness cho hàng nghìn dự án mã nguồn mở.

Bài tập

Bài tập tự luận.

1. So sánh oracle của kiểm thử chức năng và kiểm thử bảo mật bằng một ví dụ cụ thể (hàm xử lý JSON parser).
2. Giải thích vì sao branch coverage 100% *không đảm bảo* path coverage 100%; cho ví dụ một hàm có 2 điều kiện if độc lập minh họa.

3. Trình bày cách concolic execution giải quyết magic byte problem tốt hơn cả random fuzzing lẫn symbolic execution thuần túy.

   Bài tập lập trình.

4. Mở rộng `CoverageTracker` ở Listing 4.1 để hỗ trợ trọng số cho từng edge (ưu tiên giữ lại input đạt coverage hiếm gặp hơn), viết test bằng `pytest`.

5. Kết hợp `fuzz_loop` (Listing 4.2) với một hàm mục tiêu có chứa magic byte check 4 byte cố định; đo số iteration trung bình cần thiết để vượt qua check đó bằng random mutation, so sánh với thời gian `solve_magic_bytes` (Listing 4.3) giải quyết ngay lập tức.

CHƯƠNG A

Bộ công cụ `softsec-toolkit`

Phụ lục này trình bày toàn bộ mã nguồn của bộ công cụ Python `softsec-toolkit` được phát triển song song với giáo trình, tổ chức thành bốn module tương ứng với bốn chương. Mã nguồn đầy đủ được cài đặt kèm bộ kiểm thử `pytest` (hơn 60 test case) đảm bảo tính đúng đắn của từng hàm được trích dẫn trong nội dung chính.

```
softsec-toolkit/
- softsec/
  - ch1_models/    # Chuong 1: Kripke structure, test design, risk
  - ch2_memsafe/   # Chuong 2: quet loi bo nho, memory trace, struct layout
  - ch3_staticsat/ # Chuong 3: SAT solver, SMT wrapper, bounded checker
  - ch4_dynafuzz/  # Chuong 4: coverage tracker, black/white-box fuzzer
- tests/           # pytest, mot file test cho moi module
- docs/
- pyproject.toml
```

Listing A.1: Cấu trúc thư mục dự án

```
cd softsec-toolkit
pip install -e ".[dev]"
pytest -v                        # chay toan bo unit test
python -m softsec.ch1_models.kripke   # chay demo mot module cu the
```

Listing A.2: Cài đặt và chạy kiểm thử

A.1 Module `ch1_models` – Chương 1

A.1.1 `kripke.py`

```
"""
kripke.py
=========

Cai dat mo hinh Kripke Structure - nen tang hinh thuc dung trong model
checking (se hoc chi tiet o muc 1.5 va Chuong 3).

Dinh nghia hinh thuc: Kripke structure la mot bo K = (S, S0, R, L) trong do:
    S  : tap huu han cac trang thai (states)
    S0 : tap trang thai khoi tao (initial states), S0 subseteq S
    R  : quan he chuyen tiep (transition relation), R subseteq S x S
    L  : ham nhan (labeling function) gan cho moi trang thai mot tap
         cac menh de dung (atomic propositions) tai trang thai do
```

```python

Module nay cung cap:
    - Lop KripkeStructure de xay dung va truy van mo hinh
    - Kiem tra tinh dat den duoc (reachability) cua mot trang thai
    - Sinh do thi trang thai (dung de ve minh hoa trong Beamer/TikZ)
"""

from __future__ import annotations
from dataclasses import dataclass, field
from typing import Dict, List, Set, FrozenSet, Iterable


@dataclass
class KripkeStructure:
    """Mo hinh Kripke structure don gian, dung cho vi du giang day."""

    states: Set[str] = field(default_factory=set)
    initial_states: Set[str] = field(default_factory=set)
    transitions: Dict[str, Set[str]] = field(default_factory=dict)
    labels: Dict[str, Set[str]] = field(default_factory=dict)

    def add_state(self, state: str, atomic_props: Iterable[str] = ()) -> None:
        """Them mot trang thai va gan nhan (atomic propositions) cho no."""
        self.states.add(state)
        self.labels.setdefault(state, set()).update(atomic_props)
        self.transitions.setdefault(state, set())

    def set_initial(self, *states: str) -> None:
        for s in states:
            if s not in self.states:
                raise ValueError(f"Trang thai '{s}' chua duoc khai bao")
            self.initial_states.add(s)

    def add_transition(self, src: str, dst: str) -> None:
        """Them quan he chuyen tiep src -> dst. Yeu cau ca hai da ton tai."""
        if src not in self.states or dst not in self.states:
            raise ValueError("Ca hai trang thai phai duoc khai bao truoc")
        self.transitions[src].add(dst)

    def label_of(self, state: str) -> FrozenSet[str]:
        return frozenset(self.labels.get(state, set()))

    def successors(self, state: str) -> Set[str]:
        return set(self.transitions.get(state, set()))

    def reachable_states(self) -> Set[str]:
        """Tinh tap tat ca trang thai dat den duoc tu cac trang thai khoi tao
        (BFS / DFS co ban - nen tang cho reachability analysis trong model
        checking o Chuong 3)."""
        visited: Set[str] = set()
        frontier: List[str] = list(self.initial_states)
```

```
        while frontier:
            s = frontier.pop()
            if s in visited:
                continue
            visited.add(s)
            for nxt in self.successors(s):
                if nxt not in visited:
                    frontier.append(nxt)
        return visited

    def is_reachable(self, target: str) -> bool:
        return target in self.reachable_states()

    def find_path(self, target: str) -> List[str] | None:
        """Tim mot duong di tu trang thai khoi tao toi target (BFS), tra ve
        None neu khong dat den duoc. Duong di nay chinh la 'counterexample'
        khi model checking phat hien vi pham thuoc tinh an toan."""
        from collections import deque

        for start in self.initial_states:
            visited = {start}
            queue = deque([(start, [start])])
            while queue:
                node, path = queue.popleft()
                if node == target:
                    return path
                for nxt in self.successors(node):
                    if nxt not in visited:
                        visited.add(nxt)
                        queue.append((nxt, path + [nxt]))
        return None

    def to_tikz(self) -> str:
        """Xuat mo hinh sang cu phap TikZ (automata library) de nhung truc
        tiep vao Beamer slide - huu ich khi sinh vien tu ve lai mo hinh cua
        minh cho bai tap lon."""
        lines = ["\\begin{tikzpicture}[->,>=stealth,shorten >=1pt,auto,node
            distance=2.5cm,semithick]"]
        for s in sorted(self.states):
            style = "initial,accepting" if s in self.initial_states else ""
            props = ",".join(sorted(self.labels.get(s, [])))
            lines.append(f"  \\node[state,{style}] ({s}) {{{s}\\\\{{\\tiny {props
                }}}}};")
        for src, dsts in self.transitions.items():
            for dst in dsts:
                lines.append(f"  \\path ({src}) edge ({dst});")
        lines.append("\\end{tikzpicture}")
        return "\n".join(lines)


def build_login_example() -> KripkeStructure:
```

```
    """Vi du minh hoa: mo hinh Kripke cho trang thai he thong dang nhap don
    gian, dung xuyen suot cac slide 1.5 (Kripke Structure) va bai tap.

    Trang thai:
        idle          - chua dang nhap, nhan {unauth}
        authenticated - da dang nhap thanh cong, nhan {auth}
        locked        - tai khoan bi khoa sau qua nhieu lan sai, nhan {unauth,
            locked}
    """
    k = KripkeStructure()
    k.add_state("idle", {"unauth"})
    k.add_state("authenticated", {"auth"})
    k.add_state("locked", {"unauth", "locked"})

    k.set_initial("idle")

    k.add_transition("idle", "authenticated")   # dang nhap thanh cong
    k.add_transition("idle", "locked")           # qua so lan sai
    k.add_transition("authenticated", "idle")    # dang xuat
    k.add_transition("locked", "locked")         # van con bi khoa

    return k


if __name__ == "__main__":
    model = build_login_example()
    print("Cac trang thai dat den duoc:", model.reachable_states())
    print("Duong di toi 'locked':", model.find_path("locked"))
    print()
    print(model.to_tikz())
```

Listing A.3: softsec/ch1_models/kripke.py

A.1.2 test_design.py

```
"""
test_design.py
==============

Cai dat tu dong hai ky thuat thiet ke test case da hoc o muc 1.3:
    - Equivalence Partitioning (phan lop tuong duong)
    - Boundary Value Analysis (phan tich gia tri bien)

Day la cong cu ho tro sinh vien tu dong sinh test case tu mot khoang
gia tri hop le, thay vi phai liet ke thu cong.
"""

from __future__ import annotations
from dataclasses import dataclass
from typing import List
```

```
16
17
18  @dataclass
19  class EquivalenceClass:
20      name: str
21      is_valid: bool
22      representative: int
23      description: str
24
25
26  def equivalence_classes(low: int, high: int) -> List[EquivalenceClass]:
27      """Sinh 3 lop tuong duong cho mot khoang gia tri hop le [low, high]:
28      lop nho hon low (invalid), lop hop le, lop lon hon high (invalid).
29
30      Vi du: equivalence_classes(0, 120) cho quy tac tuoi 0..120.
31      """
32      return [
33          EquivalenceClass(
34              name="below_range",
35              is_valid=False,
36              representative=low - 1,
37              description=f"Gia tri < {low} (khong hop le)",
38          ),
39          EquivalenceClass(
40              name="in_range",
41              is_valid=True,
42              representative=(low + high) // 2,
43              description=f"{low} <= gia tri <= {high} (hop le)",
44          ),
45          EquivalenceClass(
46              name="above_range",
47              is_valid=False,
48              representative=high + 1,
49              description=f"Gia tri > {high} (khong hop le)",
50          ),
51      ]
52
53
54  @dataclass
55  class BoundaryValue:
56      value: int
57      expected_valid: bool
58      description: str
59
60
61  def boundary_values(low: int, high: int) -> List[BoundaryValue]:
62      """Sinh 4 gia tri bien kinh dien cho khoang [low, high]:
63      low-1, low, high, high+1 - noi loi off-by-one thuong xay ra nhat.
64      """
65      return [
66          BoundaryValue(low - 1, False, f"Ngay duoi bien duoi ({low - 1})"),
```

```
        BoundaryValue(low, True, f"Bien duoi ({low})"),
        BoundaryValue(high, True, f"Bien tren ({high})"),
        BoundaryValue(high + 1, False, f"Ngay tren bien tren ({high + 1})"),
    ]


def generate_test_table(low: int, high: int, rule_name: str = "gia tri") -> str:
    """Sinh bang test case dang Markdown, ket hop ca hai ky thuat -
    tien loi de dan truc tiep vao bao cao bai tap lon."""
    rows = [f"| ID | Loai | Gia tri | Ky vong hop le | Mo ta |",
            f"|----|------|---------|----------------|-------|"]
    idx = 1
    for ec in equivalence_classes(low, high):
        rows.append(
            f"| TC{idx:02d} | Equivalence | {ec.representative} | "
            f"{'Co' if ec.is_valid else 'Khong'} | {ec.description} |"
        )
        idx += 1
    for bv in boundary_values(low, high):
        rows.append(
            f"| TC{idx:02d} | Boundary | {bv.value} | "
            f"{'Co' if bv.expected_valid else 'Khong'} | {bv.description} |"
        )
        idx += 1
    header = f"### Bang test case cho '{rule_name}' (khoang hop le [{low}, {high
        }])\n\n"
    return header + "\n".join(rows)


if __name__ == "__main__":
    print(generate_test_table(8, 20, rule_name="do dai mat khau"))
```

Listing A.4: softsec/ch1_models/test_design.py

A.1.3 risk.py

```
"""
risk.py
=======

Cai dat cong thuc tinh rui ro don gian da trinh bay o muc 1.1:

    Risk = Likelihood x Impact

trong do Likelihood va Impact duoc cho diem tu 1 (thap) den 10 (cao).
"""

from __future__ import annotations


```

```python
def risk_score(likelihood: int, impact: int) -> int:
    """Tinh diem rui ro tho. likelihood, impact trong khoang [1, 10]."""
    if not (1 <= likelihood <= 10) or not (1 <= impact <= 10):
        raise ValueError("likelihood va impact phai nam trong khoang [1, 10]")
    return likelihood * impact


def risk_level(score: int) -> str:
    """Phan loai muc do rui ro tu diem so (thang 1-100), theo ma tran rui ro
    da trinh bay tren slide (Thap / Trung binh / Cao)."""
    if score <= 20:
        return "Thap"
    elif score <= 60:
        return "Trung binh"
    else:
        return "Cao"


if __name__ == "__main__":
    examples = [
        ("SQL Injection tren he thong thanh toan", 9, 10),
        ("Loi dinh dang ngay thang, ung dung noi bo", 2, 2),
        ("Race condition hiem gap nhung xoa du lieu", 2, 9),
    ]
    for name, l, i in examples:
        s = risk_score(l, i)
        print(f"{name}: score={s} -> {risk_level(s)}")
```

Listing A.5: softsec/ch1_models/risk.py

A.2 Module ch2_memsafe – Chương 2

A.2.1 pattern_scanner.py

```python
"""
pattern_scanner.py
==================

Cong cu static analysis don gian (pattern-based, xem muc 2.5) quet ma
nguon C tim cac ham nguy hiem da hoc o muc 2.3 (gets, strcpy, sprintf,
strcat, ...).

Day la ky thuat "syntax-based" don gian nhat trong cac ky thuat da
trinh bay (syntax-based vs semantic-based, muc 2.5) - nhanh nhung co
the co false positive (vi du: ten ham nam trong comment hoac string).
Module nay co loai bo comment va string literal truoc khi quet de
giam false positive, minh hoa buoc dau tien huong toi semantic-based
analysis.
"""
```

```
16
17  from __future__ import annotations
18  import re
19  from dataclasses import dataclass
20  from typing import List
21
22
23  # Muc do nghiem trong tuong ung tung ham, dua tren CWE da hoc o Chuong 1-2
24  DANGEROUS_FUNCTIONS = {
25      "gets": ("CRITICAL", "CWE-242", "Khong gioi han do dai, da bi loai bo khoi
           C11"),
26      "strcpy": ("HIGH", "CWE-120", "Khong kiem tra kich thuoc dich, dung strncpy/
           snprintf"),
27      "strcat": ("HIGH", "CWE-120", "Khong kiem tra kich thuoc dich, dung strncat/
           snprintf"),
28      "sprintf": ("HIGH", "CWE-120", "Khong gioi han do dai buffer dich, dung
           snprintf"),
29      "scanf": ("MEDIUM", "CWE-120", "Dung %s khong gioi han co the tran bo dem"),
30      "system": ("HIGH", "CWE-78", "Co the bi command injection neu input khong tin
           cay"),
31      "atoi": ("LOW", "CWE-20", "Khong bao loi phan tich, nen dung strtol"),
32  }
33
34
35  @dataclass
36  class Finding:
37      line: int
38      function: str
39      severity: str
40      cwe: str
41      message: str
42
43
44  def _strip_comments_and_strings(source: str) -> str:
45      """Loai bo noi dung comment (// va /* */) va string literal de giam
46      false positive khi ten ham xuat hien trong van ban khong phai ma
47      thuc thi (lien he khai niem syntax-based vs semantic-based, muc 2.5)."""
48      # Xoa block comment /* ... */
49      source = re.sub(r"/\*.*?\*/", "", source, flags=re.DOTALL)
50      # Xoa line comment // ...
51      source = re.sub(r"//.*", "", source)
52      # Thay noi dung string literal "..." bang khoang trang cung do dai
53      def blank_string(m):
54          return '"' + " " * (len(m.group(0)) - 2) + '"'
55      source = re.sub(r'"(?:[^"\\]|\\.)*"', blank_string, source)
56      return source
57
58
59  def scan_source(source: str) -> List[Finding]:
60      """Quet ma nguon C (dang chuoi) va tra ve danh sach Finding cho moi
61      lan goi ham nguy hiem tim thay.
```

```
    Vi du:
        >>> findings = scan_source(open("vulnerable.c").read())
        >>> for f in findings:
        ...     print(f"Line {f.line}: {f.function} ({f.severity})")
    """
    cleaned = _strip_comments_and_strings(source)
    findings: List[Finding] = []

    for lineno, line in enumerate(cleaned.splitlines(), start=1):
        for func, (severity, cwe, message) in DANGEROUS_FUNCTIONS.items():
            # \b dam bao khop nguyen ten ham, khong khop mot phan cua ten dai hon
            pattern = r"\b" + re.escape(func) + r"\s*\("
            if re.search(pattern, line):
                findings.append(Finding(lineno, func, severity, cwe, message))

    return findings


def format_report(findings: List[Finding]) -> str:
    """Sinh bao cao dang van ban, sap xep theo muc do nghiem trong giam dan."""
    order = {"CRITICAL": 0, "HIGH": 1, "MEDIUM": 2, "LOW": 3}
    findings_sorted = sorted(findings, key=lambda f: order.get(f.severity, 99))
    lines = [f"Tim thay {len(findings)} van de:\n"]
    for f in findings_sorted:
        lines.append(
            f"  [Line {f.line:4d}] {f.severity:8s} {f.function:10s} "
            f"({f.cwe}) - {f.message}"
        )
    return "\n".join(lines)


if __name__ == "__main__":
    sample_code = """
    #include <stdio.h>
    #include <string.h>

    void greet(char *name) {
        char msg[50];
        // Vi du binh luan co chua strcpy( nhung khong nen bi bao loi
        strcpy(msg, "Hello, ");
        strcat(msg, name);
        printf(msg);
    }

    int main() {
        char buffer[100];
        gets(buffer);   // Cuc ky nguy hiem
        return 0;
    }
    """
```

```
    findings = scan_source(sample_code)
    print(format_report(findings))
```

Listing A.6: softsec/ch2_memsafe/pattern_scanner.py

A.2.2 memory_trace.py

```
"""
memory_trace.py
================

Mo phong mot "vet thuc thi" (execution trace) don gian gom cac lenh
alloc/free/access, dung de minh hoa va phat hien tu dong cac loi bo nho
da hoc o muc 2.2: use-after-free, double-free, memory leak.

Day la mot dang "dynamic analysis" toi gian (tuong tu nguyen ly cua
Valgrind/ASan da hoc o muc 2.3), nhung hoat dong tren mot mo hinh
tuong trung (symbolic) thay vi chuong trinh C thuc su - phu hop de
sinh vien hieu ro CO CHE phat hien loi truoc khi dung cong cu that.
"""

from __future__ import annotations
from dataclasses import dataclass, field
from enum import Enum, auto
from typing import Dict, List


class PointerState(Enum):
    ALLOCATED = auto()
    FREED = auto()
    UNKNOWN = auto()


@dataclass
class MemoryViolation:
    kind: str          # "double-free", "use-after-free", "invalid-free"
    pointer: str
    step: int
    detail: str


@dataclass
class MemoryTrace:
    """Theo doi trang thai cua cac 'con tro' tuong trung qua cac buoc
    alloc / free / access, phat hien vi pham tuong tu memcheck cua
    Valgrind (nhung o muc mo phong don gian hon nhieu)."""

    _states: Dict[str, PointerState] = field(default_factory=dict)
    _sizes: Dict[str, int] = field(default_factory=dict)
    _violations: List[MemoryViolation] = field(default_factory=list)
```

```
    _step: int = 0

    def alloc(self, name: str, size: int) -> None:
        """Mo phong malloc(size) gan ket qua cho 'name'."""
        self._step += 1
        if self._states.get(name) == PointerState.ALLOCATED:
            # Cap phat lai ma chua free -> tiem an memory leak cua lan truoc
            self._violations.append(
                MemoryViolation(
                    "potential-leak", name, self._step,
                    f"'{name}' duoc cap phat lai trong khi ban cap phat truoc do "
                    f"chua duoc free (memory leak tiem an)"
                )
            )
        self._states[name] = PointerState.ALLOCATED
        self._sizes[name] = size

    def free(self, name: str) -> None:
        """Mo phong free(name). Phat hien double-free va invalid-free."""
        self._step += 1
        state = self._states.get(name, PointerState.UNKNOWN)
        if state == PointerState.FREED:
            self._violations.append(
                MemoryViolation(
                    "double-free", name, self._step,
                    f"'{name}' da duoc free truoc do, goi free() lan thu 2 (CWE-415)"
                )
            )
            return
        if state == PointerState.UNKNOWN:
            self._violations.append(
                MemoryViolation(
                    "invalid-free", name, self._step,
                    f"'{name}' chua tung duoc cap phat (khong hop le khi free)"
                )
            )
            return
        self._states[name] = PointerState.FREED

    def access(self, name: str) -> None:
        """Mo phong doc/ghi qua con tro 'name'. Phat hien use-after-free."""
        self._step += 1
        state = self._states.get(name, PointerState.UNKNOWN)
        if state == PointerState.FREED:
            self._violations.append(
                MemoryViolation(
                    "use-after-free", name, self._step,
                    f"Truy cap '{name}' sau khi da free (CWE-416)"
                )
```

```
            )
        elif state == PointerState.UNKNOWN:
            self._violations.append(
                MemoryViolation(
                    "uninitialized-access", name, self._step,
                    f"Truy cap '{name}' truoc khi tung duoc cap phat"
                )
            )

    def check_leaks(self) -> List[MemoryViolation]:
        """Goi cuoi trace de kiem tra cac con tro con 'ALLOCATED' ma chua
        bao gio duoc free - tuong duong bao cao memory leak cua Valgrind."""
        leaks = []
        for name, state in self._states.items():
            if state == PointerState.ALLOCATED:
                leaks.append(
                    MemoryViolation(
                        "memory-leak", name, self._step,
                        f"'{name}' (size={self._sizes.get(name)}) khong bao gio
                            duoc free"
                    )
                )
        return leaks

    def get_violations(self, include_leaks: bool = True) -> List[MemoryViolation
        ]:
        result = list(self._violations)
        if include_leaks:
            result.extend(self.check_leaks())
        return result


if __name__ == "__main__":
    print("=== Vi du 1: Use-after-free ===")
    trace = MemoryTrace()
    trace.alloc("p1", size=64)
    trace.free("p1")
    trace.access("p1")   # Vi pham!
    for v in trace.get_violations():
        print(f"  [{v.kind}] step={v.step}: {v.detail}")

    print("\n=== Vi du 2: Double-free ===")
    trace2 = MemoryTrace()
    trace2.alloc("p2", size=128)
    trace2.free("p2")
    trace2.free("p2")   # Vi pham!
    for v in trace2.get_violations():
        print(f"  [{v.kind}] step={v.step}: {v.detail}")

    print("\n=== Vi du 3: Memory leak ===")
    trace3 = MemoryTrace()
```

```
    trace3.alloc("p3", size=256)
    # Quen goi free("p3")
    for v in trace3.get_violations():
        print(f"  [{v.kind}] step={v.step}: {v.detail}")
```

Listing A.7: softsec/ch2_memsafe/memory_trace.py

A.2.3 struct_layout.py

```
"""
struct_layout.py
================

Tinh toan kich thuoc va padding cua mot struct C tu danh sach cac
truong (field), ap dung kien thuc muc 2.4 (Data Structure Alignment).

Ho tro cac kieu co ban voi (kich thuoc, alignment) mac dinh tren kien
truc x86-64 pho bien:
    char      : size=1, align=1
    short     : size=2, align=2
    int/float : size=4, align=4
    long/double/long long : size=8, align=8
"""

from __future__ import annotations
from dataclasses import dataclass
from typing import List, Tuple


# (size, alignment) theo kieu du lieu, gia dinh x86-64 / LP64
TYPE_INFO = {
    "char": (1, 1),
    "short": (2, 2),
    "int": (4, 4),
    "float": (4, 4),
    "long": (8, 8),
    "double": (8, 8),
    "long long": (8, 8),
    "pointer": (8, 8),
}


@dataclass
class Field:
    name: str
    type: str  # phai la mot key trong TYPE_INFO


@dataclass
class LayoutEntry:
```

```
    field_name: str
    field_type: str
    offset: int
    size: int


@dataclass
class LayoutResult:
    entries: List[LayoutEntry]
    total_size: int
    padding_bytes: int
    struct_alignment: int

    def report(self) -> str:
        lines = [f"{'Truong':<12}{'Kieu':<10}{'Offset':<8}{'Size':<6}"]
        for e in self.entries:
            lines.append(f"{e.field_name:<12}{e.field_type:<10}{e.offset:<8}{e.
                size:<6}")
        lines.append("")
        lines.append(f"Tong kich thuoc struct : {self.total_size} byte")
        lines.append(f"So byte padding        : {self.padding_bytes} byte")
        lines.append(f"Alignment cua struct    : {self.struct_alignment} byte")
        return "\n".join(lines)


def compute_layout(fields: List[Field]) -> LayoutResult:
    """Tinh bo cuc bo nho (offset tung truong, padding, tong kich thuoc)
    theo dung quy tac alignment da trinh bay o muc 2.4:
      1. Moi truong bat dau tai dia chi chia het cho alignment cua no
      2. Kich thuoc tong cua struct la boi so cua alignment LON NHAT
         trong cac truong (de mang cac struct van giu dung alignment)
    """
    entries: List[LayoutEntry] = []
    offset = 0
    max_align = 1
    data_bytes = 0

    for f in fields:
        if f.type not in TYPE_INFO:
            raise ValueError(f"Kieu khong duoc ho tro: {f.type}")
        size, align = TYPE_INFO[f.type]
        max_align = max(max_align, align)

        # Can chinh offset len boi so gan nhat cua 'align'
        remainder = offset % align
        if remainder != 0:
            offset += align - remainder

        entries.append(LayoutEntry(f.name, f.type, offset, size))
        offset += size
        data_bytes += size
```

```
92
93      # Padding cuoi cung de tong kich thuoc la boi so cua max_align
94      remainder = offset % max_align
95      if remainder != 0:
96          offset += max_align - remainder
97
98      total_size = offset
99      padding_bytes = total_size - data_bytes
100
101     return LayoutResult(entries, total_size, padding_bytes, max_align)
102
103
104 def suggest_optimal_order(fields: List[Field]) -> List[Field]:
105     """Goi y thu tu sap xep truong toi uu: giam dan theo alignment,
106     dung nguyen tac da hoc trong slide 'Sap xep lai truong de giam
107     Padding' - giup giam padding xen ke giua cac truong."""
108     return sorted(fields, key=lambda f: TYPE_INFO[f.type][1], reverse=True)
109
110
111 if __name__ == "__main__":
112     # Vi du tu slide: struct Mixed { char a; long long b; char c; int d; char e;
        }
113     fields = [
114         Field("a", "char"),
115         Field("b", "long long"),
116         Field("c", "char"),
117         Field("d", "int"),
118         Field("e", "char"),
119     ]
120
121     print("=== Bo cuc goc (chua toi uu) ===")
122     result = compute_layout(fields)
123     print(result.report())
124
125     print("\n=== Bo cuc sau khi sap xep toi uu ===")
126     optimized_fields = suggest_optimal_order(fields)
127     result_opt = compute_layout(optimized_fields)
128     print(result_opt.report())
129
130     print(f"\nTiet kiem duoc: {result.total_size - result_opt.total_size} byte "
131           f"({result.total_size} -> {result_opt.total_size})")
```

Listing A.8: softsec/ch2_memsafe/struct_layout.py

A.3 Module ch3_staticsat – Chương 3

A.3.1 sat_solver.py

```
1 """
```

```python
"""
sat_solver.py
==============

Cai dat mot SAT solver toi gian theo thuat toan DPLL (Davis-Putnam-
Logemann-Loveland) da hoc o muc 3.4, bao gom ca ky thuat Unit
Propagation. Day KHONG phai mot solver hieu nang cao (khong co CDCL,
khong co heuristic chon bien thong minh) - muc dich la de sinh vien
thay ro co che DPLL hoat dong nhu the nao ben trong.

Bieu dien:
    - Mot bien duoc danh so nguyen duong: 1, 2, 3, ...
    - Mot literal la so nguyen khac 0: so duong = bien do, so am = phu
      dinh bien do (vd: 3 nghia la x3, -3 nghia la NOT x3)
    - Mot Clause la mot list cac literal, noi voi nhau bang OR
    - Mot cong thuc CNF la mot list cac Clause, noi voi nhau bang AND
"""

from __future__ import annotations
from dataclasses import dataclass
from typing import Dict, List, Optional, Set


Clause = List[int]


@dataclass
class DPLLResult:
    satisfiable: bool
    assignment: Optional[Dict[int, bool]]
    num_decisions: int  # so lan solver phai "doan" (khong suy ra duoc)


class DPLLSolver:
    """SAT solver mini dua tren thuat toan DPLL (xem slide 'Thuat toan
    DPLL - Nen tang cua SAT Solver', muc 3.4)."""

    def __init__(self, clauses: List[Clause], num_vars: int):
        self.clauses = [list(c) for c in clauses]
        self.num_vars = num_vars
        self._decisions = 0

    def solve(self) -> DPLLResult:
        self._decisions = 0
        assignment: Dict[int, bool] = {}
        result = self._dpll(self.clauses, assignment)
        if result is None:
            return DPLLResult(False, None, self._decisions)
        return DPLLResult(True, result, self._decisions)

    def _dpll(self, clauses: List[Clause], assignment: Dict[int, bool]) ->
        Optional[Dict[int, bool]]:
```

```
52          clauses = self._simplify(clauses, assignment)
53          if clauses is None:
54              return None  # Co mau thuan (mot clause rong)
55          if not clauses:
56              return dict(assignment)  # Tat ca clause da thoa man
57
58          # Unit Propagation: neu co clause chi con 1 literal, gan gia tri bat buoc
59          unit = self._find_unit_clause(clauses)
60          if unit is not None:
61              var, value = abs(unit), unit > 0
62              new_assignment = dict(assignment)
63              new_assignment[var] = value
64              return self._dpll(clauses, new_assignment)
65
66          # Chon bien chua gan de "doan" (decision)
67          var = self._pick_unassigned_var(clauses, assignment)
68          if var is None:
69              return dict(assignment)
70
71          self._decisions += 1
72          # Thu True truoc
73          assignment_true = dict(assignment)
74          assignment_true[var] = True
75          result = self._dpll(clauses, assignment_true)
76          if result is not None:
77              return result
78
79          # Quay lui (backtrack), thu False
80          assignment_false = dict(assignment)
81          assignment_false[var] = False
82          return self._dpll(clauses, assignment_false)
83
84      @staticmethod
85      def _simplify(clauses: List[Clause], assignment: Dict[int, bool]) -> Optional
            [List[Clause]]:
86          """Loai bo cac literal/clause da duoc thoa man theo assignment hien tai.
87          Tra ve None neu phat hien mau thuan (mot clause rong -> UNSAT o nhanh nay
                )."""
88          result: List[Clause] = []
89          for clause in clauses:
90              new_clause = []
91              satisfied = False
92              for lit in clause:
93                  var, positive = abs(lit), lit > 0
94                  if var in assignment:
95                      lit_value = assignment[var] if positive else not assignment[
                            var]
96                      if lit_value:
97                          satisfied = True
98                          break
99                      # lit sai -> khong dua vao new_clause (loai literal nay)
```

```
                else:
                    new_clause.append(lit)
            if satisfied:
                continue
            if not new_clause:
                return None  # Clause rong: mau thuan
            result.append(new_clause)
        return result

    @staticmethod
    def _find_unit_clause(clauses: List[Clause]) -> Optional[int]:
        for clause in clauses:
            if len(clause) == 1:
                return clause[0]
        return None

    @staticmethod
    def _pick_unassigned_var(clauses: List[Clause], assignment: Dict[int, bool])
        -> Optional[int]:
        for clause in clauses:
            for lit in clause:
                var = abs(lit)
                if var not in assignment:
                    return var
        return None


def parse_dimacs(text: str) -> tuple[List[Clause], int]:
    """Doc cong thuc CNF dang chuan DIMACS (dinh dang pho bien cho SAT
    solver), tra ve (danh sach clause, so bien).

    Dinh dang:
        c day la comment
        p cnf <so_bien> <so_clause>
        1 -2 0
        -1 3 0
        ...
    Moi dong ket thuc bang 0, cac so la literal.
    """
    clauses: List[Clause] = []
    num_vars = 0
    for line in text.strip().splitlines():
        line = line.strip()
        if not line or line.startswith("c"):
            continue
        if line.startswith("p"):
            parts = line.split()
            num_vars = int(parts[2])
            continue
        literals = [int(x) for x in line.split() if x != "0"]
        if literals:
```

```
            clauses.append(literals)
    return clauses, num_vars


if __name__ == "__main__":
    # Vi du tu slide: phi = (a OR b) AND (NOT a OR c) AND (NOT b OR NOT c)
    # a=1, b=2, c=3
    clauses = [
        [1, 2],     # a OR b
        [-1, 3],    # NOT a OR c
        [-2, -3],   # NOT b OR NOT c
    ]
    solver = DPLLSolver(clauses, num_vars=3)
    result = solver.solve()
    print(f"SAT: {result.satisfiable}")
    if result.satisfiable:
        names = {1: "a", 2: "b", 3: "c"}
        readable = {names[v]: val for v, val in result.assignment.items()}
        print(f"Assignment: {readable}")
    print(f"So lan decision: {result.num_decisions}")
```

Listing A.9: softsec/ch3_staticsat/sat_solver.py

A.3.2 smt_wrapper.py

```
"""
smt_wrapper.py
===============

Tien ich boc quanh thu vien z3-solver cho cac bai toan thuong gap da
hoc trong muc 3.4 (SAT/SMT), 3.6 (Memory Model) va 3.7 (Aligned Memory
Model): kiem tra tran so nguyen, kiem tra vi pham alignment, va sinh
test case tu dieu kien duong di (path condition) - ung dung truc tiep
cua symbolic execution (Chuong 1) ket hop SMT.

YEU CAU: pip install z3-solver
"""

from __future__ import annotations
from dataclasses import dataclass
from typing import Dict, Optional

try:
    from z3 import (
        BitVec, BitVecVal, Solver, sat, URem, Int, And, Or, Not, simplify
    )
    Z3_AVAILABLE = True
except ImportError:  # pragma: no cover
    Z3_AVAILABLE = False

```

```
26
27  def _require_z3():
28      if not Z3_AVAILABLE:
29          raise ImportError(
30              "Module nay yeu cau thu vien z3-solver. Cai dat bang: "
31              "pip install z3-solver"
32          )
33
34
35  @dataclass
36  class OverflowResult:
37      can_overflow: bool
38      example_value: Optional[int]
39      result_after_overflow: Optional[int]
40
41
42  def check_integer_overflow(value: int, addend: int, bits: int = 8, signed: bool =
        False) -> OverflowResult:
43      """Kiem tra hinh thuc xem phep cong value + addend co gay tran so
44      tren kieu so nguyen 'bits' bit hay khong (lien he vi du Chuong 2:
45      unsigned char 250 + 10 = 4, xem slide 'Vi du mo hinh hoa bang
46      Bit-vector', muc 3.4).
47
48      Vi day la gia tri cu the (khong phai bien tu do), ham nay chi mo
49      phong lai phep tinh bit-vector chinh xac bang Z3 de xac nhan ket
50      qua tran so, thay vi tim kiem gia tri gay tran (xem
51      find_overflow_example de sinh gia tri tu do).
52      """
53      _require_z3()
54      a = BitVecVal(value, bits)
55      b = BitVecVal(addend, bits)
56      result = simplify(a + b)
57      result_int = result.as_signed_long() if signed else result.as_long()
58
59      max_val = (2 ** (bits - 1) - 1) if signed else (2 ** bits - 1)
60      min_val = -(2 ** (bits - 1)) if signed else 0
61      expected = value + addend
62      overflowed = expected > max_val or expected < min_val
63
64      return OverflowResult(overflowed, value, result_int)
65
66
67  def find_overflow_example(bits: int = 8, addend: int = 10) -> OverflowResult:
68      """Dung SMT solver TIM ộmt gia tri x (bien tu do, kieu bit-vector
69      'bits' bit) sao cho x + addend gay tran so (ket qua < x ve mat toan
70      hoc thong thuong do wrap-around)."""
71      _require_z3()
72      x = BitVec('x', bits)
73      s = Solver()
74      # Tran so unsigned: x + addend (theo bit-vector) < x (theo gia tri khong dau)
75      from z3 import ULT
```

```
    s.add(ULT(x + addend, x))
    if s.check() == sat:
        model = s.model()
        x_val = model[x].as_long()
        result_val = simplify(BitVecVal(x_val, bits) + BitVecVal(addend, bits)).
            as_long()
        return OverflowResult(True, x_val, result_val)
    return OverflowResult(False, None, None)


@dataclass
class AlignmentResult:
    can_violate: bool
    example_address: Optional[int]


def check_alignment_violation(alignment: int, guaranteed_alignment: int,
    address_bits: int = 64) -> AlignmentResult:
    """Kiem tra hinh thuc: neu mot bo cap phat (allocator) chi dam bao
    dia chi tra ve la boi so cua 'guaranteed_alignment', lieu no co the
    vi pham yeu cau alignment lon hon 'alignment' hay khong (xem slide
    'Bai tap: Kiem chung Alignment bang Z3', muc 3.7).

    Vi du: check_alignment_violation(alignment=8, guaranteed_alignment=4)
    -> True, vi mot dia chi chia het cho 4 khong nhat thiet chia het cho 8.
    """
    _require_z3()
    addr = BitVec('addr', address_bits)
    s = Solver()
    s.add(URem(addr, guaranteed_alignment) == 0)
    s.add(URem(addr, alignment) != 0)
    if s.check() == sat:
        model = s.model()
        return AlignmentResult(True, model[addr].as_long())
    return AlignmentResult(False, None)


def generate_test_for_path(constraints: list, variables: Dict[str, str]) ->
    Optional[Dict[str, int]]:
    """Sinh mot bo gia tri test case tu danh sach 'path condition' duoi
    dang chuoi bieu thuc Python-like don gian, ap dung y tuong symbolic
    execution + SMT da hoc (Chuong 1 va muc 3.4).

    variables: dict ten_bien -> kieu ("int" duoc ho tro)
    constraints: danh sach cac bieu thuc Python hop le su dung ten bien
                 (vd: ["x > 0", "x < 100"])

    Vi du:
        >>> generate_test_for_path(["x > 0", "x < 100"], {"x": "int"})
        {'x': 1}   # hoac mot gia tri bat ky thoa dieu kien
    """
```

```
    _require_z3()
    z3_vars = {name: Int(name) for name, t in variables.items() if t == "int"}
    s = Solver()
    for constraint in constraints:
        # Danh gia bieu thuc voi cac bien Z3 thay the bien Python thuong
        expr = eval(constraint, {"__builtins__": {}}, z3_vars)
        s.add(expr)
    if s.check() == sat:
        model = s.model()
        return {name: model[var].as_long() for name, var in z3_vars.items() if
            model[var] is not None}
    return None


if __name__ == "__main__":
    print("=== Kiem tra tran so cu the: 250 + 10 tren 8-bit ===")
    r = check_integer_overflow(250, 10, bits=8)
    print(f"  Tran so: {r.can_overflow}, ket qua thuc te: {r.
        result_after_overflow}")

    print("\n=== Tim vi du tran so tu dong (unsigned 8-bit, +10) ===")
    r2 = find_overflow_example(bits=8, addend=10)
    print(f"  Tim thay: x={r2.example_value} -> ket qua={r2.result_after_overflow
        }")

    print("\n=== Kiem tra vi pham alignment: yeu cau 8, allocator dam bao 4 ===")
    r3 = check_alignment_violation(alignment=8, guaranteed_alignment=4)
    print(f"  Co the vi pham: {r3.can_violate}, dia chi vi du: {r3.
        example_address}")

    print("\n=== Sinh test case tu path condition ===")
    test = generate_test_for_path(["x > 0", "x < 100"], {"x": "int"})
    print(f"  Test case sinh ra: {test}")
```

Listing A.10: softsec/ch3_staticsat/smt_wrapper.py

A.3.3 bounded_checker.py

```
"""
bounded_checker.py
====================

Bounded Model Checker (BMC) mini, ap dung truc tiep len KripkeStructure
da xay dung o Chuong 1 (softsec.ch1_models.kripke). Day la ung dung cu
the cua y tuong BMC da hoc o muc 3.5: kiem tra xem co ton tai mot
duong di do dai toi da k buoc tu trang thai khoi tao dan toi mot trong
cac "trang thai xau" (bad states) hay khong.

Cai dat nay dung tim kiem tuong minh (explicit-state BFS gioi han do
sau k) thay vi ma hoa SAT/SMT day du - don gian hon de sinh vien doi
```

```
chieu voi ly thuyet, nhung the hien dung ban chat cua BMC: chi xet
duong di <= k buoc, va ket qua chi dang tin cay trong pham vi do (xem
slide 'Completeness Threshold', muc 3.5).
"""

from __future__ import annotations
from collections import deque
from dataclasses import dataclass
from typing import List, Optional, Set, TYPE_CHECKING

if TYPE_CHECKING:
    from softsec.ch1_models.kripke import KripkeStructure


@dataclass
class BMCResult:
    violated: bool
    counterexample: Optional[List[str]]  # Duong di tu S0 toi bad state, None neu khong vi pham
    k: int
    reached_completeness_threshold: bool  # True neu da xet het reachable states truoc khi het k


def bmc_check(model: "KripkeStructure", bad_states: Set[str], k: int) -> BMCResult:
    """Kiem tra xem co ton tai duong di do dai toi da k buoc tu mot
    trang thai khoi tao cua 'model' dan toi mot trang thai trong
    'bad_states' hay khong.

    Tra ve BMCResult voi:
        - violated=True va counterexample cu the neu tim thay vi pham
        - violated=False neu khong tim thay trong pham vi k buoc
        - reached_completeness_threshold=True neu qua trinh tim kiem
          da duyet het moi trang thai reachable ma khong can dung het k
          buoc (nghia la ket qua "an toan" nay THUC SU dang tin cay,
          khong chi dung trong pham vi k - xem slide 'Completeness
          Threshold', muc 3.5)
    """
    # Kiem tra vi pham ngay tai trang thai khoi tao (buoc 0)
    for s0 in model.initial_states:
        if s0 in bad_states:
            return BMCResult(True, [s0], k, reached_completeness_threshold=True)

    visited: Set[str] = set(model.initial_states)
    # Moi phan tu trong queue: (trang_thai_hien_tai, duong_di_tu_dau)
    queue = deque([(s, [s]) for s in model.initial_states])
    exhausted_within_k = True  # Se thanh False neu con trang thai chua duyet khi het k buoc

    step = 0
```

```
    frontier = list(queue)
    while frontier and step < k:
        next_frontier = []
        for state, path in frontier:
            for nxt in model.successors(state):
                if nxt in bad_states:
                    return BMCResult(True, path + [nxt], k,
                        reached_completeness_threshold=True)
                if nxt not in visited:
                    visited.add(nxt)
                    next_frontier.append((nxt, path + [nxt]))
        frontier = next_frontier
        step += 1

    # Neu con frontier chua duyet het khi het k buoc, chua dat completeness
        threshold
    reached_ct = len(frontier) == 0

    return BMCResult(False, None, k, reached_completeness_threshold=reached_ct)


def find_completeness_threshold(model: "KripkeStructure", max_k: int = 1000) ->
    Optional[int]:
    """Tim gia tri k nho nhat sao cho BMC voi k do da duyet het toan bo
    tap trang thai reachable (tuc la dat completeness threshold - xem
    slide muc 3.5). Tra ve None neu vuot qua max_k ma van chua dat."""
    total_reachable = len(model.reachable_states())
    visited: Set[str] = set(model.initial_states)
    frontier = list(model.initial_states)

    if len(visited) == total_reachable:
        return 0

    for k in range(1, max_k + 1):
        next_frontier = []
        for state in frontier:
            for nxt in model.successors(state):
                if nxt not in visited:
                    visited.add(nxt)
                    next_frontier.append(nxt)
        frontier = next_frontier
        if len(visited) == total_reachable:
            return k
        if not frontier:
            # Khong con trang thai moi nhung chua dat total_reachable - khong the
                xay ra
            # neu reachable_states() dung, nhung kiem tra phong ngua
            return k
    return None


```

```python
if __name__ == "__main__":
    import sys
    sys.path.insert(0, "../../")
    from softsec.ch1_models.kripke import build_login_example

    model = build_login_example()

    print("=== BMC voi k=1 ===")
    result = bmc_check(model, bad_states={"locked"}, k=1)
    print(f"  Violated: {result.violated}, Counterexample: {result.counterexample}")
    print(f"  Dat completeness threshold: {result.reached_completeness_threshold}")

    print("\n=== BMC voi k=3 ===")
    result3 = bmc_check(model, bad_states={"locked"}, k=3)
    print(f"  Violated: {result3.violated}, Counterexample: {result3.counterexample}")

    print("\n=== Tim Completeness Threshold ===")
    ct = find_completeness_threshold(model)
    print(f"  Completeness Threshold = {ct}")

    print("\n=== Kiem tra trang thai khong the dat den ===")
    result_unreachable = bmc_check(model, bad_states={"nonexistent_state"}, k=5)
    print(f"  Violated: {result_unreachable.violated} (ky vong False)")
```

Listing A.11: softsec/ch3_staticsat/bounded_checker.py

A.4 Module ch4_dynafuzz – Chương 4

A.4.1 coverage_tracker.py

```python
"""
coverage_tracker.py
=====================

Theo doi code coverage (dong da thuc thi) cho mot ham Python muc tieu,
mo phong y tuong "instrumentation" da hoc o muc 4.2, 4.3: chen "diem
theo doi" vao chuong trinh de biet duong thuc thi nao da duoc di qua.

Python co san co che `sys.settrace` cho phep theo doi moi dong lenh
duoc thuc thi - day chinh la mot dang instrumentation o muc runtime,
tuong tu (ve mat nguyen ly, don gian hoa nhieu) co che ma cac cong cu
nhu gcov hay -fsanitize-coverage su dung cho C/C++.
"""

from __future__ import annotations
import sys
```

```python
from dataclasses import dataclass, field
from typing import Callable, Set, Tuple, Any


@dataclass
class CoverageTracker:
    """Theo doi tap hop cac dong da thuc thi qua nhieu lan goi ham.

    Dung nhu mot 'bo nho' coverage tich luy - giong vai tro cua bitmap
    trong AFL (da hoc o muc 4.2): moi lan chay voi input moi, coverage
    moi duoc HOP VAO coverage da tich luy tu truoc, giup fuzzer biet
    input nao dong gop dieu gi moi.
    """

    lines_hit: Set[Tuple[str, int]] = field(default_factory=set)

    def run_with_tracking(self, func: Callable, *args, **kwargs) -> Any:
        """Chay 'func(*args, **kwargs)' voi tracing bat, tra ve ket qua
        cua ham (hoac gay exception neu ham that bai - fuzzer se bat
        exception nay va coi la 'crash', xem blackbox_fuzzer.py)."""
        new_lines: Set[Tuple[str, int]] = set()

        def tracer(frame, event, arg):
            if event == "line":
                new_lines.add((frame.f_code.co_filename, frame.f_lineno))
            return tracer

        old_tracer = sys.gettrace()
        sys.settrace(tracer)
        try:
            result = func(*args, **kwargs)
        finally:
            sys.settrace(old_tracer)

        is_new = not new_lines.issubset(self.lines_hit)
        self.lines_hit |= new_lines
        return result, is_new, new_lines

    @property
    def total_lines_covered(self) -> int:
        return len(self.lines_hit)

    def reset(self) -> None:
        self.lines_hit.clear()


def trace_coverage(func: Callable, *args, **kwargs) -> Tuple[Any, Set[Tuple[str,
    int]]]:
    """Ham tien ich: chay 'func' MOT LAN va tra ve (ket qua, tap dong
    da thuc thi) - khong tich luy qua nhieu lan goi nhu CoverageTracker.
    Huu ich de kiem tra nhanh coverage cua mot lan chay don le."""
```

```
    tracker = CoverageTracker()
    result, _, new_lines = tracker.run_with_tracking(func, *args, **kwargs)
    return result, new_lines


if __name__ == "__main__":
    def example_target(x: int) -> str:
        if x > 0:
            if x > 100:
                return "large_positive"
            return "positive"
        elif x < 0:
            return "negative"
        return "zero"

    tracker = CoverageTracker()

    print("=== Lan 1: x=5 ===")
    result, is_new, lines = tracker.run_with_tracking(example_target, 5)
    print(f"  Ket qua: {result}, So dong moi: {len(lines)}, Co moi khong: {is_new
        }")

    print("\n=== Lan 2: x=5 (lap lai - khong nen co dong moi) ===")
    result, is_new, lines = tracker.run_with_tracking(example_target, 5)
    print(f"  Ket qua: {result}, Co moi khong: {is_new}")

    print("\n=== Lan 3: x=-5 (nhanh khac - nen co dong moi) ===")
    result, is_new, lines = tracker.run_with_tracking(example_target, -5)
    print(f"  Ket qua: {result}, Co moi khong: {is_new}")

    print(f"\nTong so dong da bao phu (tich luy): {tracker.total_lines_covered}")
```

Listing A.12: softsec/ch4_dynafuzz/coverage_tracker.py

A.4.2 blackbox_fuzzer.py

```
"""
blackbox_fuzzer.py
======================

Fuzzer hop den (mutation-based) voi coverage feedback, ap dung cac
khai niem da hoc o muc 4.2 (coverage-guided fuzzing) va 4.5
(mutation operators: bit flip, byte flip, arithmetic, block ops).

Fuzzer nay KHONG can biet cau truc ben trong ham muc tieu - chi quan
sat: (1) coverage co tang khong (dung CoverageTracker), (2) ham co
"crash" khong (nem exception hoac tra ve True, tuy quy uoc cua ham
muc tieu).
"""

```

```python
from __future__ import annotations
import random
from dataclasses import dataclass, field
from typing import Callable, List, Optional

from .coverage_tracker import CoverageTracker


def mutate(data: bytes) -> bytes:
    """Ap dung mot phep dot bien ngau nhien len 'data' (xem slide
    'Mutation-based Fuzzing bang Python', muc 4.5): bit flip, byte
    flip, insert, hoac delete."""
    buf = bytearray(data)
    op = random.choice(["bitflip", "byteflip", "insert", "delete", "arithmetic"])

    if op == "bitflip" and buf:
        idx = random.randrange(len(buf))
        bit = random.randrange(8)
        buf[idx] ^= (1 << bit)
    elif op == "byteflip" and buf:
        idx = random.randrange(len(buf))
        buf[idx] = random.randrange(256)
    elif op == "insert":
        idx = random.randrange(len(buf) + 1)
        buf.insert(idx, random.randrange(256))
    elif op == "delete" and len(buf) > 1:
        idx = random.randrange(len(buf))
        del buf[idx]
    elif op == "arithmetic" and buf:
        idx = random.randrange(len(buf))
        delta = random.choice([-2, -1, 1, 2])
        buf[idx] = (buf[idx] + delta) % 256

    return bytes(buf)


@dataclass
class BlackboxFuzzResult:
    found_crash: bool
    crashing_input: Optional[bytes]
    iterations_used: int
    corpus_size: int
    corpus: List[bytes] = field(default_factory=list)


def blackbox_fuzz(
    target: Callable[[bytes], bool],
    seed: bytes,
    iterations: int = 1000,
    use_coverage_feedback: bool = True,
    seed_pool: Optional[List[bytes]] = None,
```

```
) -> BlackboxFuzzResult:
    """Fuzzing hop den don gian voi coverage feedback tuy chon.

    target: ham nhan bytes, tra ve True neu phat hien "crash" gia lap,
            hoac nem exception (cung duoc coi la crash).
    seed: input hop le ban dau (xem 'Seed Corpus', muc 4.1).
    iterations: so lan thu toi da.
    use_coverage_feedback: neu True, chi giu lai input lam tang
            coverage vao corpus (coverage-guided, muc 4.2); neu False,
            fuzzing ngau nhien thuan tuy (khong coverage feedback).

    Vi day la fuzzer Python thuan tuy (khong instrumentation nhi phan
    that su), coverage duoc do bang CoverageTracker (sys.settrace) tren
    chinh ham 'target'.
    """
    corpus: List[bytes] = list(seed_pool) if seed_pool else [seed]
    tracker = CoverageTracker()

    for i in range(iterations):
        base = random.choice(corpus)
        candidate = mutate(base)

        try:
            if use_coverage_feedback:
                crashed, is_new_coverage, _ = tracker.run_with_tracking(target,
                    candidate)
            else:
                crashed = target(candidate)
                is_new_coverage = False
        except Exception:
            # Bat ky exception nao tu target duoc coi la "crash" thuc su
            return BlackboxFuzzResult(True, candidate, i + 1, len(corpus), corpus
                )

        if crashed:
            return BlackboxFuzzResult(True, candidate, i + 1, len(corpus), corpus
                )

        if use_coverage_feedback and is_new_coverage:
            corpus.append(candidate)

    return BlackboxFuzzResult(False, None, iterations, len(corpus), corpus)


if __name__ == "__main__":
    def target_needs_magic(data: bytes) -> bool:
        """Ham vi du: 'crash' gia lap neu 4 byte dau la 'FUZZ' - minh
        hoa chinh xac 'van de magic byte' da hoc o muc 4.3, 4.5."""
        if len(data) >= 4 and data[:4] == b"FUZZ":
            return True
        return False
```

```
114
115     print("=== Black-box fuzzing (khong coverage feedback) ===")
116     random.seed(42)
117     result_no_cov = blackbox_fuzz(target_needs_magic, seed=b"AAAA", iterations
            =5000,
118                                   use_coverage_feedback=False)
119     print(f"  Tim thay crash: {result_no_cov.found_crash} "
120           f"(sau {result_no_cov.iterations_used} lan thu)")
121
122     print("\n=== Black-box fuzzing (co coverage feedback) ===")
123     random.seed(42)
124     result_cov = blackbox_fuzz(target_needs_magic, seed=b"AAAA", iterations=5000,
125                                use_coverage_feedback=True)
126     print(f"  Tim thay crash: {result_cov.found_crash} "
127           f"(sau {result_cov.iterations_used} lan thu)")
128     print("  (Ca hai deu it kha nang tim thay 'FUZZ' ngau nhien -> minh hoa"
129           " ly do can white-box fuzzing, xem whitebox_fuzzer.py)")
```

Listing A.13: softsec/ch4_dynafuzz/blackbox_fuzzer.py

A.4.3 whitebox_fuzzer.py

```
1  """
2  whitebox_fuzzer.py
3  ======================
4
5  Fuzzer hop trang (white-box / concolic) don gian, ap dung truc tiep
6  nguyen ly da hoc o muc 4.4 (Concolic Execution) va 4.6 (White-box
7  Fuzzing): chay ham muc tieu voi gia tri CU THE, DONG THOI ghi lai
8  "path condition" duoi dang rang buoc Z3 (Chuong 3), roi PHU DINH tung
9  dieu kien mot (chien luoc Generational Search cua SAGE, da hoc o muc
10 4.6) de sinh gia tri moi dan toi nhanh muc tieu.
11
12 GIOI HAN SU PHAM: day la phien ban RUT GON de minh hoa nguyen ly cot
13 loi, chi ho tro ham muc tieu nhan CAC THAM SO KIEU int va dung cac
14 toan tu so sanh co ban (==, !=, <, >, <=, >=). Cac cong cu that (KLEE,
15 Angr, SAGE - da gioi thieu o muc 4.6) xu ly duoc toan bo tap lenh may/
16 bytecode phuc tap hon nhieu.
17
18 YEU CAU: pip install z3-solver
19 """
20
21 from __future__ import annotations
22 from dataclasses import dataclass, field
23 from typing import Any, Callable, Dict, List, Optional, Tuple
24
25 try:
26     from z3 import Int, Solver, sat, Not, BoolRef
27     Z3_AVAILABLE = True
28 except ImportError:  # pragma: no cover
```

```
    Z3_AVAILABLE = False


def _require_z3():
    if not Z3_AVAILABLE:
        raise ImportError(
            "Module nay yeu cau thu vien z3-solver. Cai dat bang: "
            "pip install z3-solver"
        )


class SymInt:
    """Wrapper 'concolic' cho mot gia tri int: mang theo CA gia tri cu
    the (concrete_value) LAN bieu thuc Z3 tuong trung (symbolic_expr).
    Moi phep so sanh vua tra ve ket qua concrete (de code Python goc
    chay dung binh thuong), vua ghi lai rang buoc Z3 tuong ung vao
    path_condition dung chung. Cac phep toan so hoc co ban (+, -, *)
    duoc ho tro de xu ly bieu thuc nhu 'x * 2' hay 'x + y'."""

    def __init__(self, concrete_value: int, symbolic_expr, path_condition: List):
        self.concrete_value = concrete_value
        self.symbolic_expr = symbolic_expr
        self.path_condition = path_condition

    def _wrap(self, other):
        if isinstance(other, SymInt):
            return other.concrete_value, other.symbolic_expr
        return other, other

    def _record(self, concrete_result: bool, symbolic_expr) -> bool:
        self.path_condition.append(symbolic_expr if concrete_result else Not(
            symbolic_expr))
        return concrete_result

    def __eq__(self, other):
        other_val, other_expr = self._wrap(other)
        return self._record(self.concrete_value == other_val, self.symbolic_expr
            == other_expr)

    def __ne__(self, other):
        other_val, other_expr = self._wrap(other)
        return self._record(self.concrete_value != other_val, self.symbolic_expr
            != other_expr)

    def __lt__(self, other):
        other_val, other_expr = self._wrap(other)
        return self._record(self.concrete_value < other_val, self.symbolic_expr <
             other_expr)

    def __gt__(self, other):
        other_val, other_expr = self._wrap(other)
```

```
        return self._record(self.concrete_value > other_val, self.symbolic_expr >
            other_expr)

    def __le__(self, other):
        other_val, other_expr = self._wrap(other)
        return self._record(self.concrete_value <= other_val, self.symbolic_expr
            <= other_expr)

    def __ge__(self, other):
        other_val, other_expr = self._wrap(other)
        return self._record(self.concrete_value >= other_val, self.symbolic_expr
            >= other_expr)

    def __add__(self, other):
        other_val, other_expr = self._wrap(other)
        return SymInt(self.concrete_value + other_val, self.symbolic_expr +
            other_expr, self.path_condition)

    __radd__ = __add__

    def __sub__(self, other):
        other_val, other_expr = self._wrap(other)
        return SymInt(self.concrete_value - other_val, self.symbolic_expr -
            other_expr, self.path_condition)

    def __rsub__(self, other):
        other_val, other_expr = self._wrap(other)
        return SymInt(other_val - self.concrete_value, other_expr - self.
            symbolic_expr, self.path_condition)

    def __mul__(self, other):
        other_val, other_expr = self._wrap(other)
        return SymInt(self.concrete_value * other_val, self.symbolic_expr *
            other_expr, self.path_condition)

    __rmul__ = __mul__

    def __hash__(self):
        return hash(self.concrete_value)

    def __repr__(self):
        return f"SymInt({self.concrete_value})"


@dataclass
class WhiteboxFuzzResult:
    found_target: bool
    solution: Optional[Dict[str, int]]
    iterations_used: int
    path_conditions_tried: int

```

```python

def _run_once(target: Callable, concrete_values: List[int], var_names: List[str])
    -> Tuple[Any, List]:
    """Chay 'target' mot lan voi gia tri cu the, tra ve (output, path_condition).
       """
    path_condition: List = []
    sym_args = [
        SymInt(val, Int(name), path_condition)
        for val, name in zip(concrete_values, var_names)
    ]
    output = target(*sym_args)
    return output, path_condition


def whitebox_fuzz(
    target: Callable,
    num_vars: int = 1,
    target_branch: Any = None,
    initial_values: Optional[List[int]] = None,
    max_iterations: int = 50,
) -> WhiteboxFuzzResult:
    """Concolic fuzzing don gian: tim gia tri input dan toi 'target'
    tra ve dung bang 'target_branch'.

    target: ham nhan cac tham so SymInt (hoat dong nhu int binh thuong
            qua cac phep so sanh da overload), tra ve mot gia tri de
            so sanh voi target_branch (thuong la string danh dau
            nhanh, xem vi du trong __main__).
    num_vars: so luong tham so int cua target.
    target_branch: gia tri output mong muon dat duoc.
    initial_values: gia tri cu the ban dau (mac dinh: toan so 0).
    max_iterations: so lan phu dinh/giai lai toi da (Generational
            Search, muc 4.6).
    """
    _require_z3()
    var_names = [f"x{i}" for i in range(num_vars)] if num_vars > 1 else ["x"]
    current_values = list(initial_values) if initial_values else [0] * num_vars

    tried_path_conditions = 0

    for iteration in range(max_iterations):
        output, path_condition = _run_once(target, current_values, var_names)
        tried_path_conditions += 1

        if output == target_branch:
            return WhiteboxFuzzResult(
                True,
                dict(zip(var_names, current_values)),
                iteration + 1,
                tried_path_conditions,
            )
```

```
169
170         if not path_condition:
171             # Khong co dieu kien nao de phu dinh (ham khong re nhanh theo input)
172             break
173
174         # Generational Search (SAGE, muc 4.6): thu phu dinh TUNG dieu kien
175         # mot, tu dieu kien CUOI (gan nhat voi diem re nhanh cuoi) tro ve dau
176         found_new = False
177         for idx in range(len(path_condition) - 1, -1, -1):
178             s = Solver()
179             for j, cond in enumerate(path_condition):
180                 s.add(Not(cond) if j == idx else cond)
181             if s.check() == sat:
182                 model = s.model()
183                 new_values = []
184                 for name in var_names:
185                     v = model.eval(Int(name), model_completion=True)
186                     new_values.append(v.as_long())
187                 current_values = new_values
188                 found_new = True
189                 break
190
191         if not found_new:
192             # Moi nhanh phu dinh deu UNSAT (infeasible path, muc 4.6) - dung
193             break
194
195     return WhiteboxFuzzResult(False, None, max_iterations, tried_path_conditions)
196
197
198 if __name__ == "__main__":
199     def target_magic_byte(x):
200         """Ham vi du: chi 'CRASH' khi x dung chinh xac gia tri magic
201         0x1337 - minh hoa chinh xac van de da hoc o muc 4.3/4.5/4.6."""
202         if x == 0x1337:
203             return "CRASH"
204         return "OK"
205
206     print("=== White-box fuzzing: tim gia tri magic 0x1337 ===")
207     result = whitebox_fuzz(target_magic_byte, num_vars=1, target_branch="CRASH")
208     print(f"  Tim thay: {result.found_target}")
209     print(f"  Solution: {result.solution}")
210     print(f"  So lan thu: {result.iterations_used}")
211     if result.solution:
212         found_hex = hex(result.solution['x'])
213         print(f"  (Kiem tra: {found_hex} == {hex(0x1337)} -> {found_hex == hex(0
            x1337)})")
214
215     print("\n=== White-box fuzzing: dieu kien long nhau (2 bien) ===")
216     def target_nested(x, y):
217         if x > 0:
218             if y == x * 2:
```

```
                return "CRASH"
        return "OK"

    result2 = whitebox_fuzz(target_nested, num_vars=2, target_branch="CRASH",
                            initial_values=[1, 0])
    print(f"  Tim thay: {result2.found_target}, Solution: {result2.solution}")
```

Listing A.14: softsec/ch4_dynafuzz/whitebox_fuzzer.py

CHƯƠNG B

Gợi ý giải bài tập chọn lọc

Phụ lục này cung cấp gợi ý (không phải lời giải đầy đủ) cho một số bài tập tự luận tiêu biểu ở mỗi chương, nhằm giúp sinh viên tự kiểm tra hướng tiếp cận trước khi đối chiếu với đáp án chi tiết được cung cấp trên hệ thống quản lý học tập của học phần.

B.1 Chương 1

Bài 3 (Kripke Structure cho ATM). Gợi ý: xác định tối thiểu 4 trạng thái – `cho_the`, `nhap_pin`, `giao_dich`, `khoa_the` – và chuyển đổi hợp lệ giữa chúng (ví dụ `nhap_pin` chỉ chuyển sang `khoa_the` sau đúng 3 lần nhập sai liên tiếp, không phải ngay lần đầu). Thuộc tính safety cần kiểm chứng: $G(\neg(\texttt{giao_dich} \wedge \texttt{chua_xac_thuc}))$ – không bao giờ vào trạng thái giao dịch mà chưa xác thực PIN thành công.

B.2 Chương 2

Bài 3 (sizeof struct). Trên kiến trúc 64-bit với alignment mặc định: `struct S { char a; double b; char c; short d; }` có `sizeof` bằng 24 byte (không phải $1+8+1+2=12$), do `double` yêu cầu alignment 8 byte nên cần 7 byte padding sau `a`, và cần padding cuối để tổng kích thước chia hết cho alignment lớn nhất (8). Sắp xếp lại theo thứ tự giảm dần kích thước (`double b; short d; char a; char c;`) giảm xuống còn 16 byte.

B.3 Chương 3

Bài 1 (DPLL bằng tay). Gợi ý: bắt đầu bằng unit propagation nếu có unit clause; với công thức đã cho, thử gán $x_3 = \text{True}$ trước, kiểm tra mâu thuẫn với mệnh đề $(\neg x_2 \vee \neg x_3)$ và $(x_1 \vee \neg x_3)$ để suy ra ràng buộc trên x_1, x_2; nếu nhánh này dẫn tới mâu thuẫn, quay lui và thử $x_3 = \text{False}$.

B.4 Chương 4

Bài 2 (Branch coverage không đảm bảo path coverage). Gợi ý: với hàm có hai `if` độc lập tuần tự `if (a) {...} if (b) {...}`, chỉ cần 2 test case (`a=True,b=False` và `a=False,b=True`) đã đạt 100% branch coverage cho cả 4

nhánh, nhưng tổ hợp `(a=True, b=True)` và `(a=False, b=False)` chưa từng được thực thi – 2 trong 4 đường thực thi (path) hoàn chỉnh chưa được kiểm tra.

B.5 Lời giải chi tiết bổ sung

Phần này trình bày lời giải đầy đủ (không chỉ gợi ý) cho các bài tập lập trình tiêu biểu, giúp sinh viên đối chiếu sau khi tự làm.

B.5.1 Chương 1 – Mở rộng `classify_risk` thành 5 mức

```python
def classify_risk_5(score: int) -> str:
    """Phan loai diem rui ro (1..25) thanh 5 muc."""
    if not (1 <= score <= 25):
        raise ValueError(f"score phai trong [1,25], nhan {score}")
    if score <= 4:
        return "Rat thap"
    elif score <= 9:
        return "Thap"
    elif score <= 14:
        return "Trung binh"
    elif score <= 19:
        return "Cao"
    else:
        return "Rat cao"

# Test bang pytest
def test_classify_risk_5_boundaries():
    assert classify_risk_5(1) == "Rat thap"
    assert classify_risk_5(4) == "Rat thap"
    assert classify_risk_5(5) == "Thap"
    assert classify_risk_5(9) == "Thap"
    assert classify_risk_5(10) == "Trung binh"
    assert classify_risk_5(14) == "Trung binh"
    assert classify_risk_5(15) == "Cao"
    assert classify_risk_5(19) == "Cao"
    assert classify_risk_5(20) == "Rat cao"
    assert classify_risk_5(25) == "Rat cao"

def test_classify_risk_5_invalid():
    import pytest
    with pytest.raises(ValueError):
        classify_risk_5(0)
    with pytest.raises(ValueError):
        classify_risk_5(26)
```

Listing B.1: Lời giải: phân loại rủi ro 5 mức

B.5.2 Chương 2 – Mở rộng DANGEROUS_FUNCTIONS

```
DANGEROUS_FUNCTIONS_EXTENDED = {
    "strcpy":  ("CWE-120", "Khong kiem tra do dai dich; dung strncpy/strlcpy"),
    "strcat":  ("CWE-120", "Khong kiem tra do dai dich; dung strncat/strlcat"),
    "sprintf": ("CWE-120", "Khong gioi han do dai buffer dich; dung snprintf"),
    "gets":    ("CWE-242", "Da bi loai bo khoi C11; dung fgets"),
    "scanf":   ("CWE-134", "Format string khong gioi han do dai %s; luon dat gioi
        han"),
    "system":  ("CWE-78",  "Co the bi command injection neu tham so tu input
       nguoi dung"),
    "strtok":  ("CWE-362", "Khong an toan trong da luong (khong reentrant); dung
       strtok_r"),
    "alloca":  ("CWE-770", "Cap phat tren stack khong kiem soat, co the gay stack
        overflow"),
}

import re

def scan_source_v2(source: str) -> list[dict]:
    findings = []
    in_block_comment = False
    for i, line in enumerate(source.splitlines(), start=1):
        stripped = line.strip()
        # Bo qua don gian: dong nam trong comment mot dong hoac string literal
        if stripped.startswith("//") or stripped.startswith("*"):
            continue
        if stripped.startswith('"') and stripped.endswith('"'):
            continue
        for func, (cwe, reason) in DANGEROUS_FUNCTIONS_EXTENDED.items():
            if re.search(rf"\b{func}\s*\(", line) and "//" not in line.split(func
                )[0]:
                findings.append({
                    "line": i, "function": func, "cwe": cwe,
                    "reason": reason, "snippet": stripped,
                })
    return findings

# Test truong hop false positive da biet: strcpy trong comment
def test_scan_ignores_comment():
    src = '// vi du: strcpy(a, b) la nguy hiem\nint x = 1;'
    findings = scan_source_v2(src)
    assert len(findings) == 0
```

Listing B.2: Lời giải: mở rộng danh sách hàm nguy hiểm với xử lý false positive

B.5.3 Chương 3 – Kiểm tra mảng với biểu thức chỉ số phức tạp

```
def check_expression_bounds(array_size: int, a_range: range, b_range: range):
```

```
    """Duyet moi to hop (a,b), tinh i = (a*3+b) % 12, kiem tra co vuot
    bien array_size hay khong. Mo phong don gian BMC cho bieu thuc phuc tap."""
    violations = []
    for a in a_range:
        for b in b_range:
            i = (a * 3 + b) % 12
            if not (0 <= i < array_size):
                violations.append((a, b, i))
    return violations

# Voi array_size=10, range(0,5), range(0,5):
# i = (a*3+b) % 12 co the dat gia tri 10, 11 (>= 10) -> VI PHAM
# Vi du: a=3,b=1 -> i=10; a=3,b=2 -> i=11
result = check_expression_bounds(10, range(0, 5), range(0, 5))
assert len(result) > 0   # xac nhan tim thay counterexample
```

Listing B.3: Lời giải: kiểm tra bounds cho biểu thức chỉ số i = (a*3+b) mod 12

B.5.4 Chương 4 – Đo hiệu quả random mutation vs. SMT solving

```
import random, time

MAGIC = bytes([0x89, 0x50, 0x4E, 0x47])  # 4 byte magic (giong PNG header)

def target_with_magic_check(data: bytes) -> bool:
    """Tra ve True neu 'vuot qua' duoc magic byte check."""
    return data[:4] == MAGIC

def random_mutation_attempts(max_tries: int = 2_000_000) -> int | None:
    for attempt in range(1, max_tries + 1):
        candidate = bytes(random.randint(0, 255) for _ in range(4))
        if target_with_magic_check(candidate):
            return attempt
    return None   # khong tim thay trong gioi han thu

# Ket qua thuc te: xac suat trung 1 lan la 1/256^4 (~2.3 ty),
# nen voi max_tries=2 trieu, ham tren hau nhu LUON tra ve None.
# Trong khi do solve_magic_bytes (Listing 4.wbfuzz o Chuong 4) giai
# duoc CHINH XAC trong mot loi goi solver duy nhat (< 10ms thuc te).
print("Random mutation: khong kha thi trong thoi gian hop ly")
print("SMT solving: giai duoc ngay lap tuc, do phuc tap khong phu thuoc")
print("vao kich thuoc khong gian gia tri, ma phu thuoc vao so luong")
print("rang buoc can giai (o day chi 4 rang buoc dang thuc).")
```

Listing B.4: Lời giải: so sánh thời gian vượt qua magic byte check

CHƯƠNG C

Đề thi mẫu và đáp án chi tiết

Phụ lục này cung cấp bốn đề thi mẫu đầy đủ, mỗi đề tương ứng với nội dung một chương, theo đúng cấu trúc đề thi giữa kỳ/cuối kỳ của học phần CSE703093: Phần I (trắc nghiệm, 20 câu, 4 điểm), Phần II (tự luận ngắn, 3 câu, 3 điểm), Phần III (bài tập lập trình/tính toán, 2 câu, 3 điểm). Đáp án chi tiết kèm giải thích được cung cấp ngay sau mỗi đề để sinh viên tự đối chiếu.

C.1 Đề thi mẫu Chương 1: Giới thiệu về an toàn phần mềm

Phần I – Trắc nghiệm (mỗi câu 0.2 điểm)

1. Thuộc tính nào trong CIA đảm bảo dữ liệu không bị sửa đổi trái phép? **(a)** Confidentiality **(b)** Integrity **(c)** Availability **(d)** Authenticity
2. Một hệ thống chống được mọi cuộc tấn công mạng nhưng vẫn có thể gây tai nạn do lỗi logic thuần túy. Hệ thống này: **(a)** An toàn bảo mật cao, an toàn vận hành thấp **(b)** An toàn vận hành cao, an toàn bảo mật thấp **(c)** Cả hai đều cao **(d)** Cả hai đều thấp
3. CVE-2014-0160 (Heartbleed) thuộc lớp lỗi CWE nào? **(a)** SQL Injection **(b)** Use-after-free **(c)** Out-of-bounds Read **(d)** Race Condition
4. Trong công thức $Risk = Likelihood \times Impact$, nếu Likelihood=3, Impact=5 (thang 1–5), phân loại rủi ro theo ngưỡng đã học (Thấp $\leq$6, Trung bình $\leq$14, Cao $>$14) là: **(a)** Thấp **(b)** Trung bình **(c)** Cao **(d)** Không xác định
5. Chữ ”S” trong STRIDE đại diện cho mối đe dọa nào? **(a)** Spoofing **(b)** Sniffing **(c)** Scanning **(d)** Session hijacking
6. Kỹ thuật kiểm thử nào dựa trên tổ hợp điều kiện logic? **(a)** Equivalence partitioning **(b)** Decision table **(c)** Boundary value analysis **(d)** Error guessing
7. Trong Kripke Structure $K = (S, S_0, R, L)$, hàm L có vai trò gì? **(a)** Xác định trạng thái khởi tạo **(b)** Gán nhãn mệnh đề nguyên tử cho mỗi trạng thái **(c)** Xác định quan hệ chuyển trạng thái **(d)** Tính số lượng trạng thái
8. Công thức LTL $G\varphi$ có nghĩa là gì? **(a)** φ đúng tại trạng thái kế tiếp **(b)** φ đúng tại một thời điểm nào đó trong tương lai **(c)** φ đúng tại mọi trạng thái trên toàn bộ path **(d)** φ chưa từng đúng

9. Verification khác Validation ở điểm nào? **(a)** Verification kiểm tra đúng theo đặc tả, Validation kiểm tra đáp ứng nhu cầu thực tế **(b)** Verification luôn làm trước Validation **(c)** Không có sự khác biệt **(d)** Validation chỉ áp dụng cho phần cứng
10. Nguyên tắc đặc quyền tối thiểu (Least Privilege) nhằm mục đích gì? **(a)** Tăng hiệu năng hệ thống **(b)** Giảm thiệt hại nếu một thành phần bị xâm nhập **(c)** Đơn giản hóa mã nguồn **(d)** Giảm chi phí phát triển
11. CWE là viết tắt của: **(a)** Common Weakness Enumeration **(b)** Common Web Exploit **(c)** Central Weakness Explorer **(d)** Critical Web Error
12. Trong ma trận rủi ro 3x3, một lỗ hổng có Likelihood=Cao, Impact=Thấp thường được xếp vào mức: **(a)** Thấp **(b)** Trung bình **(c)** Cao **(d)** Rất cao
13. Attack tree sử dụng phép toán logic nào để biểu diễn ”cần thực hiện đồng thời nhiều bước con để đạt mục tiêu”? **(a)** OR **(b)** AND **(c)** XOR **(d)** NOT
14. Test case nào sau đây thuộc kỹ thuật Boundary Value Analysis cho miền hợp lệ $[1, 100]$? **(a)** 50 **(b)** 0, 1, 2, 99, 100, 101 **(c)** -999, 999 **(d)** Chỉ giá trị 1
15. Trong mô hình chữ V, System Test tương ứng với giai đoạn thiết kế nào? **(a)** Module Design **(b)** Architecture Design **(c)** System Design **(d)** Requirements
16. Use Case Testing thiết kế test case dựa trên: **(a)** Tổ hợp điều kiện logic **(b)** Kịch bản sử dụng thực tế (main/alternative/exception flow) **(c)** Giá trị biên của miền dữ liệu **(d)** Kinh nghiệm cá nhân của tester
17. Trong Kripke Structure, yêu cầu ”total” của quan hệ chuyển trạng thái R nghĩa là gì? **(a)** Mọi trạng thái đều có ít nhất một trạng thái kế tiếp **(b)** R phải là quan hệ đối xứng **(c)** $|S|$ phải là số hữu hạn **(d)** Mọi trạng thái đều là trạng thái khởi tạo
18. Công thức LTL $\varphi \, U \, \psi$ (Until) có nghĩa: **(a)** φ đúng ngay sau khi ψ đúng **(b)** φ đúng liên tục cho tới khi ψ trở thành đúng **(c)** φ và ψ không bao giờ cùng đúng **(d)** ψ phải đúng trước φ
19. ALE (Annualized Loss Expectancy) được tính bằng công thức nào? **(a)** $ALE = SLE + ARO$ **(b)** $ALE = SLE \times ARO$ **(c)** $ALE = SLE / ARO$ **(d)** $ALE = SLE - ARO$

Đáp án Phần I

1-b, 2-a, 3-c, 4-b, 5-a, 6-b, 7-b, 8-c, 9-a, 10-b, 11-b, 12-a, 13-b, 14-b, 15-b, 16-b, 17-b, 18-a, 19-b, 20-b.

Phần II – Tự luận (mỗi câu 1 điểm)

Câu 1. Phân biệt Authentication và Authorization bằng một ví dụ cụ thể trong hệ thống ngân hàng trực tuyến.

Đáp án gợi ý: Authentication là bước xác minh khách hàng đăng nhập đúng là chủ tài khoản (qua mật khẩu + OTP – MFA). Authorization là bước xác định sau khi đăng nhập, khách hàng đó được phép chuyển khoản tối đa bao nhiêu, có được xem lịch sử giao dịch của tài khoản khác không. Nhầm lẫn hai bước này là lỗi *Broken Access Control* kinh điển: một hệ thống có thể xác thực đúng nhưng vẫn cho phép khách hàng A xem được thông tin tài khoản khách hàng B nếu chỉ dựa vào ID trên URL mà không kiểm tra quyền sở hữu ở tầng server (lỗi IDOR).

Câu 2. Vẽ Kripke Structure cho hệ thống đèn giao thông đơn giản (3 trạng thái: đỏ, vàng, xanh) và viết một công thức LTL biểu diễn thuộc tính "đèn không bao giờ chuyển thẳng từ đỏ sang xanh".

Đáp án gợi ý: $S = \{do, vang, xanh\}$, $R = \{(do, xanh)$ *bị loại bỏ*$, (xanh, vang), (vang, do)\}$ – tức chu trình bắt buộc $xanh \to vang \to do \to xanh$. Thuộc tính an toàn: $G(do \to \neg X(xanh))$: tại mọi trạng thái, nếu đang ở đỏ thì trạng thái kế tiếp không phải là xanh.

Câu 3. Giải thích vì sao Định lý Rice khiến không công cụ phân tích tĩnh nào có thể vừa sound vừa complete cho mọi chương trình.

Đáp án gợi ý: Rice's Theorem chứng minh mọi thuộc tính ngữ nghĩa không tầm thường của chương trình (ví dụ "chương trình này có bao giờ crash không") là bất khả quyết (undecidable) – tương đương bài toán dừng. Nếu một công cụ vừa sound (không bỏ sót lỗi thật) vừa complete (không báo động giả), nó sẽ giải quyết được bài toán bất khả quyết đó một cách chính xác cho mọi chương trình – mâu thuẫn với Rice's Theorem. Do đó mọi công cụ thực tế phải hy sinh một trong hai tính chất, hoặc giới hạn phạm vi bài toán (như BMC giới hạn độ sâu k).

Phần III – Bài tập tính toán/lập trình (mỗi câu 1.5 điểm)

Câu 1. Cho hàm kiểm tra tuổi hợp lệ để mua rượu: $18 \le tuoi \le 100$. Liệt kê đầy đủ các lớp tương đương và 6 giá trị biên.

Đáp án: Lớp tương đương: $\{tuoi < 18\}$ (không hợp lệ), $\{18 \le tuoi \le 100\}$ (hợp lệ), $\{tuoi > 100\}$ (không hợp lệ). Giá trị biên: 17, 18, 19, 99, 100, 101.

Câu 2. Sử dụng module `ch1_models.risk`, viết đoạn code Python tính điểm rủi ro và phân loại cho 3 lỗ hổng có (Likelihood, Impact) lần lượt là (2,2), (4,4), (5,3).

Đáp án gợi ý:

```
from softsec.ch1_models.risk import risk_score, classify_risk
for l, i in [(2,2),(4,4),(5,3)]:
    s = risk_score(l, i)
    print(l, i, s, classify_risk(s))
# Ket qua: (2,2)->4 Thap; (4,4)->16 Cao; (5,3)->15 Cao
```

C.2 Đề thi mẫu Chương 2: Lập trình C an toàn

Phần I – Trắc nghiệm (chọn lọc 8 câu)

1. Vùng nhớ nào tăng địa chỉ từ thấp lên cao trong mô hình bộ nhớ tiến trình Linux điển hình? **(a)** Stack **(b)** Heap **(c)** Text **(d)** Cả 3 vùng
2. Sau khi gọi `free(ptr)`, giá trị của `ptr`: **(a)** Tự động thành `NULL` **(b)** Vẫn giữ nguyên địa chỉ cũ (dangling) **(c)** Gây lỗi biên dịch **(d)** Không xác định về mặt cú pháp
3. Hàm nào sau đây được xem là an toàn nhất để copy chuỗi có giới hạn độ dài đích? **(a)** `strcpy` **(b)** `strcat` **(c)** `snprintf` **(d)** `gets`
4. Tràn số nguyên có dấu (signed integer overflow) trong C là: **(a)** Được định nghĩa rõ ràng (wrap-around) **(b)** Undefined Behavior **(c)** Luôn gây lỗi biên dịch **(d)** Chỉ xảy ra trên kiến trúc 32-bit
5. Cơ chế nào chèn giá trị ngẫu nhiên giữa biến local và địa chỉ trả về để phát hiện stack buffer overflow? **(a)** ASLR **(b)** Stack Canary **(c)** DEP/NX **(d)** RELRO
6. AddressSanitizer phát hiện lỗi bằng cơ chế nào? **(a)** Giả lập toàn bộ CPU (như Valgrind) **(b)** Shadow memory với redzone quanh mỗi buffer **(c)** Phân tích tĩnh trước khi chạy **(d)** Kiểm tra kiểu dữ liệu runtime
7. Trường padding trong một struct C: **(a)** Luôn được khởi tạo bằng 0 **(b)** Không được khởi tạo, có thể chứa dữ liệu cũ trên stack/heap **(c)** Không tồn tại trên kiến trúc 64-bit **(d)** Chỉ xuất hiện khi dùng `#pragma pack`
8. CVE-2021-3156 (Sudo Baron Samedit) thuộc loại lỗi nào? **(a)** Use-after-free **(b)** Heap buffer overflow do tính sai độ dài **(c)** SQL Injection **(d)** Race condition
9. Attack surface trong mã C KHÔNG bao gồm: **(a)** Tham số dòng lệnh **(b)** Dữ liệu từ file/socket **(c)** Hằng số được khai báo cứng trong mã nguồn **(d)** Dữ liệu từ thư viện định dạng file bên thứ ba
10. Trong mô hình bộ nhớ tiến trình, vùng nào chứa mã lệnh và thường ở chế độ chỉ đọc? **(a)** Heap **(b)** Stack **(c)** Text **(d)** BSS
11. Double free có thể dẫn tới hậu quả gì nghiêm trọng nhất? **(a)** Chỉ làm chậm chương trình **(b)** Hỏng metadata heap allocator, có thể dẫn tới RCE **(c)** Luôn gây lỗi biên dịch **(d)** Không có hậu quả nếu con trỏ là NULL
12. Vì sao alignment quan trọng với CPU? **(a)** CPU đọc/ghi bộ nhớ theo word có kích thước cố định, bắt đầu tại địa chỉ chia hết cho kích thước đó **(b)** Chỉ để tiết kiệm dung lượng đĩa **(c)** Không có ảnh hưởng thực tế nào **(d)** Chỉ áp dụng cho kiểu `char`
13. cppcheck và clang-tidy thuộc loại công cụ nào? **(a)** Dynamic analysis **(b)** Static analysis **(c)** Fuzzing **(d)** Formal proof assistant
14. Valgrind Memcheck phát hiện lỗi bằng cách nào? **(a)** Phân tích mã nguồn tĩnh **(b)** Giả lập CPU để theo dõi từng byte bộ nhớ khi chạy **(c)** Symbolic execution **(d)** SAT solving

15. Nguyên tắc ”mỗi hàm tạo có một hàm hủy đối xứng” là tiền thân trực tiếp của khái niệm nào trong C++/Rust? **(a)** Garbage collection **(b)** RAII **(c)** Reflection **(d)** Templates
16. Lỗi format string (`printf(user_input)`) nguy hiểm vì: **(a)** Luôn làm chương trình chạy chậm **(b)** Cho phép kẻ tấn công đọc/ghi bộ nhớ qua các specifier như %x, %n **(c)** Chỉ ảnh hưởng tới định dạng hiển thị **(d)** Không tồn tại trong C hiện đại
17. Sự kiện Morris Worm (1988) khai thác lỗ hổng nào? **(a)** SQL Injection trong web server **(b)** Buffer overflow qua hàm `gets()` trong `fingerd` **(c)** Race condition trong kernel **(d)** Format string trong `syslog`
18. Stack canary hoạt động dựa trên nguyên lý nào? **(a)** Mã hóa toàn bộ stack **(b)** Chèn giá trị ngẫu nhiên giữa biến local và return address, kiểm tra trước khi return **(c)** Giới hạn kích thước stack tối đa **(d)** Không cho phép đệ quy

Đáp án Phần I

1-b, 2-b, 3-c, 4-b, 5-b, 6-b, 7-b, 8-b, 9-c, 10-b, 11-c, 12-b, 13-a, 14-b, 15-b, 16-b, 17-b.

Phần II – Tự luận

Câu 1. Giải thích vì sao trình biên dịch tối ưu hóa có thể loại bỏ một kiểm tra bảo mật hợp lý về mặt logic nếu kiểm tra đó dựa trên hành vi undefined behavior. Cho ví dụ cụ thể.

Đáp án gợi ý: Xem Listing 2.3 (Chương 2): kiểm tra `if (a + b < a)` để phát hiện tràn số dựa trên giả định phép cộng signed int có thể ”wrap around” – nhưng chuẩn C quy định tràn số signed là UB, nghĩa là trình biên dịch được phép giả định nó *không bao giờ xảy ra*. Dưới giả định đó, trình biên dịch tối ưu suy luận `a + b < a` luôn sai (vì nếu không tràn, tổng luôn $\geq a$ khi $b \geq 0$), nên có thể loại bỏ toàn bộ nhánh `if` đó – vô hiệu hóa chính kiểm tra bảo mật mà lập trình viên đã cẩn thận viết ra.

Câu 2. Trình bày 3 cơ chế giảm nhẹ (mitigation) tầng hệ điều hành/ trình biên dịch và giải thích tại sao chúng không thay thế được việc sửa lỗi gốc.

Đáp án gợi ý: Xem bảng mitigation ở Chương 2 (Stack Canary, ASLR, DEP/NX). Chúng làm tăng độ khó khai thác (ví dụ ASLR khiến kẻ tấn công khó đoán địa chỉ) nhưng không loại bỏ lỗ hổng gốc – một kẻ tấn công có kỹ thuật đủ tinh vi (ví dụ rò rỉ thông tin để vượt qua ASLR, hoặc dùng kỹ thuật ROP để vượt qua DEP/NX) vẫn có thể khai thác thành công nếu lỗi bộ nhớ gốc chưa được sửa.

Câu 3. Tính toán padding cho struct sau trên kiến trúc 64-bit và đề xuất cách tối ưu:

```
struct Packet { char type; long timestamp; short id; };
```

Đáp án gợi ý: `long` (8 byte) yêu cầu alignment 8, nên sau `char type` (1 byte) cần 7 byte padding trước `timestamp`. Sau `short id` (2 byte), cần 6 byte padding

cuối để tổng kích thước chia hết cho 8. Tổng: $1+7+8+2+6 = 24$ byte. Sắp xếp lại: `long timestamp; short id; char type;` giảm còn $8 + 2 + 1 + 5(padding) = 16$ byte.

Phần III – Bài tập lập trình

Câu 1. Viết một chương trình C chứa lỗi double-free có chủ đích, biên dịch với ASan, và trình bày thông báo lỗi dự kiến.

Đáp án gợi ý: Chương trình gọi `free(p); free(p);`. ASan sẽ báo lỗi `"attempting double-free"` kèm 3 stack trace: nơi xảy ra double-free hiện tại, nơi `free()` lần đầu đã xảy ra, và nơi `malloc()` đã cấp phát ban đầu – giúp xác định chính xác nguồn gốc lỗi mà không cần công cụ debug thủ công.

Câu 2. Sử dụng `pattern_scanner.py` (Phụ lục A) để quét một file mã nguồn C cho trước, liệt kê tất cả các lời gọi hàm nguy hiểm và đề xuất hàm thay thế.

C.3 Đề thi mẫu Chương 3: Phân tích tĩnh

Phần I – Trắc nghiệm (chọn lọc 8 câu)

1. Một công cụ ”sound” đảm bảo điều gì? **(a)** Không bao giờ báo động giả **(b)** Không bao giờ bỏ sót lỗi thật **(c)** Luôn nhanh hơn kiểm thử động **(d)** Luôn tìm được counterexample
2. SAT là bài toán thuộc lớp độ phức tạp nào? **(a)** P **(b)** NP-complete **(c)** Bất khả quyết **(d)** PSPACE
3. Kỹ thuật nào KHÔNG thuộc DPLL? **(a)** Unit propagation **(b)** Pure literal elimination **(c)** Conflict-driven clause learning gốc **(d)** Backtracking
4. SMT khác SAT ở điểm nào? **(a)** SMT nhanh hơn SAT luôn luôn **(b)** SMT cho phép ràng buộc trong các lý thuyết nền (bit-vector, array...) **(c)** SMT không dùng SAT solver bên trong **(d)** SMT chỉ áp dụng cho phần cứng
5. Bounded Model Checking quy bài toán kiểm chứng về: **(a)** Bài toán tối ưu tuyến tính **(b)** Bài toán SAT/SMT **(c)** Bài toán đồ thị Euler **(d)** Bài toán quy hoạch động
6. Kết quả UNSAT của BMC ở độ sâu k có ý nghĩa gì? **(a)** Chương trình an toàn mãi mãi **(b)** Chương trình an toàn trong giới hạn k bước **(c)** Chương trình chắc chắn có lỗi **(d)** Không có ý nghĩa gì
7. K-induction bổ sung gì cho BMC thuần túy? **(a)** Tăng tốc độ solver **(b)** Cho phép chứng minh unbounded (không giới hạn) **(c)** Giảm bộ nhớ sử dụng **(d)** Loại bỏ nhu cầu dùng SMT solver
8. Theory of Arrays trong SMT mô hình hóa gì? **(a)** Số thực dấu phẩy động **(b)** Bộ nhớ như một hàm từ địa chỉ tới giá trị **(c)** Chuỗi ký tự Unicode **(d)** Đồ thị có hướng

9. Binary Decision Diagram (BDD) biểu diễn hàm Boolean dưới dạng: **(a)** Bảng chân trị đầy đủ **(b)** Đồ thị có hướng không chu trình đã rút gọn, canonical với thứ tự biến cố định **(c)** Ma trận thưa **(d)** Danh sách liên kết đơn
10. Abstract Interpretation đảm bảo tính chất nào theo thiết kế? **(a)** Completeness **(b)** Soundness **(c)** Tốc độ tối ưu **(d)** Không cần miền trừu tượng
11. Công cụ Astrée đã được dùng để chứng minh điều gì cho phần mềm điều khiển bay Airbus A380? **(a)** Không có race condition **(b)** Không có runtime error (buffer overflow, division by zero...) **(c)** Không có SQL injection **(d)** Tốc độ thực thi tối ưu
12. seL4 là: **(a)** Một SAT solver thương mại **(b)** Hạt nhân hệ điều hành được chứng minh hình thức đầy đủ bằng Isabelle/HOL **(c)** Một ngôn ngữ lập trình mới thay thế C **(d)** Một fuzzer cho kernel Linux
13. DPLL(T) trong kiến trúc SMT solver đề cập tới: **(a)** Kết hợp SAT solver CDCL ở tầng Boolean với theory solver chuyên biệt **(b)** Một biến thể của thuật toán sắp xếp **(c)** Kỹ thuật nén dữ liệu **(d)** Giao thức mạng

Đáp án Phần I

1-b, 2-b, 3-c, 4-b, 5-b, 6-b, 7-b, 8-b, 9-b, 10-b, 11-b, 12-b, 13-b, 14-a.

Phần II – Tự luận

Câu 1. Trình bày sự khác biệt giữa Symbolic Model Checking (BDD) và Bounded Model Checking (SAT/SMT) về độ đầy đủ và khả năng mở rộng.

Đáp án gợi ý: Xem bảng so sánh Chương 3. BDD-based: đầy đủ (unbounded) nhưng rất nhạy cảm với thứ tự biến, dễ bùng nổ kích thước với hệ thống lớn; phù hợp mạch phần cứng. SAT/SMT-based BMC: chỉ đúng trong giới hạn k (trừ khi kết hợp k-induction), nhưng mở rộng tốt hơn cho phần mềm nhờ lý thuyết bit-vector/mảng mô hình hóa trực tiếp ngữ nghĩa C.

Câu 2. Giải một công thức CNF bằng DPLL: $(x_1) \wedge (\neg x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge (\neg x_3)$.

Đáp án gợi ý: Unit propagation: (x_1) là unit clause $\Rightarrow x_1 = True$. Áp vào mệnh đề 2: $(\neg x_1 \vee x_2)$ với $x_1 = True$ trở thành unit $(x_2) \Rightarrow x_2 = True$. Áp vào mệnh đề 3: trở thành unit $(x_3) \Rightarrow x_3 = True$. Áp vào mệnh đề 4: $(\neg x_3)$ với $x_3 = True$ là **False** – mâu thuẫn. Kết luận: **UNSAT**.

Câu 3. Giải thích Astrée đã chứng minh điều gì cho phần mềm Airbus A380 và kỹ thuật nền tảng đứng sau nó là gì.

Đáp án gợi ý: Xem Case Study Chương 3. Astrée dùng Abstract Interpretation (miền trừu tượng như octagon, polyhedra) để chứng minh *sound* rằng toàn bộ phần mềm điều khiển bay không có runtime error (buffer overflow, division by zero...), không cần liệt kê từng trường hợp cụ thể mà suy luận trên miền giá trị trừu tượng bao trùm mọi khả năng.

Phần III – Bài tập lập trình

Câu 1. Dùng `sat_solver.py` để giải công thức ở Câu 2 Phần II bằng code, xác nhận kết quả UNSAT.

Câu 2. Dùng `smt_wrapper.check_overflow_smt` với `bit_width=8` để tìm một cặp (a, b) gây tràn số không dấu; giải thích kết quả solver trả về.

C.4 Đề thi mẫu Chương 4: Phân tích động

Phần I – Trắc nghiệm (chọn lọc 8 câu)

1. Oracle đơn giản nhất trong fuzzing là gì? **(a)** So sánh output với giá trị mong đợi chính xác **(b)** Crash / không crash **(c)** Đo thời gian thực thi **(d)** Kiểm tra log file
2. Path explosion xảy ra vì: **(a)** Số đường thực thi tăng tuyến tính với số điều kiện rẽ nhánh **(b)** Số đường thực thi tăng theo hàm mũ với số điều kiện rẽ nhánh độc lập **(c)** Bộ nhớ RAM không đủ **(d)** CPU quá chậm
3. Coverage-guided fuzzing khác random fuzzing ở điểm nào? **(a)** Dùng coverage làm tín hiệu phản hồi để giữ lại input ”tốt” **(b)** Không cần thực thi chương trình **(c)** Chỉ áp dụng cho mã Python **(d)** Luôn nhanh hơn nhưng kém chính xác hơn
4. Magic byte problem là gì? **(a)** Lỗi tràn số khi xử lý byte đầu tiên **(b)** Random fuzzing khó tự sinh đúng chuỗi byte cố định cần thiết để vượt qua kiểm tra định dạng **(c)** Lỗi liên quan tới alignment **(d)** Lỗi chỉ xảy ra trên kiến trúc little-endian
5. Concolic execution là viết tắt/kết hợp của: **(a)** Concrete + Symbolic **(b)** Concurrent + Logic **(c)** Control + Logic **(d)** Configuration + Lexical
6. AddressSanitizer sử dụng cơ chế nào để phát hiện buffer overflow? **(a)** Shadow memory với redzone **(b)** Symbolic execution **(c)** SAT solving **(d)** Kiểm tra kiểu dữ liệu tĩnh
7. Structure-aware fuzzing giải quyết vấn đề gì? **(a)** Tăng tốc độ biên dịch **(b)** Đảm bảo input đột biến vẫn hợp lệ về cú pháp cho định dạng có cấu trúc phức tạp **(c)** Giảm dung lượng corpus **(d)** Không liên quan tới fuzzing
8. Differential fuzzing cần gì để phát hiện lỗi? **(a)** Một oracle biết trước output đúng **(b)** Hai hoặc nhiều cài đặt độc lập của cùng một chức năng để so sánh kết quả **(c)** Symbolic execution bắt buộc **(d)** Mã nguồn phải là C/C++
9. AFL (American Fuzzy Lop) sử dụng chiến lược nào là chính? **(a)** Random fuzzing thuần túy không có phản hồi **(b)** Coverage-guided mutation-based fuzzing **(c)** Chỉ symbolic execution **(d)** Chỉ kiểm tra chữ ký định dạng file

10. Dự án OSS-Fuzz của Google có mục tiêu gì? **(a)** Fuzzing liên tục trên hàng nghìn dự án mã nguồn mở quan trọng **(b)** Chỉ kiểm thử phần mềm nội bộ của Google **(c)** Thay thế hoàn toàn static analysis **(d)** Chỉ áp dụng cho kernel

11. KLEE là công cụ dựa trên kỹ thuật nào? **(a)** Coverage-guided black-box fuzzing **(b)** Symbolic execution trên nền LLVM **(c)** Abstract interpretation **(d)** BDD-based model checking

12. Corpus minimization giải quyết bài toán gì? **(a)** Tăng tốc độ biên dịch **(b)** Loại bỏ input dư thừa, giữ lại tập nhỏ nhất vẫn bảo toàn coverage đã đạt được **(c)** Mã hóa dữ liệu corpus **(d)** Không liên quan tới fuzzing

13. Syzkaller thu thập coverage từ kernel Linux qua cơ chế nào? **(a)** ASan **(b)** KCOV **(c)** Valgrind **(d)** GDB

Đáp án Phần I

1-b, 2-b, 3-a, 4-b, 5-a, 6-a, 7-b, 8-b, 9-b, 10-b, 11-a, 12-b, 13-b.

Phần II – Tự luận

Câu 1. Trình bày luồng dữ liệu taint tracking hoàn chỉnh từ nguồn (source) tới điểm nhạy cảm (sink) cho một lỗi SQL Injection cụ thể.

Đáp án gợi ý: Nguồn: tham số HTTP `request.GET['name']` (tainted). Lan truyền: `query = "SELECT * FROM t WHERE n='" + name + "'"` (kết quả nối chuỗi vẫn tainted vì phép toán trên dữ liệu bẩn tạo ra dữ liệu bẩn). Sink: `cursor.execute(query)` – dữ liệu tainted chảy trực tiếp vào câu lệnh SQL thực thi mà không qua sanitization (tham số hóa) – đây chính là điểm lỗ hổng.

Câu 2. So sánh black-box fuzzing và white-box fuzzing về chi phí, tốc độ, và khả năng vượt qua magic byte problem.

Đáp án gợi ý: Xem bảng so sánh Chương 4. Black-box: nhanh, rẻ, phủ rộng nhưng bị chặn bởi magic byte. White-box (symbolic/concolic + SMT): chậm hơn nhiều (giải constraint tốn kém) nhưng giải quyết chính xác magic byte problem bằng cách giải ngược ràng buộc nhánh điều kiện. Thực tế công nghiệp dùng hybrid (Driller) kết hợp cả hai.

Câu 3. Giải thích vì sao Syzkaller đạt hiệu quả cao trong việc tìm lỗi kernel Linux dù mã nguồn đã được review bởi hàng nghìn kỹ sư.

Đáp án gợi ý: Xem Case Study Chương 4. Review thủ công khó phát hiện race condition và use-after-free trong tương tác đa luồng phức tạp của kernel; Syzkaller dùng coverage-guided fuzzing với đặc tả syscall (structure-aware) và coverage feedback trực tiếp từ kernel (KCOV) để khám phá tự động các tổ hợp trạng thái mà con người khó hình dung hết.

Phần III – Bài tập lập trình

Câu 1. Mở rộng `CoverageTracker` để log số lần mỗi edge được truy cập (thay vì chỉ có/không), dùng để phát hiện các edge ”hiếm” (hit count thấp) cần ưu tiên fuzzing sâu hơn.

Câu 2. Viết harness fuzzing đơn giản cho một hàm phân tích cú pháp số nguyên từ chuỗi (`parse_int`), kết hợp `fuzz_loop` (Listing 4.2) và `CoverageTracker`; báo cáo số crash tìm được sau 10.000 iteration.

C.5 Đề thi cuối kỳ tổng hợp (mẫu)

Đề thi cuối kỳ tổng hợp toàn bộ bốn chương, theo cấu trúc: Phần I (30 câu trắc nghiệm, 6 điểm), Phần II (4 câu tự luận, 2.5 điểm), Phần III (2 bài tập tổng hợp yêu cầu kết hợp kiến thức nhiều chương, 1.5 điểm). Thời gian làm bài: 90 phút, không được sử dụng tài liệu (trừ một tờ ghi chú A4 viết tay theo quy định của Khoa).

Phần I – Trắc nghiệm tổng hợp (chọn lọc 15 câu tiêu biểu)

1. Thuộc tính nào bị vi phạm khi kẻ tấn công đọc trộm dữ liệu qua lỗi Heartbleed? **(a)** Availability **(b)** Confidentiality **(c)** Integrity **(d)** Non-repudiation
2. Kỹ thuật kiểm thử nào thiết kế test case dựa trên chuyển đổi trạng thái của hệ thống? **(a)** Boundary Value Analysis **(b)** State Transition Testing **(c)** Decision Table **(d)** Error Guessing
3. Trong C, hành vi nào sau đây LÀ undefined behavior? **(a)** Unsigned integer overflow **(b)** Signed integer overflow **(c)** Chia số thực cho 0.0 **(d)** So sánh hai con trỏ cùng kiểu
4. Cơ chế bảo vệ nào ngăn thực thi mã trên vùng nhớ dữ liệu? **(a)** ASLR **(b)** Stack Canary **(c)** DEP/NX **(d)** RELRO
5. Một công cụ phân tích tĩnh ”sound” đảm bảo: **(a)** Không báo động giả **(b)** Không bỏ sót lỗi thật **(c)** Luôn nhanh **(d)** Luôn đầy đủ (complete)
6. DPLL sử dụng kỹ thuật nào để lan truyền giá trị khi một mệnh đề chỉ còn một literal chưa gán? **(a)** Pure literal elimination **(b)** Unit propagation **(c)** Backtracking **(d)** Resolution
7. Bounded Model Checking quy bài toán kiểm chứng phần mềm về: **(a)** Bài toán đồ thị **(b)** Bài toán SAT/SMT **(c)** Bài toán quy hoạch tuyến tính **(d)** Bài toán tối ưu tổ hợp
8. Path explosion là hiện tượng: **(a)** Số đường thực thi tăng theo hàm mũ với số nhánh điều kiện độc lập **(b)** CPU quá tải **(c)** Bộ nhớ heap bị tràn **(d)** Trình biên dịch bị lỗi
9. Magic byte problem được giải quyết hiệu quả nhất bằng: **(a)** Random mutation với số lượng iteration lớn hơn **(b)** Symbolic/concolic execution kết hợp SMT solving **(c)** Tăng kích thước corpus ban đầu **(d)** Giảm timeout của fuzzer
10. CWE-416 tương ứng với loại lỗi nào? **(a)** SQL Injection **(b)** Use-after-free **(c)** Integer overflow **(d)** Race condition

11. Thuật toán gán nhãn CTL tính tập $Sat(\varphi)$ theo chiến lược nào? **(a)** Top-down từ công thức tổng thể **(b)** Bottom-up quy nạp trên cấu trúc cú pháp **(c)** Ngẫu nhiên **(d)** Chỉ áp dụng cho công thức LTL
12. AddressSanitizer phát hiện use-after-free bằng cách: **(a)** Kiểm tra kiểu dữ liệu tĩnh **(b)** Đánh dấu vùng nhớ đã free trong shadow memory là ”không hợp lệ” **(c)** Symbolic execution **(d)** SAT solving
13. Trong quy trình threat modeling STRIDE, ”Repudiation” liên quan tới việc thiếu cơ chế nào? **(a)** Mã hóa **(b)** Audit log không thể sửa đổi **(c)** Rate limiting **(d)** Load balancing
14. k-induction cho phép chứng minh: **(a)** Chỉ đúng trong k bước **(b)** Đúng mãi mãi (unbounded) nếu bước quy nạp thành công **(c)** Chỉ áp dụng cho BDD **(d)** Không liên quan tới SAT/SMT
15. Differential fuzzing đặc biệt hữu ích khi: **(a)** Có oracle chính xác biết trước **(b)** Có nhiều cài đặt độc lập của cùng một chức năng để đối chiếu **(c)** Mã nguồn chỉ có một phiên bản duy nhất **(d)** Không cần thực thi chương trình

Đáp án Phần I (tổng hợp)

1-b, 2-b, 3-b, 4-c, 5-b, 6-b, 7-b, 8-a, 9-b, 10-b, 11-b, 12-b, 13-b, 14-b, 15-b.

Phần II – Tự luận tổng hợp

Câu 1. Với mô hình bộ nhớ tiến trình đã học (Chương 2) và Kripke Structure (Chương 1), hãy mô tả cách một công cụ BMC (Chương 3) mô hình hóa *đồng thời* cả không gian trạng thái chương trình và bộ nhớ vật lý để kiểm chứng một thuộc tính an toàn bộ nhớ cụ thể.

Đáp án gợi ý: BMC mô hình chương trình như một Kripke Structure mở rộng, trong đó mỗi trạng thái bao gồm cả giá trị biến chương trình *và* nội dung bộ nhớ (dùng Theory of Arrays, mục Chương 3). Quan hệ chuyển trạng thái R tương ứng với ngữ nghĩa thực thi từng câu lệnh; việc trải phẳng (unroll) k bước tương đương với việc ”duyệt” k bước đầu tiên của Kripke Structure đó. Thuộc tính an toàn bộ nhớ (ví dụ ”không truy cập ngoài biên mảng”) được biểu diễn dưới dạng công thức phủ định cần kiểm tra SAT tại mọi trạng thái đã unroll – kết nối trực tiếp lý thuyết Chương 1 với kỹ thuật Chương 3.

Câu 2. So sánh vai trò của ”oracle” trong ba kỹ thuật: kiểm thử chức năng cổ điển (Chương 1), Bounded Model Checking (Chương 3), và fuzzing (Chương 4).

Đáp án gợi ý: Kiểm thử chức năng: oracle là output mong đợi chính xác đã biết trước (do người thiết kế test case xác định). BMC: oracle là công thức logic hình thức (đặc tả) – kết quả SAT/UNSAT là oracle toán học tuyệt đối trong giới hạn k. Fuzzing: oracle yếu nhất – thường chỉ là ”crash hay không crash” hoặc ”vi phạm bất biến do sanitizer phát hiện”, không cần biết trước output đúng là gì.

Câu 3. Một sinh viên đề xuất ”chỉ cần dùng BMC là đủ, không cần fuzzing” cho dự án của mình. Phản biện đề xuất này dựa trên kiến thức đã học.

Đáp án gợi ý: BMC chỉ đảm bảo an toàn trong giới hạn độ sâu k (trừ khi kết hợp k-induction, vốn tốn kém và không phải lúc nào cũng thành công); BMC cũng gặp khó khăn với mã có vòng lặp/đệ quy sâu, con trỏ hàm động, hoặc gọi thư viện ngoài không thể mô hình hóa formaly. Fuzzing bù đắp được các hạn chế này: chạy được trên toàn bộ chương trình thực tế (không cần mô hình hóa formal), tìm được lỗi runtime cụ thể có thể tái hiện ngay, và với sanitizer, phát hiện được nhiều lớp lỗi bộ nhớ tinh vi. Hai kỹ thuật bổ trợ nhau, không thay thế nhau – đúng theo nguyên lý defense in depth xuyên suốt giáo trình.

Câu 4. Giải thích mối liên hệ giữa CWE Top 25 (Chương 1) và việc ưu tiên đầu tư giữa Chương 2 (secure C) và các kỹ thuật kiểm chứng ở Chương 3–4.

Đáp án gợi ý: Như đã phân tích ở mục CWE Top 25 (Chương 1), phần lớn các CWE nguy hiểm nhất (out-of-bounds write/read, use-after-free) là lỗi bộ nhớ đặc trưng của C/C++ – đây chính là động lực để giáo trình dành trọn Chương 2 cho chủ đề này, và Chương 3–4 cung cấp bộ công cụ kiểm chứng (BMC, fuzzing với ASan) trực tiếp nhắm vào chính các lớp lỗi này, tạo thành một chuỗi logic: hiểu vấn đề (Ch.1) $\rightarrow$ viết mã giảm thiểu lỗi (Ch.2) $\rightarrow$ kiểm chứng để bắt phần lỗi còn sót lại bằng hai kỹ thuật bổ trợ (Ch.3–4).

Phần III – Bài tập tổng hợp

Câu 1. Cho file mẫu `vulnerable_sample.c` ở Phụ lục E. Với mỗi hàm trong file, xác định: (a) hàm nào nên được đưa vào static scanning (Chương 2); (b) hàm nào nên được đánh dấu `@verify` cho BMC (Chương 3); (c) hàm nào là mục tiêu tốt nhất cho fuzzing (Chương 4). Giải thích lý do lựa chọn cho mỗi trường hợp.

Câu 2. Thiết kế một pipeline CI/CD tối thiểu (mô tả bằng sơ đồ hoặc danh sách bước) tích hợp cả bốn kỹ thuật đã học (threat modeling, static scanning, BMC, fuzzing) cho một dự án phần mềm C thực tế, chỉ rõ thứ tự thực hiện và lý do sắp xếp theo thứ tự đó (liên hệ khái niệm ”shift-left security” đã học ở Chương 3).

CHƯƠNG D

Hướng dẫn bài tập lớn (Capstone Project)

Bài tập lớn của học phần CSE703093 yêu cầu sinh viên tổng hợp toàn bộ kiến thức bốn chương thành một sản phẩm phần mềm hoàn chỉnh: một công cụ phân tích bảo mật mã C mini, xây dựng dựa trên nền tảng `softsec-toolkit` đã giới thiệu ở Phụ lục A.

D.1 Mục tiêu

Xây dựng công cụ **MiniSecScan** – một pipeline phân tích bảo mật mã nguồn C tích hợp bốn giai đoạn tương ứng bốn chương:

1. **Threat modeling (Chương 1)**: từ một file cấu hình mô tả kiến trúc hệ thống (các thành phần, luồng dữ liệu, trust boundary), tự động sinh danh sách câu hỏi STRIDE cần trả lời và một Kripke Structure biểu diễn vòng đời xử lý request.
2. **Static pattern scanning (Chương 2)**: quét mã C tìm các hàm nguy hiểm và tính điểm rủi ro cho từng phát hiện bằng module `risk`.
3. **Bounded verification (Chương 3)**: với các hàm được đánh dấu ”cần kiểm chứng” (qua comment đặc biệt `/* @verify */`), sinh ràng buộc SMT đơn giản kiểm tra an toàn truy cập mảng trong giới hạn k bước.
4. **Fuzzing tự động (Chương 4)**: sinh harness fuzzing tối thiểu cho các hàm nhận tham số `char*`/`int`, chạy black-box fuzzer trong ngân sách thời gian cố định, báo cáo crash kèm coverage đạt được.

D.2 Yêu cầu kỹ thuật

Bảng D.1: Tiêu chí đánh giá bài tập lớn

Hạng mục	Mô tả	Điểm
Threat modeling module	Sinh báo cáo STRIDE + Kripke Structure (dạng TikZ hoặc DOT) từ file cấu hình đầu vào	2.0
Static scanning module	Tích hợp `pattern_scanner`, mở rộng thêm tối thiểu 5 mẫu lỗi mới, tính điểm rủi ro tổng hợp cho toàn bộ file	2.0
Bounded verification module	Parse được ít nhất cấu trúc `if/for` đơn giản, sinh và giải ràng buộc SMT bằng Z3, báo cáo counterexample nếu có	2.5
Fuzzing module	Sinh harness tự động, chạy tối thiểu 10.000 iteration, báo cáo coverage và crash	2.0
Chất lượng kỹ thuật	Unit test đầy đủ (pytest, coverage $\geq$ 80%), tài liệu README, xử lý lỗi hợp lý	1.0
Báo cáo và bảo vệ	Báo cáo bằng văn bản (5–10 trang) và thuyết trình bảo vệ trước hội đồng	0.5

D.3 Cấu trúc dự án đề xuất

```
minisecscan/
- minisecscan/
  - threat_model.py     # Chuong 1: sinh STRIDE + Kripke tu config
  - static_scan.py       # Chuong 2: mo rong pattern_scanner
  - bounded_verify.py    # Chuong 3: parse + sinh rang buoc SMT
  - fuzz_harness.py      # Chuong 4: sinh harness + chay fuzzer
  - report.py            # tong hop ket qua 4 giai doan thanh 1 bao cao
  - cli.py                # entry point: minisecscan scan <file.c>
- tests/
- examples/
  - vulnerable_sample.c  # file mau chua nhieu loi de test toan bo pipeline
- README.md
- pyproject.toml
```

Listing D.1: Cấu trúc thư mục đề xuất cho MiniSecScan

D.4 Lịch trình đề xuất (8 tuần)

Bảng D.2: Lịch trình thực hiện bài tập lớn

Tuần	Giai đoạn	Công việc
1–2	Thiết kế	Xác định kiến trúc tổng thể, format file cấu hình threat model, viết test case mẫu cho `vulnerable_sample.c`
3	Threat modeling	Cài đặt `threat_model.py`, tích hợp `KripkeStructure` từ Phụ lục A
4–5	Static scanning	Mở rộng `pattern_scanner`, tích hợp `risk.py`, viết báo cáo tổng hợp điểm rủi ro
6	Bounded verification	Cài đặt parser đơn giản, tích hợp `smt_wrapper`, kiểm thử trên các trường hợp đã biết (Listing Chương 3)
7	Fuzzing	Cài đặt sinh harness tự động, tích hợp `blackbox_fuzzer` và `CoverageTracker`
8	Hoàn thiện	Viết báo cáo, chuẩn bị demo, unit test đầy đủ, đảm bảo coverage $\geq 80\%$

D.5 Ví dụ file mã nguồn mẫu cho kiểm thử pipeline

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* @verify */
int access_buffer(int idx) {
    int buf[10] = {0};
    return buf[idx];              /* Chuong 3: can kiem chung bien idx */
}

void copy_input(char *input) {
    char local[16];
    strcpy(local, input);          /* Chuong 2: ham nguy hiem */
}

void double_free_bug(void) {
    char *p = malloc(32);
    free(p);
    free(p);                       /* Chuong 2: double free */
}

int parse_length(const char *data, int declared_len) {
    char buffer[64];
    memcpy(buffer, data, declared_len);  /* Chuong 4: fuzz muc tieu nay */
    return declared_len;
```

```
}

int main(int argc, char **argv) {
    if (argc > 1) copy_input(argv[1]);
    printf("%d\n", access_buffer(5));
    return 0;
}
```

Listing D.2: `examples/vulnerable_sample.c` – file chứa nhiều lớp lỗi để kiểm thử toàn bộ pipeline

Kiểm soát chất lượng

File mẫu trên cố ý chứa đại diện của cả bốn lớp lỗi đã học xuyên suốt giáo trình: truy cập mảng cần kiểm chứng hình thức (Chương 3), hàm nguy hiểm phát hiện được bằng static scanning (Chương 2), lỗi ownership (double free, Chương 2), và một hàm lý tưởng để làm mục tiêu fuzzing vì có tham số độ dài do người dùng kiểm soát trực tiếp điều khiển kích thước `memcpy` (Chương 4) – đúng mẫu hình gốc rễ của Heartbleed đã phân tích ở Chương 1.

D.6 Quy định nộp bài và bảo vệ

- Nộp mã nguồn đầy đủ (bao gồm lịch sử commit Git thể hiện quá trình phát triển từng tuần, không chấp nhận một commit duy nhất).
- Báo cáo văn bản 5–10 trang theo mẫu chuẩn của Khoa, trình bày rõ kiến trúc, các quyết định thiết kế, giới hạn đã biết (known limitations) của công cụ.
- Bảo vệ trực tiếp trước hội đồng 2–3 giảng viên, thời lượng 15 phút trình bày + 10 phút hỏi đáp, yêu cầu demo trực tiếp trên file mẫu do hội đồng cung cấp tại chỗ (không được biết trước).
- Sinh viên cần giải thích được *tại sao* công cụ của mình có thể bỏ sót một số lỗi nhất định (liên hệ trực tiếp tới khái niệm soundness/ completeness đã học ở Chương 3) – đây là một tiêu chí đánh giá quan trọng, không phải một điểm yếu cần che giấu.

CHƯƠNG E

Bảng thuật ngữ Việt – Anh

Phần đầu của phụ lục là bảng đối chiếu thuật ngữ nhanh; phần sau cung cấp định nghĩa ngắn gọn cho các thuật ngữ kỹ thuật cốt lõi, tiện tra cứu độc lập với nội dung chương.

E.1 Bảng đối chiếu nhanh

Thuật ngữ tiếng Việt	Tiếng Anh
An toàn bảo mật	Security
An toàn vận hành	Safety
Độ tin cậy	Reliability
Bảo mật	Confidentiality
Toàn vẹn	Integrity
Sẵn sàng	Availability
Xác thực	Authentication
Phân quyền	Authorization
Không thể chối bỏ	Non-repudiation
Trách nhiệm giải trình	Accountability
Tài sản	Asset
Mối đe dọa	Threat
Lỗ hổng	Vulnerability
Rủi ro	Risk
Cây tấn công	Attack tree
Đặc quyền tối thiểu	Principle of least privilege
Tràn bộ đệm	Buffer overflow
Sử dụng sau khi giải phóng	Use-after-free
Chèn mã	Injection
Điều kiện tranh chấp	Race condition
Kiểm chứng	Verification
Thẩm định	Validation
Kiểm thử tĩnh	Static testing
Kiểm thử động	Dynamic testing
Phân vùng tương đương	Equivalence partitioning
Phân tích giá trị biên	Boundary value analysis
Bảng quyết định	Decision table
Kiểm thử chuyển trạng thái	State transition testing
Cấu trúc Kripke	Kripke structure

Thuật ngữ tiếng Việt	Tiếng Anh
Tính đúng đắn (không bỏ sót)	Soundness
Tính đầy đủ (không báo động giả)	Completeness
Bài toán thỏa mãn Boolean	Boolean Satisfiability (SAT)
Thỏa mãn theo lý thuyết	Satisfiability Modulo Theories (SMT)
Kiểm chứng mô hình giới hạn	Bounded Model Checking (BMC)
Trải phẳng (vòng lặp)	Unrolling
Mô hình bộ nhớ	Memory model
Căn chỉnh bộ nhớ	Memory alignment
Đệm (khoảng trống trong struct)	Padding
Kiểm thử mờ	Fuzzing
Phủ mã nguồn	Code coverage
Theo dõi lan truyền dữ liệu bẩn	Taint tracking
Thực thi ký hiệu	Symbolic execution
Thực thi cụ thể-ký hiệu	Concolic execution
Kiểm thử hộp đen	Black-box testing
Kiểm thử hộp trắng	White-box testing
Hành vi không xác định	Undefined behavior
Trừu tượng hóa	Abstract interpretation
Sơ đồ quyết định nhị phân	Binary Decision Diagram (BDD)
Quy nạp giới hạn	K-induction
Bộ theo dõi thực thi	Execution tracer
Bùng nổ đường thực thi	Path explosion
Bùng nổ không gian trạng thái	State explosion
Byte ma thuật	Magic byte
Kiểm thử mờ có hướng dẫn coverage	Coverage-guided fuzzing
Kiểm thử mờ nhận biết cấu trúc	Structure-aware fuzzing
Kiểm thử mờ vi sai	Differential fuzzing
Thu gọn bộ dữ liệu mẫu	Corpus minimization
Phân loại và loại trùng sự cố	Crash triage / deduplication
Bộ làm sạch địa chỉ	AddressSanitizer
Bộ làm sạch hành vi không xác định	UndefinedBehaviorSanitizer

E.2 Định nghĩa thuật ngữ cốt lõi (tra cứu nhanh)

Asset Bất kỳ thứ gì có giá trị cần bảo vệ trong hệ thống: dữ liệu, mã nguồn, uy tín, dịch vụ, hạ tầng.

Attack surface Tổng hợp mọi điểm mà kẻ tấn công có thể tương tác để đưa dữ liệu không tin cậy vào hệ thống.

Buffer overflow Ghi dữ liệu vượt quá kích thước bộ đệm đã cấp phát, ghi đè lên vùng nhớ liền kề.

CDCL Conflict-Driven Clause Learning – mở rộng của DPLL, học thêm mệnh đề mới khi gặp mâu thuẫn để tránh lặp lại nguyên nhân đó.

Completeness Tính chất của một công cụ phân tích: mọi cảnh báo đưa ra đều là lỗi thật (không có false positive).

CVE Common Vulnerabilities and Exposures – danh mục các lỗ hổng cụ thể đã phát hiện trong phần mềm/phiên bản thực tế.

CWE Common Weakness Enumeration – hệ thống phân loại chuẩn hóa các loại lỗi phần mềm có thể dẫn tới lỗ hổng.

Fuzzing Kỹ thuật kiểm thử động tự động sinh lượng lớn input và quan sát dấu hiệu bất thường (crash, vi phạm bất biến).

Kripke Structure Bộ tứ (S, S_0, R, L) biểu diễn không gian trạng thái của một hệ thống, dùng để kiểm chứng các thuộc tính temporal logic.

Magic byte problem Hiện tượng random fuzzing khó tự sinh đúng chuỗi byte cố định cần thiết để vượt qua kiểm tra định dạng đầu vào.

Ownership Quy ước lập trình xác định thành phần nào chịu trách nhiệm giải phóng một vùng nhớ động đã cấp phát.

Rice's Theorem Định lý chứng minh không tồn tại thuật toán quyết định được cho mọi thuộc tính ngữ nghĩa không tầm thường của chương trình tùy ý.

SAT Boolean Satisfiability Problem – bài toán quyết định có tồn tại phép gán giá trị chân lý thỏa mãn một công thức Boolean cho trước.

Soundness Tính chất của một công cụ phân tích: không bao giờ bỏ sót lỗi thật (không có false negative).

STRIDE Khung phân loại mối đe dọa sáu loại: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege.

Taint tracking Kỹ thuật đánh dấu và theo dõi lan truyền dữ liệu bắt nguồn từ nguồn không tin cậy qua các phép toán trong chương trình.

Undefined Behavior Hành vi mà chuẩn ngôn ngữ không quy định, cho phép trình biên dịch xử lý tùy ý, kể cả loại bỏ các kiểm tra liên quan.

Use-after-free Lỗi tiếp tục sử dụng một con trỏ sau khi vùng nhớ nó trỏ tới đã được giải phóng.

Tài liệu tham khảo

[1] ISTQB. *Certified Tester Foundation Level Syllabus*. International Software Testing Qualifications Board.

[2] MITRE Corporation. *Common Weakness Enumeration (CWE)*. https://cwe.mitre.org.

[3] MITRE Corporation. *Common Vulnerabilities and Exposures (CVE)*. https://cve.mitre.org.

[4] Clarke, E. M., Grumberg, O., Peled, D. *Model Checking*. MIT Press, 1999.

[5] Biere, A., Heule, M., van Maaren, H., Walsh, T. (eds.) *Handbook of Satisfiability*. IOS Press, 2009.

[6] Kroening, D., Strichman, O. *Decision Procedures: An Algorithmic Point of View*. Springer, 2016.

[7] Clarke, E., Kroening, D., Lerda, F. "A Tool for Checking ANSI-C Programs" (CBMC). *TACAS 2004*.

[8] Seacord, R. C. *Secure Coding in C and C++*. 2nd ed., Addison-Wesley, 2013 (SEI CERT C Coding Standard).

[9] OWASP Foundation. *OWASP Top 10*. https://owasp.org.

[10] Microsoft Corporation. *The STRIDE Threat Model*. Microsoft Security Development Lifecycle documentation.

[11] Zalewski, M. *American Fuzzy Lop (AFL) – Technical Whitepaper*. https://lcamtuf.coredump.cx/afl/technical_details.txt.

[12] Cadar, C., Dunbar, D., Engler, D. "KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs". *OSDI 2008*.

[13] Godefroid, P., Levin, M. Y., Molnar, D. "SAGE: Whitebox Fuzzing for Security Testing". *Communications of the ACM*, 2012.

[14] Stephens, N. et al. "Driller: Augmenting Fuzzing Through Selective Symbolic Execution". *NDSS 2016*.

[15] Durumeric, Z. et al. "The Matter of Heartbleed". *ACM Internet Measurement Conference (IMC)*, 2014.

[16] ISO/IEC 9899. *Programming Languages – C*. International Organization for Standardization.

[17] RTCA. *DO-178C: Software Considerations in Airborne Systems and Equipment Certification*.

[18] ISO 26262. *Road Vehicles – Functional Safety*. International Organization for Standardization.

[19] Vũ Quang Dũng. *Đề cương chi tiết học phần CSE703093 – An toàn phần mềm*. Trường Đại học Phenikaa, Khoa Hệ thống thông tin.

[20] Vũ Quang Dũng. *Bài giảng CSE703093 – An toàn phần mềm (Chương 1–4)*. Trường Đại học Phenikaa.